

## **II JPA and Hibernate**

## 7. Why JPA and Hibernate matter

Although JDBC does a very good job of exposing a common API that hides the database vendor-specific communication protocol, it suffers from the following shortcomings:

- The API is undoubtedly verbose, even for trivial tasks.
- Batching is not transparent from the data access layer perspective, requiring a specific API than its non-batched statement counterpart.
- Lack of built-in support for explicit locking and optimistic concurrency control.
- For local transactions, the data access is tangled with transaction management semantics.
- Fetching joined relations requires additional processing to transform the `ResultSet` into Domain Models or DTO (Data Transfer Object) graphs.

Although the primary goal of an ORM (Object-Relational Mapping) tool is to automatically translate object state transitions into SQL statements, this chapter aims to demonstrate that Hibernate can address all the aforementioned JDBC shortcomings.

### Java Persistence history

The EJB 1.1 release offered a higher-level persistence abstraction through scalable enterprise components, known as Entity Beans. Although the design looked good on paper, in reality, the heavyweight RMI-based implementation proved to be disastrous from a performance perspective. Neither the EJB 2.0 support for *local interfaces* could revive the Entity Beans popularity, and, due to high-complexity and vendor-specific implementation details, most projects chose JDBC instead.

Hibernate was born out of all the frustration of using the Entity Bean developing model. As an open-source project, Hibernate managed to gain a lot of popularity, and so it soon became the *de facto* Java persistence framework.

In response to all the criticism associated with Entity Bean persistence, the Java Community Process advanced a lightweight POJO-based approach, and the JDO specification was born. Although JDO is data source agnostic, being capable of operating with both relational databases as well as NoSQL or even flat files, it never managed to hit mainstream popularity. For this reason, the Java Community Process decided that EJB3 would be based on a new specification, inspired by Hibernate and TopLink, and JPA (Java Persistence API) became the standard Java persistence technology.

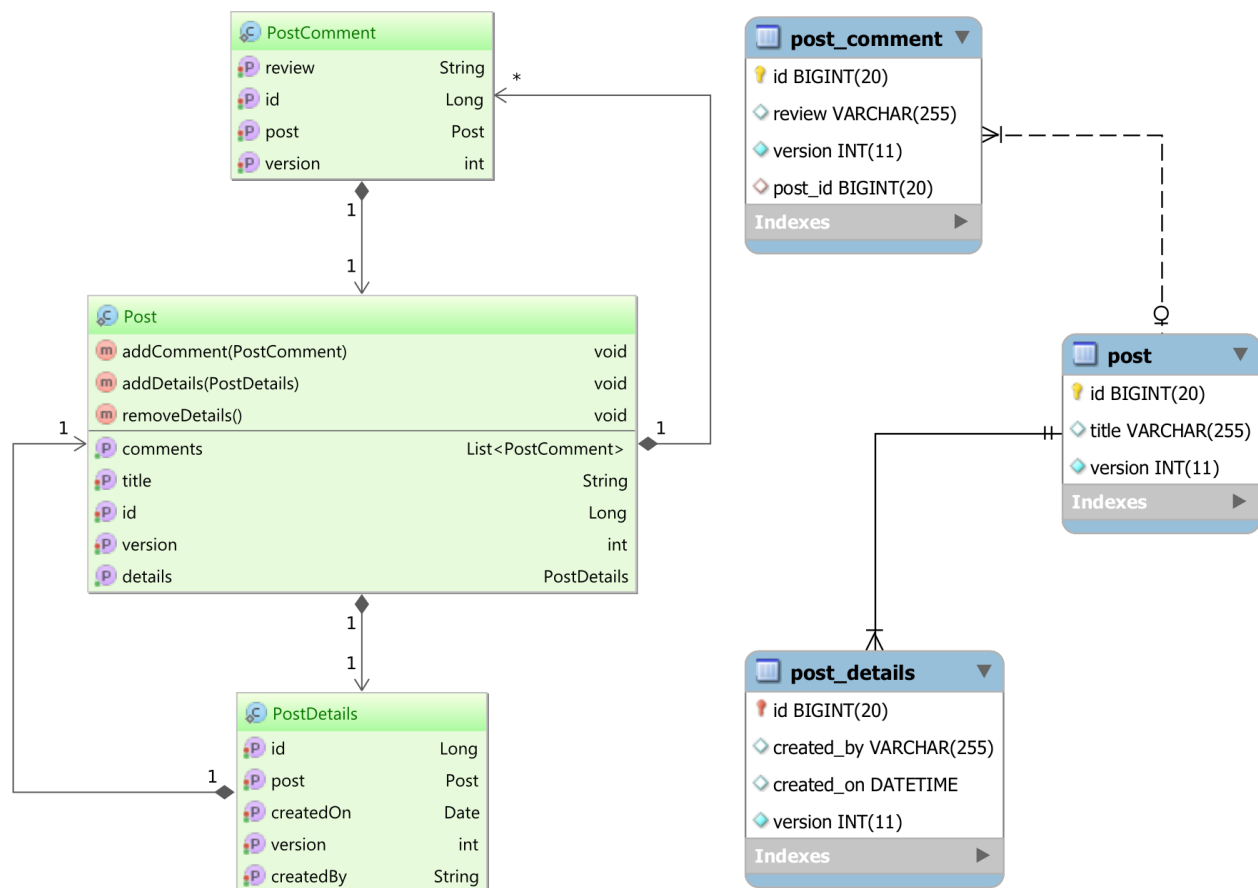
The morale of this is that persistence is a very complex topic, and it demands a great deal of knowledge of both the database and the data access usage patterns.

## 7.1 The impedance mismatch

When a relational database is manipulated through an object-oriented program, the two different data representations start conflicting.

In a relational database, data is stored in tables, and the relational algebra defines how data associations are formed. On the other hand, an object-oriented programming (OOP) language allows objects to have both state and behavior, and bidirectional associations are permitted.

The burden of converging these two distinct approaches has generated much tension, and it has been haunting enterprise systems for a very long time.



**Figure 7.1: Object/Relational Mapping**

The above diagram portrays the two different schemas that the data access layer needs to correlate. While the database schema is driven by the SQL standard specification, the Domain Model comes with an object-oriented schema representation as well.

The Domain Model encapsulates the business logic specifications and captures both data structures and the behavior that governs business requirements. OOP facilitates Domain Modeling, and many modern enterprise systems are implemented on top of an object-oriented programming language.

Because the underlying data resides in a relational database, the Domain Model must be adapted to the database schema and the SQL-driven communication protocol. The ORM design pattern helps to bridge these two different data representations and close the technological gap between them. Every database row is associated with a Domain Model object (Entity in JPA terminology), and so the ORM tool can translate the entity state transitions into DML statements.

From an application development point of view, this is very convenient since it is much easier to manipulate Domain Model relationships rather than visualizing the business logic through its underlying SQL statements.

## 7.2 JPA vs. Hibernate

JPA is only a specification. It describes the interfaces that the client operates with and the standard object-relational mapping metadata (Java annotations or XML descriptors). Beyond the API definition, JPA also explains (although not exhaustively) how these specifications are ought to be implemented by the JPA providers. JPA evolves with the Java EE platform itself (Java EE 6 featuring JPA 2.0 and Java EE 7 introducing JPA 2.1).

Hibernate was already a full-featured Java ORM implementation by the time the JPA specification was released for the first time. Although it implements the JPA specification, Hibernate retains its native API for both backward compatibility and to accommodate non-standard features.

Even if it is best to adhere to the JPA standard, in reality, many JPA providers offer additional features targeting a high-performance data access layer requirements. For this purpose, Hibernate comes with the following non-JPA compliant features:

- extended identifier generators (hi/lo, pooled, pooled-lo)
- transparent prepared statement batching
- customizable CRUD (@SQLInsert, @SQLUpdate, @SQLDelete) statements
- static/dynamic entity/collection filters (e.g. @FilterDef, @Filter, @Where)
- mapping attributes to SQL fragments (e.g. @Formula)
- immutable entities (e.g. @Immutable)
- more flush modes (e.g. FlushMode.MANUAL, FlushMode.ALWAYS)
- querying the second-level cache by the natural key of a given entity
- entity-level cache concurrency strategies  
(e.g. Cache(usage = CacheConcurrencyStrategy.READ\_WRITE))
- versioned bulk updates through HQL
- exclude fields from optimistic locking check (e.g. @OptimisticLock(excluded = true))
- versionless optimistic locking (e.g. OptimisticLockType.ALL, OptimisticLockType.DIRTY)
- support for skipping (without waiting) pessimistic lock requests
- support for Java 8 Date and Time and stream()
- support for multitenancy



If JPA is the interface, Hibernate is one implementation and implementation details always matter from a performance perspective.

The JPA implementation details leak and ignoring them might hinder application performance or even lead to data inconsistency issues. As an example, the following JPA attributes have a peculiar behavior, which can surprise someone who is familiar with the JPA specification only:

- The `FlushModeType.AUTO`<sup>1</sup> does not trigger a flush for native SQL queries like it does for JPQL or Criteria API.
- The `FetchType.EAGER`<sup>2</sup> might choose a SQL join or a secondary select whether the entity is fetched directly from the `EntityManager` or through a JPQL (Java Persistence Query Language) or a Criteria API query.

That is why this book is focused on how Hibernate manages to implement both the JPA specification and its non-standard native features (that are relevant from an efficiency perspective).

## Portability concerns

Like other non-functional requirements, portability is a feature, and there is still a widespread fear of embracing database-specific or framework-specific features. In reality, it is more common to encounter enterprise applications facing data access performance issues than having to migrate from one technology to the other (be it a relational database or a JPA provider).

The lowest common denominator of many RDBMS is a superset of the SQL-92 standard (although not entirely supported either). SQL-99 supports Common Table Expressions, but MySQL 5.7 does not. Although SQL-2003 defines the `MERGE` operator, PostgreSQL 9.5 favored the `UPSERT` operation instead. By adhering to a SQL-92 syntax, one could achieve a higher degree of database portability, but the price of giving up database-specific features can take a toll on application performance. Portability can be addressed either by subtracting non-common features or through specialization. By offering different implementations, for each supported database system (like the jOOQ framework does), portability can still be achieved.

The same argument is valid for JPA providers too. By layering the application, it is already much easier to swap JPA providers, if there is even a compelling reason for switching one mature JPA implementation with another.

<sup>1</sup><https://docs.oracle.com/javaee/7/api/javax/persistence/FlushModeType.html>

<sup>2</sup><https://docs.oracle.com/javaee/7/api/javax/persistence/FetchType.html#EAGER>

## 7.3 Schema ownership

Because of data representation duality, there has been a rivalry between taking ownership of the underlying schema. Although theoretically, both the database and the Domain Model could drive the schema evolution, for practical reasons, the schema belongs to the database.

An enterprise system might be too large to fit into a single application, so it is not uncommon to split in into multiple subsystems, each one serving a specific goal. As an example, there can be front-end web applications, integration web services, email schedulers, full-text search engines and back-end batch processors that need to load data into the system. All these subsystems need to use the underlying database, whether it is for displaying content to the users or dumping data into the system.

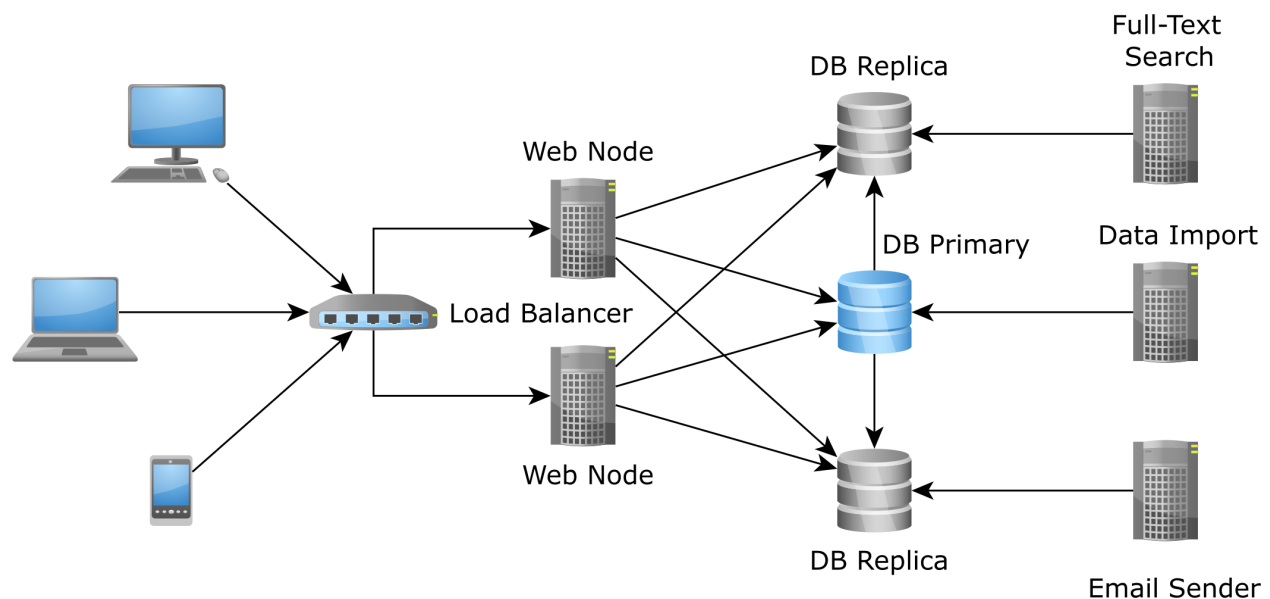


Figure 7.2: Database-centric integration

Although it might not fit any enterprise system, having the database as a central integration point can still be a choice for many reasonable size enterprise systems.

The relational database concurrency models offer strong consistency guarantees, therefore having a significant advantage to application development. If the integration point does not provide transactional semantics, it will be much more difficult to implement a distributed concurrency control mechanism.

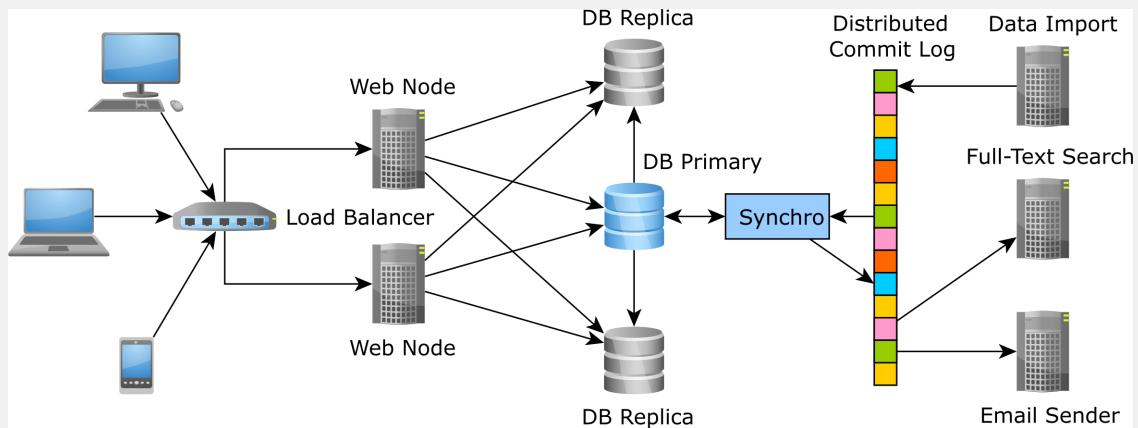
Most database systems already offer support for various replication topologies, which can provide more capacity for accommodating an increase in the incoming request traffic. Even if the demand for more data continues to grow, the hardware is always getting better and better (and cheaper too), and database vendors keep on improving their engines to cope with more data.

For these reasons, having the database as an integration point is still a relevant enterprise system design consideration.

## The distributed commit log

For very large enterprise systems, where data is split among different providers (relational database systems, caches, Hadoop, Spark), it is no longer possible to rely on the relational database to integrate all disparate subsystems.

In this case, [Apache Kafka](http://kafka.apache.org/)<sup>a</sup> offers a fault-tolerant and scalable append-only log structure, which every participating subsystem can read and write concurrently.



**Figure 7.3: Distributed commit log integration**

The commit log becomes the integration point, each distributed node individually traversing it and maintaining client-specific pointers in the sequential log structure. This design resembles a database replication mechanism, and so it offers durability (the log is persisted on disk), write performance (append-only logs do not require random access) and read performance (concurrent reads do not require blocking) as well.

<sup>a</sup><http://kafka.apache.org/>

No matter what architecture style is chosen, there is still need to correlate the transient Domain Model with the underlying persistent data.

The data schema evolves along the enterprise system itself, and so the two different schema representations must remain congruent at all times.

Even if the data access framework can auto-generate the database schema, the schema must be migrated incrementally, and all changes need to be traceable in the VCS (Version Control System) as well. Along with table structure, indexes and triggers, the database schema is, therefore, accompanying the Domain Model source code itself. A tool like [Flywaydb](http://flywaydb.org/)<sup>3</sup> can automate the database schema migration, and the system can be deployed continuously, whether it is a test or a production environment.

<sup>3</sup><http://flywaydb.org/>



The schema ownership goes to the database, and the data access layer must assist the Domain Model to communicate with the underlying data.

## 7.4 Entity state transitions

JPA shifts the developer mindset from SQL statements to entity state transitions. An entity can be in one of the following states:

**Table 7.1: JPA entity states**

| State                | Description                                                                                                                                                                                                                                                       |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| New (Transient)      | A newly created entity which is not mapped to any database row is considered to be in the New or Transient state. Once it becomes managed, the Persistence Context issues an insert statement at flush time.                                                      |
| Managed (Persistent) | A Persistent entity is associated with a database row, and it is being managed by the currently running Persistence Context. State changes are detected by the <i>dirty checking</i> mechanism and propagated to the database as update statements at flush time. |
| Detached             | Once the currently running Persistence Context is closed, all the previously managed entities become detached. Successive changes are no longer tracked, and no automatic database synchronization is going to happen.                                            |
| Removed              | A removed entity is only scheduled for deletion, and the actual database delete statement is executed during Persistence Context flushing.                                                                                                                        |

The Persistence Context captures entity state changes, and, during flushing, it translates them into SQL statements. The JPA [EntityManager](http://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html#persist-java.lang.Object-)<sup>4</sup> and the Hibernate [Session](https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/Session.html)<sup>5</sup> (which includes additional methods for moving an entity from one state to the other) interfaces are gateways towards the underlying Persistence Context, and they define all the entity state transition operations.

<sup>4</sup><http://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html#persist-java.lang.Object->

<sup>5</sup><https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/Session.html>

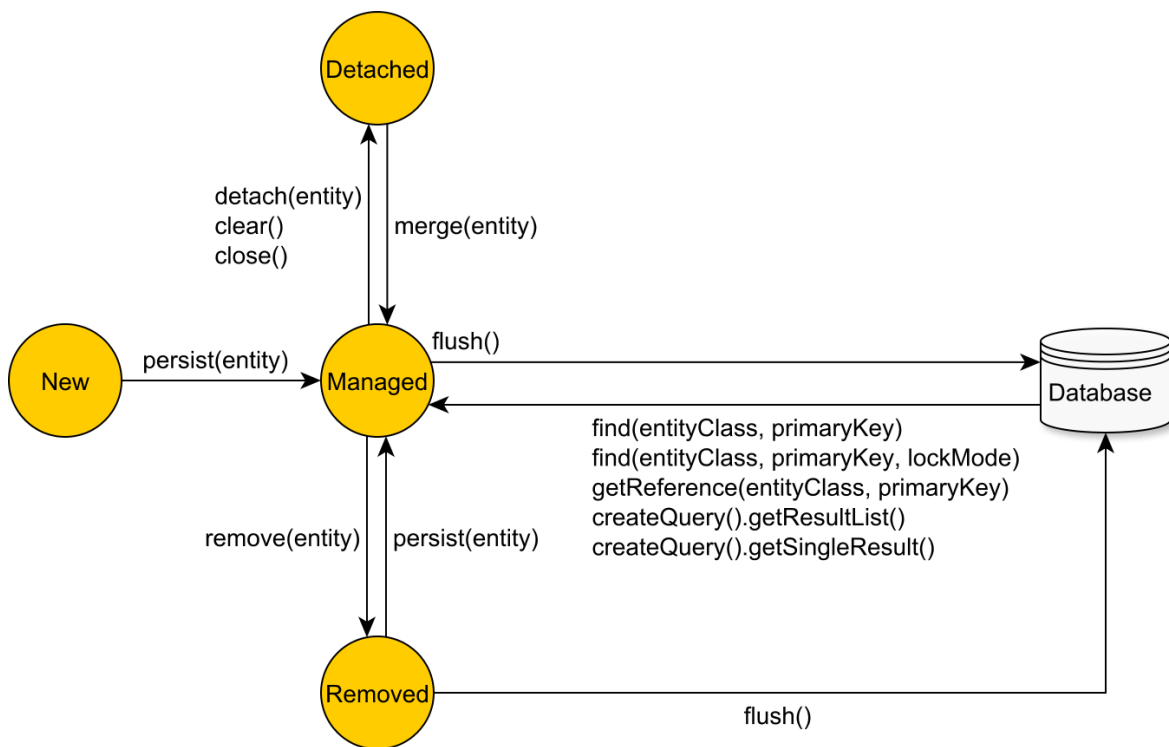


Figure 7.4: JPA entity state transitions

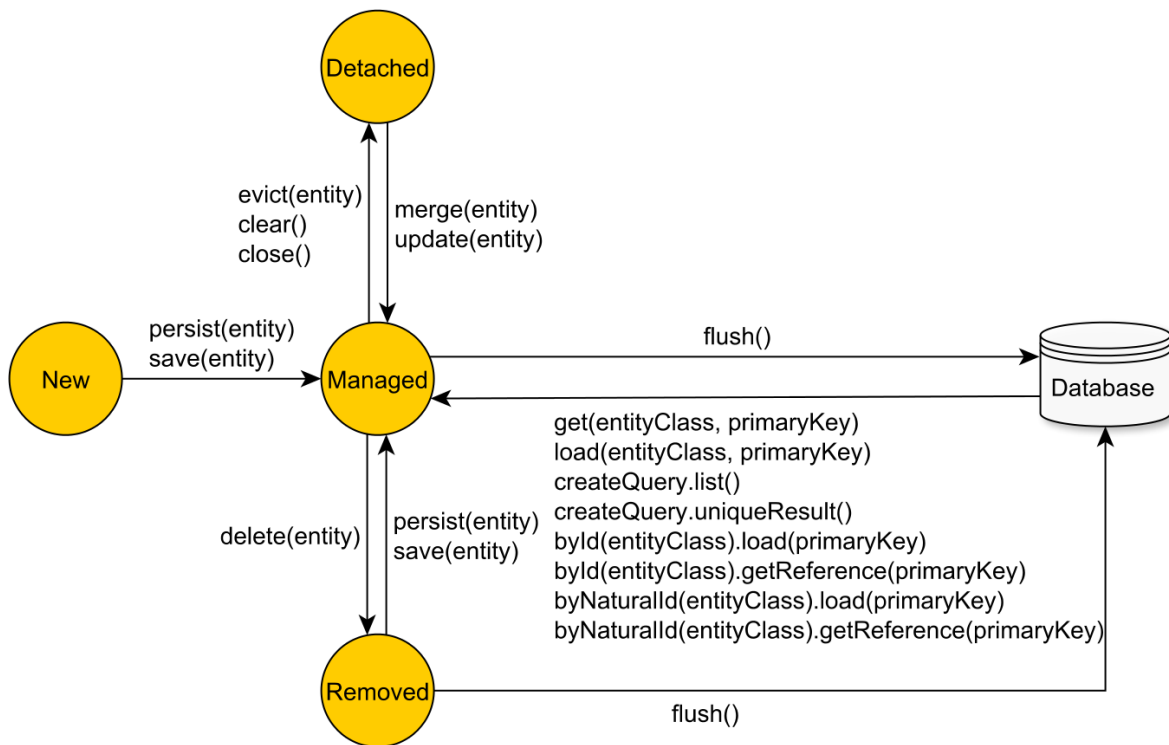


Figure 7.5: Hibernate entity state transitions

## 7.5 Write-based optimizations

### SQL injection prevention

By managing the SQL statement generation, the JPA tool can assist in minimizing the risk of SQL injection attacks. The less the chance of manipulating SQL `String` statements, the safer the application can get. The risk is not completely eliminated because the application developer can still recur to concatenating SQL or JPQL fragments, so rigor is advised.



Hibernate uses `PreparedStatement(s)` exclusively, so not only it protects against SQL injection, but the data access layer can better take advantage of server-side and client-side statement caching as well.

### Auto-generated DML statements

The enterprise system database schema evolves with time, and the data access layer must mirror all these modifications as well.

Because the JPA provider auto-generates insert and update statements, the data access layer can easily accommodate database table structure modifications. By updating the entity model schema, Hibernate can automatically adjust the modifying statements accordingly.

This applies to changing database column types as well. If the database schema needs to migrate a postal code from an `INT` database type to a `VARCHAR(6)`, the data access layer will need only to change the associated Domain Model attribute type from an `Integer` to a `String`, and all statements are going to be automatically updated. Hibernate defines a highly customizable JDBC-to-database type mapping system, and the application developer can override a default type association, or even add support for new database types (that are not currently supported by Hibernate).

The entity fetching process is automatically managed by the JPA implementation, which auto-generates the select statements of the associated database tables. This way, JPA can free the application developer from maintaining entity selection queries as well.

Hibernate allows customizing all the CRUD statements, in which case the application developer is responsible for maintaining the associated DML statements.

Although it takes care of the entity selection process, most enterprise systems need to take advantage of the underlying database querying capabilities. For this reason, whenever the database schema changes, all the native SQL queries need to be updated manually (according to their associated business logic requirements).

## Write-behind cache

The Persistence Context acts as a transactional write-behind cache, deferring entity state flushing up until the last possible moment.

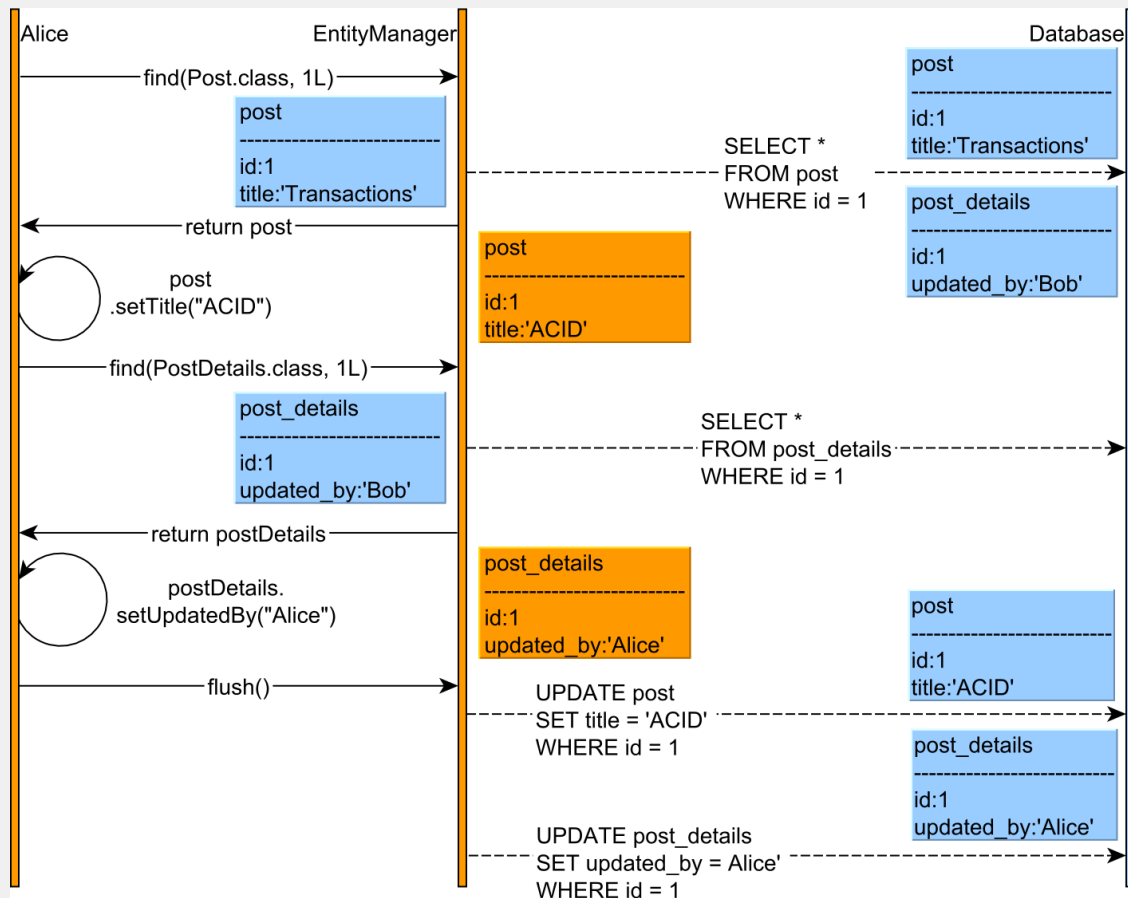


Figure 7.6: Persistence Context

Because every modifying DML statement requires locking (to prevent dirty writes), the write behind cache can reduce the database lock acquisition interval, therefore increasing concurrency.



However, caches introduce consistency challenges, and the Persistence Context requires a flush prior to executing any JPQL or native SQL query, as otherwise it might break the read-your-own-write consistency guarantee.

As detailed in the following chapters, Hibernate does not automatically flush pending changes when a native query is about to be executed, and the application developer must explicitly instruct what database tables are needed to be synchronized.

## Transparent statement batching

Since all changes are being flushed at once, Hibernate may benefit from batching JDBC statements. Batch updates can be enabled transparently, even after the data access logic has been implemented. Most often, performance tuning is postponed until the system is already running in production, and switching to batching statements should not require a considerable development effort.



With just one configuration, Hibernate can execute all prepared statements in batches.

## Application-level concurrency control

As previously explained, no database isolation level can protect against losing updates when executing a multi-request long conversation. JPA supports both optimistic and pessimistic locking.

The JPA optimistic locking mechanism allows preventing lost updates because it imposes a *happens-before* event ordering. However, in multi-request conversations, optimistic locking requires maintaining old entity snapshots, and JPA makes it possible through *Extended Persistence Contexts* or detached entities.



A Java EE application server can preserve a given Persistence Context across several web requests, therefore providing application-level repeatable reads. However, this strategy is not free since the application developer must make sure the Persistence Context is not bloated with too many entities, which, apart from consuming memory, it can also affect the performance of the Hibernate default dirty checking mechanism.

Even when not using Java EE, the same goal can be achieved using detached entities, which provide fine-grained control over the amount of data needed to be preserved from one web request to the other. JPA allows merging detached entities, which rebecome managed and automatically synchronized with the underlying database system.

JPA also supports a pessimistic locking query abstraction, which comes in handy when using lower-level transaction isolation modes.



Hibernate has a native pessimistic locking API, which brings support for timing out lock acquisition requests or skipping already acquired locks.

## 7.6 Read-based optimizations

Following the SQL standard, the JDBC `ResultSet` is a tabular representation of the underlying fetched data. The Domain Model being constructed as an entity graph, the data access layer must transform the flat `ResultSet` into a hierarchical structure.

Although the goal of the ORM tool is to reduce the gap between the object-oriented Domain Model and its relational counterpart, it is very important to remember that the source of data is not an in-memory repository, and the fetching behavior influences the overall data access efficiency.



The database cannot be abstracted out of this context, and pretending that entities can be manipulated just like any other plain objects is very detrimental to application performance. When it comes to reading data, the impedance mismatch becomes even more apparent, and, for performance reasons, it is mandatory to keep in mind the SQL statements associated with every fetching operation.

In the following example, the *posts* records are fetched along with all their associated *comments*. Using JDBC, this task can be accomplished using the following code snippet:

```
doInJDBC(connection -> {
    try (PreparedStatement statement = connection.prepareStatement(
        "SELECT * " +
        "FROM post AS p " +
        "JOIN post_comment AS pc ON p.id = pc.post_id " +
        "WHERE " +
        "    p.id BETWEEN ? AND ? + 1"
    )) {
        statement.setInt(1, id);
        statement.setInt(2, id);
        try (ResultSet resultSet = statement.executeQuery()) {
            List<Post> posts = toPosts(resultSet);
            assertEquals(expectedCount, posts.size());
        }
    } catch (SQLException e) {
        throw new DataAccessException(e);
    }
});
```

When joining *many-to-one* or *one-to-one* associations, each `ResultSet` record corresponds to a pair of entities, so both the parent and the child can be resolved in each iteration. For *one-to-many* or *many-to-many* relationships, because of how the SQL join works, the `ResultSet` contains a duplicated parent record for each associated child.

Constructing the hierarchical entity structure requires manual `ResultSet` transformation, and, to resolve duplicates, the parent entity references are stored in a `Map` structure.

```
List<Post> toPosts(ResultSet resultSet) throws SQLException {
    Map<Long, Post> postMap = new LinkedHashMap<>();
    while (resultSet.next()) {
        Long postId = resultSet.getLong(1);
        Post post = postMap.get(postId);
        if (post == null) {
            post = new Post(postId);
            postMap.put(postId, post);
            post.setTitle(resultSet.getString(2));
            post.setVersion(resultSet.getInt(3));
        }
        PostComment comment = new PostComment();
        comment.setId(resultSet.getLong(4));
        comment.setReview(resultSet.getString(5));
        comment.setVersion(resultSet.getInt(6));
        post.addComment(comment);
    }
    return new ArrayList<>(postMap.values());
}
```

The JDBC 4.2 `PreparedStatement` supports only positional parameters, and the first ordinal starts from 1. JPA allows named parameters as well, which are especially useful when a parameter needs to be referenced multiple times, so the previous example can be rewritten as follows:

```
doInJPA(entityManager -> {
    List<Post> posts = entityManager.createQuery(
        "select distinct p " +
        "from Post p " +
        "join fetch p.comments " +
        "where " +
        "    p.id BETWEEN :id AND :id + 1", Post.class)
    .setParameter("id", id)
    .getResultList();
    assertEquals(expectedCount, posts.size());
});
```

In both examples, the object–relation transformation takes place either implicitly or explicitly. In the JDBC use case, the associations must be manually resolved, while JPA does it automatically (based on the entity schema).

### The fetching responsibility

Besides mapping database columns to entity attributes, the entity associations can also be represented in terms of object relationships. More, the fetching behavior can be hard-wired to the entity schema itself, which is most often a terrible thing to do.

Fetching multiple one-to-many or many-to-many associations is even more problematic because they might require a Cartesian Product, therefore affecting performance. Controlling the hard-wired schema fetching policy is cumbersome as it prevents overriding an eager retrieval with a lazy loading mechanism.



Each business use case has different data access requirements, and one policy cannot anticipate all possible use cases, so the **fetching strategy should always be set up on a query basis**.

### Prefer projections for read-only views

Although it is very convenient to fetch entities along with all their associated relationships, it is better to take into consideration the performance impact as well. As previously explained, fetching too much data is not suitable because it increases the transaction response time.

In reality, not all use cases require loading entities, and not all read operations need to be served by the same fetching mechanism. Sometimes a custom projection (selecting only a few columns from an entity) is much more suitable, and the data access logic can even take advantage of database-specific SQL constructs that might not be supported by the JPA query abstraction.



As a rule of thumb, fetching entities is suitable when the logical transaction requires modifying them, even if that only happens in a successive web request. With this in mind, it is much easier to reason on which fetching mechanism to employ for a given business logic use case.

## The second-level cache

If the Persistence Context acts as a transactional write-behind cache, its lifetime will be bound to that of a logical transaction. For this reason, the Persistence Context is also known as the first-level cache, and so it cannot be shared by multiple concurrent transactions.

On the other hand, the second-level cache is associated with an `EntityManagerFactory`, and all Persistence Contexts have access to it. The second-level cache can store entities as well as entity associations (one-to-many and many-to-many relationships) and even entity query results. Because JPA does not make it mandatory, each provider takes a different approach to caching (as opposed to EclipseLink, by default, Hibernate disables the second-level cache).

Most often, caching is a trade-off between consistency and performance. Because the cache becomes another source of truth, inconsistencies might occur, and they can be prevented only when all database modifications happen through a single `EntityManagerFactory` or a synchronized distributed caching solution. In reality, this is not practical since the application might be clustered on multiple nodes (each one with its own `EntityManagerFactory`) and the database might be accessed by multiple applications.



Although the second-level cache can mitigate the entity fetching performance issues, it requires a distributed caching implementation, which might not elude the network-ing penalties anyway.

## 7.7 Wrap-up

Bridging two highly-specific technologies is always a difficult problem to solve. When the enterprise system is built on top of an object-oriented language, the object-relational impedance mismatch becomes inevitable. The ORM pattern aims to close this gap although it cannot completely abstract it out.

In the end, all the communication flows through JDBC and every execution happens in the database engine itself. A high-performance enterprise application must resonate with the underlying database system, and the ORM tool must not disrupt this relationship.

Just like the problem it tries to solve, Hibernate is a very complex framework with many subtleties that require a thorough knowledge of both database systems, JDBC, and the framework itself. This chapter is only a summary, meant to present JPA and Hibernate into a different perspective that prepares the reader for high-performance object-relational mapping. There is no need to worry if some topics are not entirely clear because the upcoming chapters analyze all these concepts in greater detail.

## 8. Connection Management and Monitoring

As previously explained in the JDBC Connection Management chapter, for performance reasons, database connections are better off reused. Because JPA providers generate SQL statements on behalf of users, it is very important to monitor this process and acknowledge its outcome. This chapter explains the Hibernate connection provider mechanism and ways to monitor statement execution.

### 8.1 JPA connection management

Like the whole Java EE suite, the JPA 1.0 specification was very much tied to enterprise application servers. In a Java EE container, all database connections are managed by the application server which provides connection pooling, monitoring, and JTA capabilities.

Once configured, the Java EE `DataSource` can be located through JNDI. In the `persistence.xml` configuration file, the application developer must supply the JNDI name of the associated JTA or `RESOURCE_LOCAL DataSource`. The `transaction-type` attribute must also match the data source transaction capabilities.

```
<persistence-unit name="persistenceUnit" transaction-type="JTA">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <jta-data-source>java:global/jdbc/flexypool</jta-data-source>
</persistence-unit>
```

A `RESOURCE_LOCAL` transaction must use a non-jta-data-source `DataSource`.

```
<persistence-unit name="persistenceUnit" transaction-type="RESOURCE_LOCAL">
  <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
  <non-jta-data-source>java:/comp/env/jdbc/hsqldb</non-jta-data-source>
</persistence-unit>
```

While for a Java EE application it is perfectly fine to rely on the application server for providing a full-featured `DataSource` reference, stand-alone applications are usually configured using dependency injection rather than JNDI.

From the JPA implementation perspective, the `DataSource` can be either configured externally or by the JPA provider itself. Most often, configuring an external `DataSource` is still the preferred alternative as it gives more flexibility in decorating the connection providing mechanism (e.g. logging, monitoring).

JPA providers can fetch connections through the underlying JDBC Driver since JPA 2.0 has standardized the database connection configuration properties:

**Table 8.1: JPA connection properties**

| Property                                     | Description                                                            |
|----------------------------------------------|------------------------------------------------------------------------|
| <code>javax.persistence.jdbc.driver</code>   | Driver full class name (e.g. <code>org.hsqldb.jdbc.JDBCDriver</code> ) |
| <code>javax.persistence.jdbc.url</code>      | Driver Url (e.g. <code>jdbc:hsqldb:mem:test</code> )                   |
| <code>javax.persistence.jdbc.user</code>     | Database user's name                                                   |
| <code>javax.persistence.jdbc.password</code> | Database user's password                                               |

Unfortunately, these properties alone are not sufficient because most enterprise applications need connection pooling and monitoring capabilities anyway. For this reason, JPA connection management is still an implementation-specific topic, and the upcoming sections dive into the connection provider mechanism employed by Hibernate.

## 8.2 Hibernate connection providers

Hibernate needs to operate both in Java EE and stand-alone environments, and the database connectivity configuration can be done either declaratively or programmatically. To accommodate JDBC Driver connections as well as RESOURCE\_LOCAL and JTA DataSource configurations, Hibernate defines its own connection factory abstraction, represented by the `org.hibernate.engine.jdbc.connections.spi.ConnectionProvider` interface:

```
public interface ConnectionProvider extends Service, Wrapped {  
  
    public Connection getConnection() throws SQLException;  
  
    public void closeConnection(Connection connection) throws SQLException;  
  
    public boolean supportsAggressiveRelease();  
}
```

Because the connection provider might influence transaction response time, each provider is analyzed from a high-performance OLTP system perspective.

### 8.2.1 DriverManagerConnectionProvider

Hibernate picks this provider when being given the aforementioned JPA 2.0 connection properties or the Hibernate-specific configuration counterpart:

- `hibernate.connection.driver_class`
- `hibernate.connection.url`
- `hibernate.connection.username`
- `hibernate.connection.password`.



Although it fetches database connections through the underlying `DriverManager`, this provider tries to avoid the connection acquisition overhead by using a trivial pooling implementation. The Hibernate documentation does not recommend using the `DriverManagerConnectionProvider` in a production setup.

### 8.2.2 C3P0ConnectionProvider

[C3p0](http://www.mchange.com/projects/c3p0/)<sup>1</sup> is a mature connection pooling solution that has proven itself in many production environments, and, using the underlying JDBC connection properties, Hibernate can replace the built-in connection pool with a `c3p0 DataSource`. To activate this provider, the application developer must supply at least one configuration property starting with the `hibernate.c3p0` prefix:

```
<property name="hibernate.c3p0.max_size" value="5"/>
```

`C3p0` (released in 2001) and [Apache DBCP](https://commons.apache.org/proper/commons-dbcp/)<sup>2</sup> (released in 2002) are the oldest and the most deployed stand-alone Java connection pooling solutions. Later in 2010, [BoneCP](https://github.com/wwadge/bonecp)<sup>3</sup> emerged as a high-performance alternative for `c3p0` and `Apache DBCP`.

Nowadays, the `BoneCP` GitHub page says it is been deprecated in favor of [HikariCP](https://github.com/brettwooldridge/HikariCP)<sup>4</sup>.

---

<sup>1</sup><http://www.mchange.com/projects/c3p0/>

<sup>2</sup><https://commons.apache.org/proper/commons-dbcp/>

<sup>3</sup><https://github.com/wwadge/bonecp>

<sup>4</sup><https://github.com/brettwooldridge/HikariCP>

As of writing, the most attractive Java connection pools are HikariCP, [Vibur DBCP<sup>5</sup>](#), and Apache DBCP2. HikariCP and Vibur DBCP offer built-in Hibernate connection providers.

### 8.2.3 HikariCPConnectionProvider

Hibernate 5 supports HikariCP (one of the fastest connection pools) via the following dependency:

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-hikaricp</artifactId>
  <version>${hibernate.version}</version>
</dependency>
```

By specifying the `hibernate.connection.provider_class` property, the application developer can override the default connection provider mechanism:

```
<property name="hibernate.connection.provider_class"
  value="org.hibernate.hikaricp.internal.HikariCPConnectionProvider"/>
```

Unlike `DriverManagerConnectionProvider` or `C3P0ConnectionProvider`, HikariCP requires [specific configuration properties<sup>6</sup>](#):

**Table 8.2: HikariCP connection properties**

| Property                                          | Description              |
|---------------------------------------------------|--------------------------|
| <code>hibernate.hikari.dataSourceClassName</code> | Driver full class name   |
| <code>hibernate.hikari.dataSource.url</code>      | Driver Url               |
| <code>hibernate.hikari.dataSource.user</code>     | Database user's name     |
| <code>hibernate.hikari.dataSource.password</code> | Database user's password |
| <code>hibernate.hikari.maximumPoolSize</code>     | Maximum pool size        |

---

<sup>5</sup><https://github.com/vibur/vibur-dbcp>

<sup>6</sup><https://github.com/brettwooldridge/HikariCP>

## 8.2.4 DataSourceConnectionProvider

This provider is chosen when the JPA configuration file defines a `non-jta-data-source` or a `jta-data-source` element, or when supplying a `hibernate.connection.datasource` configuration property.



Unlike other providers, this one is compatible with JTA transactions, which are mandatory in Java EE.

Spring works with both stand-alone JTA transaction managers (e.g. Bitronix or Atomikos) and Java EE `DataSource(s)`, and, because it offers the best control over the actual `DataSource` configuration, the `DataSourceConnectionProvider` is the preferred choice (even for HikariCP).

## 8.2.5 Connection release modes

Hibernate defers the database connection acquisition until the current transaction has to execute its first SQL statement (either triggered by a read or a write operation). This optimization allows Hibernate to reduce the physical transaction interval, therefore increasing the chance of getting a connection from the pool.

The connection release strategy is controlled through the `hibernate.connection.release_mode` property which can take the following values:

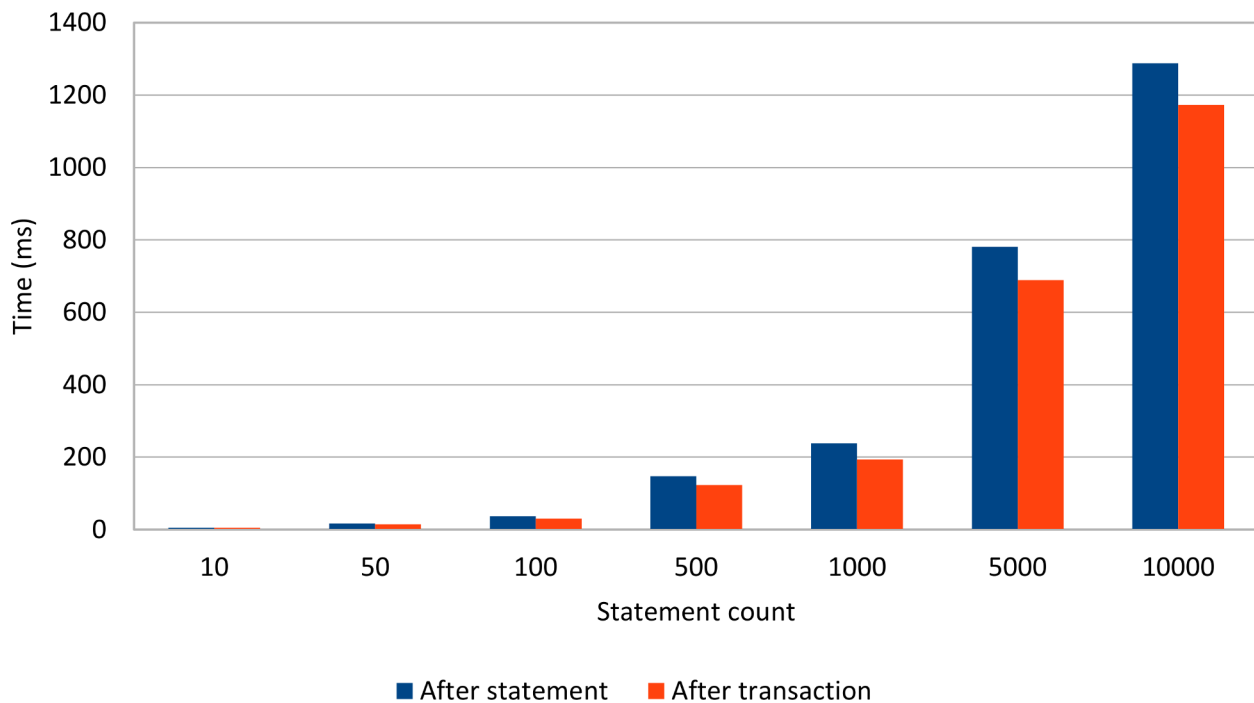
**Table 8.3: Connection release modes**

| Value                          | Description                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>after_transaction</code> | Once acquired, the database connection is released only after the current transaction either commits or rolls back.                                                                                                                                                                                                                                           |
| <code>after_statement</code>   | The connection is released after each statement execution and reacquired prior to running the next statement. Although not required by either JDBC or JTA specifications, this strategy is meant to prevent application servers from <a href="#">mistakenly detecting</a> <sup>7</sup> a connection leak between successive EJB (Enterprise Java Beans) calls |
| <code>auto</code>              | This is the default value, and for <code>RESOURCE_LOCAL</code> transactions, it uses the <code>after_transaction</code> mode, while for JTA transactions it falls back to <code>after_statement</code> .                                                                                                                                                      |

<sup>7</sup><http://lists.jboss.org/pipermail/hibernate-dev/2006-December/000903.html>

For JTA transactions, the default mode might be too strict since not all Java EE application servers exhibit the same behavior for managing transactional resources. This way, it is important to check if database connections can be closed outside of the EJB component that triggered the connection acquisition event. Spring-based enterprise systems do not use Enterprise Java Beans, and, even when using a stand-alone JTA transaction manager, the `after_transaction` connection release mode might be just fine.

It is somehow intuitive that the `after_statement` mode incurs some performance penalty associated with the frequent acquisition/releasing connection cycles. For this reason, the following test measures the connection acquisition overhead when using Bitronix in a Spring application context. Each transaction executes the same statement (fetching the current timestamp) for a given number of times (represented on the x-axis). The y-axis captures the recorded transaction response times for both `after_statement` and `after_transaction` connection release modes.



**Figure 8.1: Connection release mode**

The more statements a transaction executes, the greater the penalty of reacquiring the associated database connection from the underlying connection pool. To better visualize the connection acquisition overhead, the test runs up to 10 000 statements, even if this number is probably too high for the typical OLTP transaction.

Ideally, database transactions should be as short as possible, and the number of statements should not be too high either. This requirement stems from the fact that the number of pooled connections is limited and locks are better released sooner than later.



The `after_transaction` connection release mode is more efficient than the default JTA `after_statement` strategy, and so it should be used if the JTA transaction resource management logic does not interfere with this connection releasing strategy.

## 8.3 Monitoring connections

As previously concluded, using an externally configured `DataSource` is preferred because the actual `DataSource` can be decorated with connection pooling, monitoring and logging capabilities. Because that is exactly how [FlexyPool<sup>8</sup>](https://github.com/vladmihalcea/flexy-pool) works too, the following diagram captures the `DataSource` proxying mechanism:

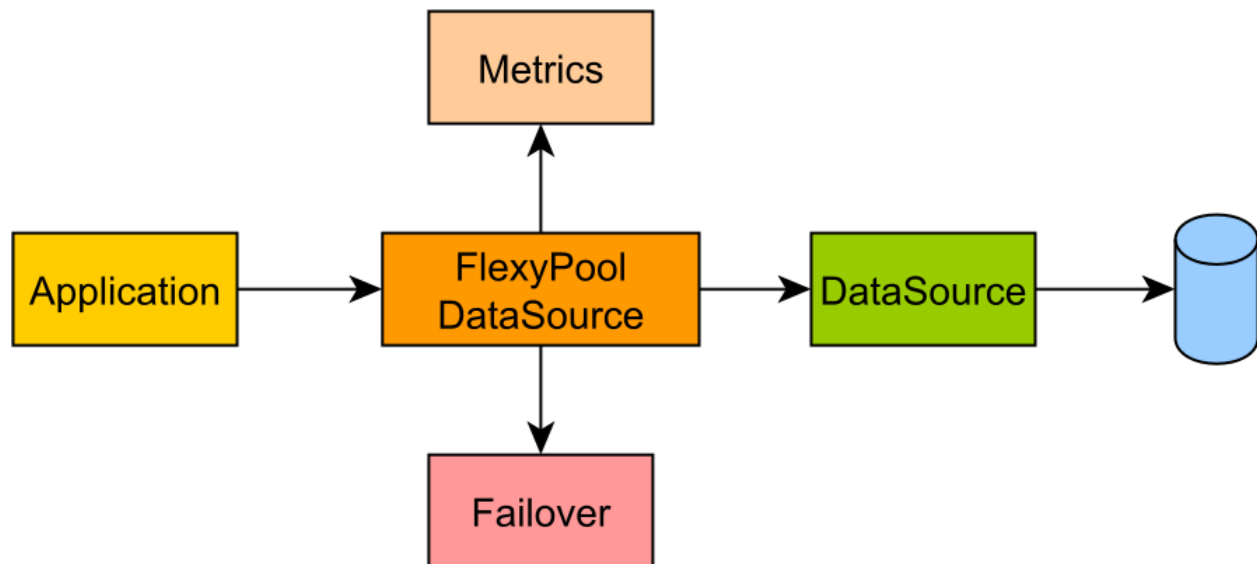


Figure 8.2: FlexyPool architecture

Instead of getting the actual `DataSource` instance, the data access layer gets a proxy reference. The proxy intercepts connection acquisition and releasing requests, and, this way, it can monitor its usage.

When using Spring, setting up FlexyPool is fairly easy because the application has total control over the `DataSource` configuration.

In Java EE, database connections should always be fetched from a managed `DataSource`, and one simple way of integrating FlexyPool is to extend the default `DatasourceConnectionProviderImpl` and substitute the original `DataSource` with the `FlexyPoolDataSource`.

<sup>8</sup><https://github.com/vladmihalcea/flexy-pool>

For this reason, FlexyPool comes with the following Hibernate connection provider:

```
public class FlexyPoolHibernateConnectionProvider
    extends DataSourceConnectionProviderImpl {

    private transient FlexyPoolDataSource<DataSource> flexyPoolDataSource;

    @Override
    public void configure(Map props) {
        super.configure(props);
        flexyPoolDataSource = new FlexyPoolDataSource<>(getDataSource());
    }

    @Override
    public Connection getConnection() throws SQLException {
        return flexyPoolDataSource.getConnection();
    }

    @Override
    public boolean isUnwrappableAs(Class unwrapType) {
        return super.isUnwrappableAs(unwrapType) ||
            getClass().isAssignableFrom(unwrapType);
    }

    @Override
    public void stop() {
        flexyPoolDataSource.stop();
        super.stop();
    }
}
```

To use the `FlexyPoolHibernateConnectionProvider`, the application must configure the `hibernate.connection.provider_class` property:

```
<property
    name="hibernate.connection.provider_class"
    value="com.vladmihalcea.flexypool.adaptor.FlexyPoolHibernateConnectionProvider"
/>
```

### 8.3.1 Hibernate statistics

Hibernate has a built-in statistics collector which gathers notifications related to database connections, `Session` transactions and even second-level caching usage. The `StatisticsImplementor` interface defines the contract for intercepting various Hibernate internal events:

| StatisticsImplementor                                                                                                                 |                   |
|---------------------------------------------------------------------------------------------------------------------------------------|-------------------|
|  <code>openSession()</code>                          | <code>void</code> |
|  <code>closeSession()</code>                         | <code>void</code> |
|  <code>flush()</code>                                | <code>void</code> |
|  <code>connect()</code>                              | <code>void</code> |
|  <code>prepareStatement()</code>                     | <code>void</code> |
|  <code>closeStatement()</code>                       | <code>void</code> |
|  <code>endTransaction(boolean)</code>                | <code>void</code> |
|  <code>loadEntity(String)</code>                     | <code>void</code> |
|  <code>fetchEntity(String)</code>                    | <code>void</code> |
|  <code>updateEntity(String)</code>                   | <code>void</code> |
|  <code>insertEntity(String)</code>                   | <code>void</code> |
|  <code>deleteEntity(String)</code>                   | <code>void</code> |
|  <code>optimisticFailure(String)</code>              | <code>void</code> |
|  <code>loadCollection(String)</code>                | <code>void</code> |
|  <code>fetchCollection(String)</code>              | <code>void</code> |
|  <code>updateCollection(String)</code>             | <code>void</code> |
|  <code>recreateCollection(String)</code>           | <code>void</code> |
|  <code>removeCollection(String)</code>             | <code>void</code> |
|  <code>secondLevelCachePut(String)</code>          | <code>void</code> |
|  <code>secondLevelCacheHit(String)</code>          | <code>void</code> |
|  <code>secondLevelCacheMiss(String)</code>         | <code>void</code> |
|  <code>naturalIdCachePut(String)</code>            | <code>void</code> |
|  <code>naturalIdCacheHit(String)</code>            | <code>void</code> |
|  <code>naturalIdCacheMiss(String)</code>           | <code>void</code> |
|  <code>naturalIdQueryExecuted(String, long)</code> | <code>void</code> |
|  <code>queryCachePut(String, String)</code>        | <code>void</code> |
|  <code>queryCacheHit(String, String)</code>        | <code>void</code> |
|  <code>queryCacheMiss(String, String)</code>       | <code>void</code> |
|  <code>queryExecuted(String, int, long)</code>     | <code>void</code> |
|  <code>updateTimestampsCacheHit()</code>           | <code>void</code> |
|  <code>updateTimestampsCacheMiss()</code>          | <code>void</code> |
|  <code>updateTimestampsCachePut()</code>           | <code>void</code> |

**Figure 8.3: Hibernate `StatisticsImplementor` interface**

There is a great variety of metrics Hibernate can collect on user's behalf, but, for performance reasons, the statistics mechanism is disabled by default.

To enable the statistics gathering mechanism, the following property must be configured first:

```
<property name="hibernate.generate_statistics" value="true"/>
```

Once statistics are being collected, in order to print them into the current application log, the following logger configuration must be set up:

```
<logger  
  name="org.hibernate.engine.internal.StatisticalLoggingSessionEventListener"  
  level="info" />
```

With these two settings in place, whenever a Hibernate `Session` (Persistence Context) ends, the following report is displayed in the currently running log.

```
37125102 nanoseconds spent acquiring 10000 JDBC connections;  
25521714 nanoseconds spent releasing 10000 JDBC connections;  
95242323 nanoseconds spent preparing 10000 JDBC statements;  
923615040 nanoseconds spent executing 10000 JDBC statements;
```

The default statistics collector just counts the number of times a certain callback method was called, and, if that is not satisfactory, the application developer can supply its own custom `StatisticsImplementor` implementation.

## Dropwizard Metrics

In a high-throughput transaction system, the amount of metric data needed to be recorded can be overwhelming, so storing all these values into memory is not practical at all.

To reduce the memory footprint, the [Dropwizard Metrics](https://github.com/dropwizard/metrics)<sup>a</sup> project uses various [reservoir sampling](https://en.wikipedia.org/wiki/Reservoir_sampling)<sup>b</sup> strategies, which either employ a fixed-size sampler or a time-based sampling window.

Not only it supports a great variety of metrics (e.g. timers, histograms, gauges), but Dropwizard Metrics can use multiple reporting channels as well (e.g. SLF4J, JMX, Ganglia, Graphite).



For all these reasons, it is better to use a mature framework such as Dropwizard Metrics instead of building a custom implementation from scratch.

<sup>a</sup><https://github.com/dropwizard/metrics>

<sup>b</sup>[https://en.wikipedia.org/wiki/Reservoir\\_sampling](https://en.wikipedia.org/wiki/Reservoir_sampling)

### 8.3.1.1 Customizing statistics

Although the built-in metrics are rather informative, Hibernate is not limited to the default statistics collector mechanism which can be completely customized.

In the upcoming example, the statistics collector also provides the following metrics:

- the distribution of physical transaction time (the interval between the moment a connection is first acquired and when it gets released)
- a histogram of the number of connections acquisition requests during the lifespan of any given transaction (due to the `after_statement` release mode).

The `StatisticsReport` class provides metric storage and report generation features on top of Dropwizard Metrics:

```
public class StatisticsReport {

    private final Logger LOGGER = LoggerFactory.getLogger(getClass());

    private MetricRegistry metricRegistry = new MetricRegistry();

    private Histogram connectionCountHistogram = metricRegistry.
        histogram("connectionCountHistogram");

    private Timer transactionTimer = metricRegistry.
        timer("transactionTimer");

    private Slf4jReporter logReporter = Slf4jReporter
        .forRegistry(metricRegistry)
        .outputTo(LOGGER)
        .build();

    public void transactionTime(long nanos) {
        transactionTimer.update(nanos, TimeUnit.NANOSECONDS);
    }

    public void connectionsCount(long count) {
        connectionCountHistogram.update(count);
    }

    public void generate() {
        logReporter.report();
    }

}
```

The `StatisticsImplementor` interface defines the contract between the Hibernate internal API and the various custom statistics gathering implementations. For simplicity sake, the following `StatisticsImplementor` interface implementation extends the default `ConcurrentStatisticsImpl` class, as it only needs to override the `connect` and `endTransaction` methods.

```
public class TransactionStatistics extends ConcurrentStatisticsImpl {

    private static final ThreadLocal<AtomicLong> startNanos =
        ThreadLocal.withInitial(AtomicLong::new);

    private static final ThreadLocal<AtomicLong> connectionCounter =
        ThreadLocal.withInitial(AtomicLong::new);

    private StatisticsReport report = new StatisticsReport();

    @Override
    public void connect() {
        connectionCounter.get().incrementAndGet();
        startNanos.get().compareAndSet(0, System.nanoTime());
        super.connect();
    }

    @Override
    public void endTransaction(boolean success) {
        try {
            report.transactionTime(System.nanoTime() - startNanos.get().get());
            report.connectionsCount(connectionCounter.get().get());
            report.generate();
        } finally {
            startNanos.remove();
            connectionCounter.remove();
        }
        super.endTransaction(success);
    }
}
```

The `StatisticsImplementor` is a singleton instance, therefore the `ThreadLocal` counters must be reset after a transaction is ended. When a transaction ends, the report is generated, and both the physical transaction time and the number of connection requests, issued during a particular transaction, are flushed to the log.

Because a Persistence Context can run multiple concurrent transactions, the report is generated at the end of each individual transaction.

To use a custom `StatisticsImplementor` instance, Hibernate requires a `StatisticsFactory` supplied as a configuration property. Taking a `SessionFactoryImplementor` parameter, the `StatisticsImplementor` building process has access to the Hibernate configuration data as well.

```
public class TransactionStatisticsFactory implements StatisticsFactory {  
    @Override public StatisticsImplementor buildStatistics(  
        SessionFactoryImplementor sessionFactory) {  
        return new TransactionStatistics();  
    }  
}
```

The `hibernate.stats.factory` configuration property must contain the fully qualified name of the `StatisticsFactory` implementation class:

```
<property name="hibernate.stats.factory" value="com.vladmihalcea.book.hpjp.hibernate\  
.statistics.TransactionStatisticsFactory" />
```

When running the previous JTA connection release mode example along with this custom statistics collector, the following output is being displayed:

```
type=HISTOGRAM, name=connectionCounterHistogram, count=107,  
min=1, max=10000, mean=162.41, stddev=1096.69,  
median=1.0, p75=1.0, p95=50.0, p98=1000.0, p99=5000.0, p999=10000.0
```

```
type=TIMER, name=transactionTimer, count=107,  
min=0.557524, max=1272.75, mean=27.16, stddev=152.57,  
median=0.85, p75=1.24, p95=41.25, p98=283.50, p99=856.19, p999=1272.75,  
mean_rate=36.32, rate_unit=events/second, duration_unit=milliseconds
```



For a high-performance data access layer, statistics and metrics becomes mandatory requirements. The Hibernate statistics mechanism is a very powerful tool, allowing the development team to get a better insight into Hibernate inner workings.

## 8.4 Statement logging

An ORM tool can automatically generate DML statements, and it is the application developer responsibility to validate both their effectiveness as well as their overall performance impact. Deferring the SQL statement validation until the data access layer starts showing performance issues is risky, and it can even impact development cost. For this reason, SQL statement logging becomes relevant from the early stages of application development.



When a business logic is implemented, the Definition of Done should include a review of all the associated data access layer operations. Following this rule can save much hassle when the enterprise system is deployed into production.

Although the JPA 2.1 does not feature a standard configuration property for logging SQL statements, most JPA implementations support this feature through framework-specific setups. For this purpose, Hibernate defines the following configuration properties:

**Table 8.4: Connection release modes**

| Property                                | Description                                                          |
|-----------------------------------------|----------------------------------------------------------------------|
| <code>hibernate.show_sql</code>         | Prints SQL statements to the console                                 |
| <code>hibernate.format_sql</code>       | Formats SQL statements before being logged or printed to the console |
| <code>hibernate.use_sql_comments</code> | Adds comments to the automatically generated SQL statement           |

Using the `System` console for logging is bad practice, a logging framework (e.g. Logback or Log4j) being a better alternative for it supports configurable appenders and logging levels.



Because it prints to the console, the `hibernate.show_sql` property should be avoided.

Hibernate logs all SQL statements on a debug level in the `org.hibernate.SQL` logging hierarchy.

To enable statement logging, the following Logback logger must be added to the associated configuration file:

```
<logger name="org.hibernate.SQL" level="debug"/>
```

Because Hibernate uses `PreparedStatement(s)` exclusively, the bind parameter values are not available when the statement gets printed into the log:

```
INSERT INTO post (title, version, id) VALUES (?, ?, ?)
```



Although bind parameters might be logged separately (e.g. `org.hibernate.type.descriptor.sql`), the most straight-forward way of logging SQL statements along with their runtime bind parameter values is to use an external `DataSource` proxy. Because the proxy intercepts all statement executions, the bind parameter values can be introspected and printed as well.

### 8.4.1 Statement formatting

By default, every SQL statement, no matter how long, is written as a single line of text. To increase readability, Hibernate can transform SQL statements in a human-readable format that spans over multiple log lines. This feature can be activated by setting the following configuration property:

```
<property name="hibernate.format_sql" value="true" />
```

With this setting in place, the previous statement can be formatted as follows:

```
insert
  into
    post
    (title, version, id)
  values
    (?, ?, ?)
```

Although formatting statements can improve readability, this setting is only suitable for the development phase. In a production system, logs are often parsed and aggregated in a centralized system, and the multi-line statement format can impact the log parsing mechanism. Once aggregated, logged queries can be formatted prior to being displayed in the application performance monitoring user interface.



The `hibernate.format_sql` property applies to logged statements only, and it does not propagate to the underlying JDBC `Driver` (SQL statements are still sent as single lines of text).

This way, the statement formatting does not have any effect when statements are logged through an external `DataSource` proxy.

### 8.4.2 Statement-level comments

Besides formatting, Hibernate can explain the statement generation process by appending SQL-level comments into the statement body. This feature allows the application developer to get a better understanding of the following processes:

- the entity state transition that triggered the currently executing statement
- the reason for choosing a join when fetching a given result set
- the explicit locking mechanism employed by the current statement.

By default, Hibernate does not append any SQL comment in the automatically generated statements, and, to enable this mechanism, the following Hibernate property must be configured:

```
<property name="hibernate.use_sql_comments" value="true" />
```

When persisting a `Post` entity, Hibernate explains the entity state transition associated with this particular statement through the following comment:

```
/* insert com.vladmihalcea.book.hppj.util.providers.BlogEntityProvider$Post */  
INSERT INTO post (title, version, id) VALUES (?, ?, ?)
```



As opposed to SQL statement formatting, SQL comments are generated not only during logging as they propagate to the underlying `Driver` as well.



Although it might be a useful technique for debugging purposes, in a production environment, it is better to leave it disabled, to reduce the database request networking overhead.

### 8.4.3 Logging parameters

Either the JDBC `Driver` or the `DataSource` must be proxied to intercept statement executions and log them along with the actual parameter values. Besides statement logging, a JDBC proxy can provide other cross-cutting features like long-running query detection or custom statement execution listeners.

#### 8.4.3.1 DataSource-proxy

A lesser-known JDBC logging framework, [datasource-proxy](https://github.com/ttddyy/datasource-proxy)<sup>9</sup> provides support for custom JDBC statement execution listeners. In Java EE, not all application servers allow configuring an external `DataSource`, as they rely on their own custom implementations that bind the user supplied JDBC `Driver`. Because it can only decorate a `DataSource`, `datasource-proxy` might not be suitable for all Java EE environments.

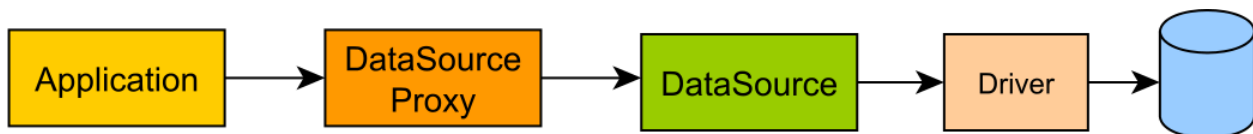


Figure 8.4: DataSource-Proxy Architecture

On the other hand, the programmatic configuration support fits the Java-based configuration approach taken by most modern Spring applications:

```

@Bean
public DataSource dataSource() {
    SLF4JQueryLoggingListener loggingListener = new SLF4JQueryLoggingListener();
    loggingListener.setQueryLogEntryCreator(new InlineQueryLogEntryCreator());
    return ProxyDataSourceBuilder
        .create(actualDataSource())
        .name(DATA_SOURCE_PROXY_NAME)
        .listener(loggingListener)
        .build();
}
  
```

<sup>9</sup><https://github.com/ttddyy/datasource-proxy>

In the following example, `datasource-proxy` is used to log a batch insert of three `PreparedStatement(s)`. Although normally a batch is printed in a single line of log, the output was split into multiple lines to fit the current page layout.

```
Name:DATA_SOURCE_PROXY, Time:6, Success:True,
Type:Prepared, Batch:True, QuerySize:1, BatchSize:3,
Query:["insert into post (title, version, id) values (?, ?, ?)"],
Params:[(Post no. 0, 0, 0), (Post no. 1, 0, 1), (Post no. 2, 0, 2)]
```

Not only the bind parameter values are now present, but, because they are grouped all together, it is very easy to visualize the batching mechanism too.



With the custom statement listener support, `datasource-proxy` allows building a query count validator to assert the auto-generated statement count and, therefore, prevent N+1 query problems during the development phase.

#### 8.4.3.2 P6Spy

**P6Spy**<sup>10</sup> was released in 2002, in an era when J2EE application servers were ruling the world of enterprise systems. Because Java EE application servers do not allow programmatic `DataSource` configuration, P6Spy supports a declarative configuration approach (through a `spy.properties` file).

P6Spy offers support for proxying both a JDBC `Driver` (which is suitable for Java EE applications) or a JDBC `DataSource` (supported by some Java EE containers and common practice for Spring enterprise applications).

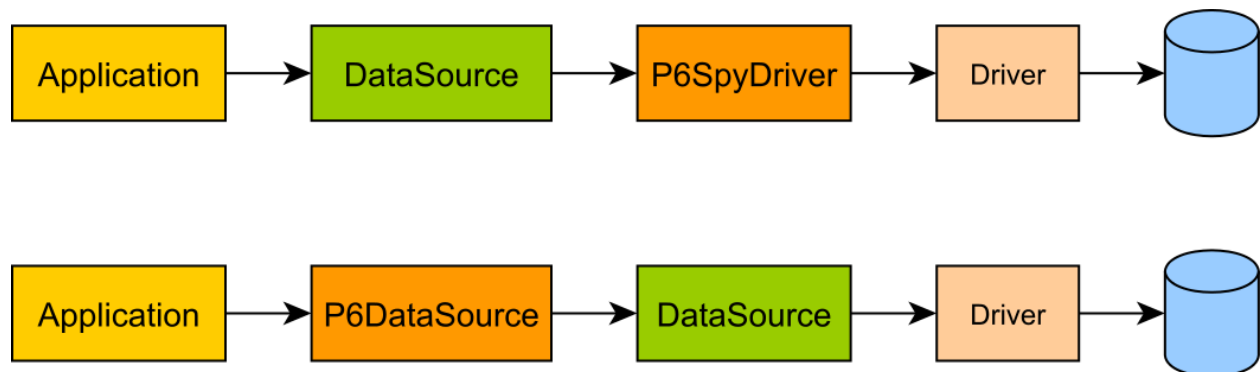


Figure 8.5: P6Spy Architecture

<sup>10</sup><https://github.com/p6spy/p6spy>

Running the previous example gives the following output (formatting was also applied):

```
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 0', 0, 0)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 1', 0, 1)
p6spy - 1448122491807|0|batch|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
p6spy - 1448122491812|5|statement|connection 7|
        insert into post (title, version, id) values (?, ?, ?)|
        insert into post (title, version, id) values ('Post no. 2', 0, 2)
```

In the order of their occurrence, the output is built out of the following columns:

**Table 8.5: P6Spy output**

| Field               | Description                                                                        |
|---------------------|------------------------------------------------------------------------------------|
| Timestamp           | The statement execution timestamp                                                  |
| Execution time      | The statement execution duration (in milliseconds)                                 |
| Category            | The current statement category (e.g. statement, batch)                             |
| Connection          | The database connection identifier (as assigned by P6Spy)                          |
| Original statement  | The original statement that was intercepted by P6Spy                               |
| Formatted statement | The statement with all parameter placeholders replaced with the actual bind values |

The first three lines are associated with adding statements to the batch, while the fourth line is logging the actual batch execution (which also explains the *execution time* column value).



One very useful configuration is the `outagedetection` property, which can detect long-running statements.

## 9. Mapping Types and Identifiers

JPA addresses the Object/Relational mismatch by associating Java object types to database structures. Assuming there is a `task` database table having four columns (e.g. `id`, `created_by`, `changed_on`, and `status`), the JPA provider must map it to Domain Model consisting of two Java class (e.g. `Task` and `Change`).

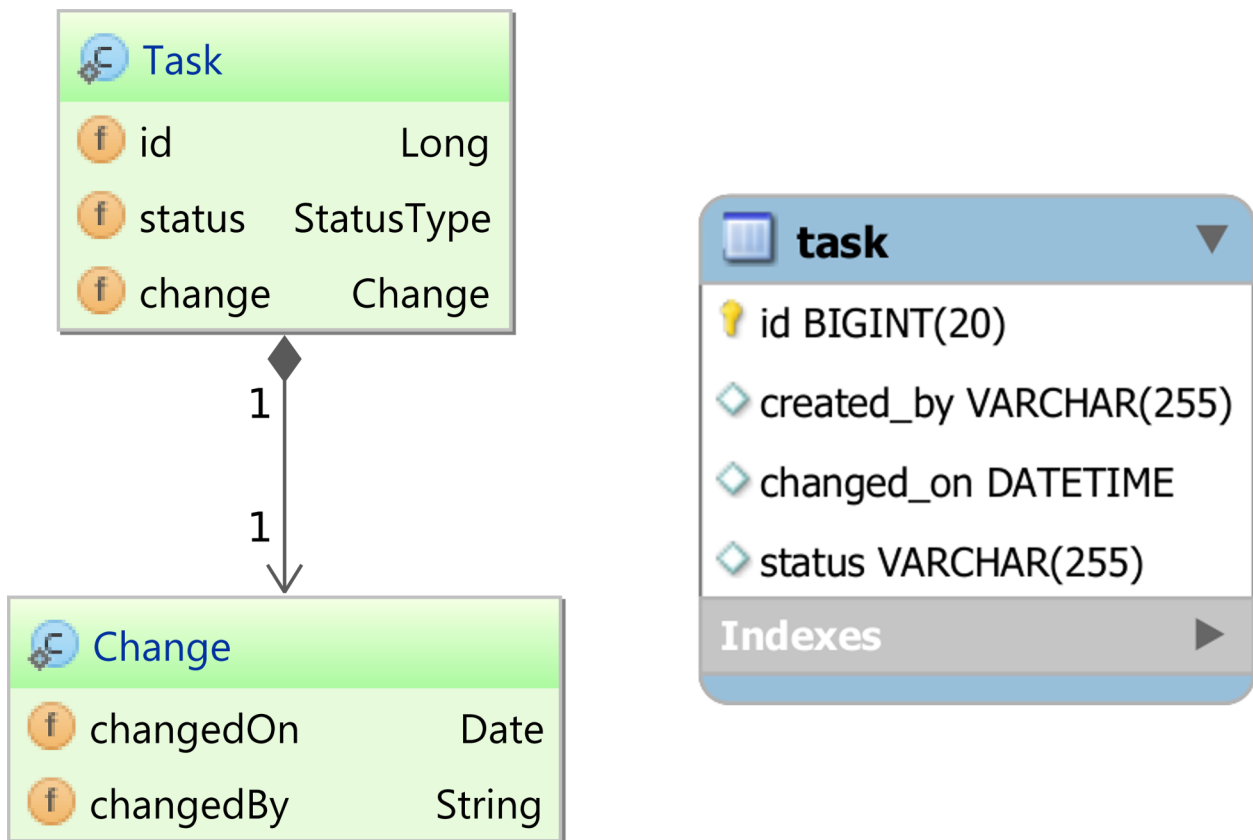


Figure 9.1: Type Mapping

JPA uses three main Object-Relational mapping elements: *type*, *embeddable*, and *entity*. In the previous diagram, the `Task` object is an entity, while the `Change` object is an embeddable type.

Both the entity and the embeddable group multiple Domain Model attributes by relying on Hibernate `Type(s)` to associate database column types with Java value objects (e.g. `String`, `Integer`, `Date`). The major difference between an entity and an embeddable is the presence of an identifier, which is used to associate a database table unique key (usually the primary key) with a Domain Model object attribute.



Although it is common practice to map all database columns, this is not a strict requirement. Sometimes it is more practical to use a root entity and several sub-entities, so each business case fetches just as much info as needed (while still benefiting from entity state management).

Identifiers are mandatory for entity elements, and an embeddable type is forbidden to have an identity of its own. Knowing the database table and the column that uniquely identifies any given row, Hibernate can correlate database rows with Domain Model entities.

An embeddable type groups multiple attributes in a single reusable component.

```
@Embeddable
public class Change {
    @Column(name = "changed_on")
    private Date changedOn;

    @Column(name = "created_by")
    private String changedBy;
}
```



The Domain Model can share state between multiple entities either by using inheritance or composition. Embeddable types can reuse state through composition.

The [composition association](#)<sup>1</sup>, defined by UML, is the perfect analogy for the relationship between an entity and an embeddable. When an entity includes an embeddable type, all its attributes become part of the owner entity.

The embeddable object cannot define its own identifier because otherwise, the entity has more than one identities. Lacking an identifier, the embeddable object cannot be managed by a Persistence Context, and its state is controlled by its parent entity.

Because they can have a significant impact on the overall application performance, the rest of the chapter discusses types and identifiers in greater detail.

---

<sup>1</sup>[https://en.wikipedia.org/wiki/Object\\_composition](https://en.wikipedia.org/wiki/Object_composition)

## 9.1 Types

For every supported database type, JDBC defines a `java.sql.JDBCType` enumeration. Since it builds on top of JDBC, Hibernate does the mapping between JDBC types and their associated Java counterparts (primitives or Objects).

### 9.1.1 Primitive types

**Table 9.1: Primitive Types**

| Hibernate type                  | JDBC type                                   | Java type                                      |
|---------------------------------|---------------------------------------------|------------------------------------------------|
| <code>BooleanType</code>        | <code>BIT</code>                            | <code>boolean</code> , <code>Boolean</code>    |
| <code>NumericBooleanType</code> | <code>INTEGER</code> (e.g. 0, 1)            | <code>boolean</code> , <code>Boolean</code>    |
| <code>TrueFalseType</code>      | <code>CHAR</code> (e.g. 'F', 'f', 'T', 't') | <code>boolean</code> , <code>Boolean</code>    |
| <code>YesNoType</code>          | <code>CHAR</code> (e.g. 'N', 'n', 'Y', 'y') | <code>boolean</code> , <code>Boolean</code>    |
| <code>ByteType</code>           | <code>TINYINT</code>                        | <code>byte</code> , <code>Byte</code>          |
| <code>ShortType</code>          | <code>SMALLINT</code>                       | <code>short</code> , <code>Short</code>        |
| <code>CharacterType</code>      | <code>CHAR</code>                           | <code>char</code> , <code>Character</code>     |
| <code>CharacterNCharType</code> | <code>NCHAR</code>                          | <code>char</code> , <code>Character</code>     |
| <code>IntegerType</code>        | <code>INTEGER</code>                        | <code>int</code> , <code>Integer</code>        |
| <code>LongType</code>           | <code>BIGINT</code>                         | <code>long</code> , <code>Long</code>          |
| <code>FloatType</code>          | <code>FLOAT</code>                          | <code>float</code> , <code>Float</code>        |
| <code>DoubleType</code>         | <code>DOUBLE</code>                         | <code>double</code> , <code>Double</code>      |
| <code>CharArrayType</code>      | <code>VARCHAR</code>                        | <code>char[]</code> , <code>Character[]</code> |

From one database system to another, the boolean type can be represented either as a `BIT`, `BYTE`, `BOOLEAN` or `CHAR` database type, so defines four `Type(s)` to resolve the boolean primitive type.



Only non-nullable database columns can be mapped to Java primitives (`boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, `double`). For mapping nullable columns, it is better to use the primitive wrappers instead (`Boolean`, `Byte`, `Short`, `Char`, `Integer`, `Long`, `Float`, `Double`).

### 9.1.2 String types

A Java `String` can consume as much memory as the Java Heap has available. On the other hand, database systems define both limited-size types (`VARCHAR` and `NVARCHAR`) and unlimited ones (`TEXT`, `NTEXT`, `BLOB`, and `NCLOB`).

To accommodate this mapping discrepancy, Hibernate defines the following Type(s):

**Table 9.2: String Types**

| Hibernate type        | JDBC type    | Java type |
|-----------------------|--------------|-----------|
| StringType            | VARCHAR      | String    |
| StringNVarcharType    | NVARCHAR     | String    |
| TextType              | LONGVARCHAR  | String    |
| NTextType             | LONGNVARCHAR | String    |
| MaterializedClobType  | CLOB         | String    |
| MaterializedNClobType | NCLOB        | String    |

### 9.1.3 Date and Time types

When it comes to time, there are multiple Java or database representations, which explains the vast number of time-related Hibernate Type(s).

**Table 9.3: Date and Time Types**

| Hibernate type   | JDBC type | Java type                   |
|------------------|-----------|-----------------------------|
| DateType         | DATE      | Date                        |
| TimeType         | TIME      | Time                        |
| TimestampType    | TIMESTAMP | Timestamp, Date             |
| DbTimestampType  | TIMESTAMP | Timestamp, Date             |
| CalendarType     | TIMESTAMP | Calendar, GregorianCalendar |
| CalendarDateType | DATE      | Calendar, GregorianCalendar |
| CalendarTimeType | TIME      | Calendar, GregorianCalendar |
| TimeZoneType     | VARCHAR   | TimeZone                    |



Handling time is tricky because of various time zones, leap seconds and daylight saving conventions. Storing timestamps in UTC (Coordinated Universal Time) and doing time zone transformations in the data layer is common practice.

### 9.1.4 Numeric types

Oracle can represent numbers up to 38 digits, therefore only fitting in a `BigInteger` or a `BigDecimal` (`java.lang.Long` and `java.lang.Double` can only store up to 8 bytes).

**Table 9.4: Numeric Types**

| Hibernate type              | JDBC type | Java type               |
|-----------------------------|-----------|-------------------------|
| <code>BigIntegerType</code> | NUMERIC   | <code>BigInteger</code> |
| <code>BigDecimalType</code> | NUMERIC   | <code>BigDecimal</code> |

### 9.1.5 Binary types

For binary types, most database systems offer multiple storage choices (e.g. RAW, VARBINARY, BYTEA, BLOB, CLOB). In Java, the data access layer can use an array of `byte(s)`, a JDBC `Blob` or `Clob`, or even a `Serializable` type, if the Java object was marshaled prior to being saved to the database.

**Table 9.5: Binary Types**

| Hibernate type                      | JDBC type     | Java type                   |
|-------------------------------------|---------------|-----------------------------|
| <code>BinaryType</code>             | VARBINARY     | <code>byte[], Byte[]</code> |
| <code>BlobType</code>               | BLOB          | <code>Blob</code>           |
| <code>ClobType</code>               | CLOB          | <code>Clob</code>           |
| <code>NClobType</code>              | NCLOB         | <code>Clob</code>           |
| <code>MaterializedBlobType</code>   | BLOB          | <code>byte[], Byte[]</code> |
| <code>ImageType</code>              | LONGVARBINARY | <code>byte[], Byte[]</code> |
| <code>SerializableType</code>       | VARBINARY     | <code>Serializable</code>   |
| <code>SerializableToBlobType</code> | BLOB          | <code>Serializable</code>   |

### 9.1.6 UUID types

There are various ways of persisting a Java `UUID` (Universally Unique Identifier), and, based on the memory footprint, the most efficient storage types are the database-specific `UUID` column types.

**Table 9.6: UUID Types**

| Hibernate type                | JDBC type | Java type         |
|-------------------------------|-----------|-------------------|
| <code>UUIDBinaryType</code>   | BINARY    | <code>UUID</code> |
| <code>UUIDCharType</code>     | VARCHAR   | <code>UUID</code> |
| <code>PostgresUUIDType</code> | OTHER     | <code>UUID</code> |



When not natively supported, a `BINARY` type requires fewer bytes than a `VARCHAR`, so the associated index has a smaller memory footprint too.

### 9.1.7 Other types

Hibernate can also map Java `Enum(s)`, `Class`, `URL`, `Locale` and `Currency` too.

**Table 9.7: Other Types**

| Hibernate type | JDBC type                                                                                         | Java type |
|----------------|---------------------------------------------------------------------------------------------------|-----------|
| EnumType       | CHAR, LONGVARCHAR, VARCHAR<br>INTEGER, NUMERIC, SMALLINT, TINYINT, BIGINT, DECIMAL, DOUBLE, FLOAT | Enum      |
| ClassType      | VARCHAR                                                                                           | Class     |
| CurrencyType   | VARCHAR                                                                                           | Currency  |
| LocaleType     | VARCHAR                                                                                           | Locale    |
| UrlType        | VARCHAR                                                                                           | URL       |

### 9.1.8 Custom types

Not only that it has a very rich set of data types, but PostgreSQL allows adding custom types as well (using the `CREATE DOMAIN`<sup>2</sup> DDL statement). Choosing the appropriate database type for each Domain Model field can really make a difference in terms of data access performance. Although there is a great variety of built-in `Type(s)`, the application developer is not limited to the off-the-shelf ones only, and new `Type(s)` can be added without too much effort.

In the following example, the business logic requires monitoring access to an enterprise application. For this purpose, the data access layer stores the IP (Internet Protocol) addresses of each logged-in user.

Assuming this internal application uses the IPv4 protocol only, the IP addresses are stored in the Classless Inter-Domain Routing format (e.g. `192.168.123.231/24`). PostgreSQL can store IPv4 addresses either in a `cidr` or `inet` type, or it can use a `VARCHAR(18)` column type.

The `VARCHAR(18)` column requires 18 characters, and, assuming a UTF-8 encoding, each IPv4 address needs at most 18 bytes. The smallest size address (e.g. `0.0.0.0/0`) taking 9 characters, the `VARCHAR(18)` approach requires between 9 and 18 characters for each IPv4 address.

The `inet` type is specially designed for IPv4 and IPv6 network addresses, and it also supports various network address specific operators (e.g. `<`, `>`, `&&`), as well as other address transforming

<sup>2</sup><http://www.postgresql.org/docs/9.5/static/sql-createdomain.html>

functions (e.g. `host(inet)`, `netmask(inet)`). As opposed to the `VARCHAR(18)` approach, the `inet` type requires only 7 bytes for each IPv4 address.

For it has a more compact size (the index can better fit into memory) and supporting many specific operators, the `inet` type is a much more attractive choice. Although, by default, Hibernate does not support `inet` types, adding a custom Hibernate `Type` is a straightforward task. The IPv4 address is encapsulated in its own wrapper, which can also define various address manipulation functions too.

```
public class IPv4 implements Serializable {

    private final String address;

    public IPv4(String address) {
        this.address = address;
    }

    public String getAddress() {
        return address;
    }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        return Objects.equals(address, IPv4.class.cast(o).address);
    }

    @Override public int hashCode() {
        return Objects.hash(address);
    }

    public InetAddress toInetAddress() throws UnknownHostException {
        return InetAddress.getByAddress(address);
    }
}
```

When an entity wants to change an `IPv4` field, it must provide a new object instance. An immutable type is much easier to handle since its internal state does not change throughout the currently running Persistence Context.

All custom types must implement the `UserType` interface, and, since the `ImmutableType` takes care of most `UserType` implementation details, the `IPv4Type` can focus on type-specific conversation logic.

```
public class IPv4Type extends ImmutableType<IPv4> {

    public IPv4Type() {
        super(IPv4.class);
    }

    @Override public int[] sqlTypes() { return new int[]{ Types.OTHER}; }

    @Override public IPv4 get(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException {
        String ip = rs.getString(names[0]);
        return (ip != null) ? new IPv4(ip) : null;
    }

    @Override public void set(PreparedStatement st, IPv4 value, int index,
        SessionImplementor session) throws SQLException {
        if (value == null) {
            st.setNull(index, Types.OTHER);
        } else {
            PGobject holder = new PGobject();
            holder.setType("inet");
            holder.setValue(value.getAddress());
            st.setObject(index, holder);
        }
    }
}
```

The `get()` method is used to map the `inet` field to an `IPv4` object instance, while the `set()` is used for transforming the `IPv4` object to the PostgreSQL JDBC driver `inet` equivalent.



`Types.OTHER` is used for mapping database types not supported by JDBC.

```
public abstract class ImmutableType<T> implements UserType {
    private final Class<T> clazz;

    protected ImmutableType(Class<T> clazz) { this.clazz = clazz; }

    @Override public Object nullSafeGet(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException {
        return get(rs, names, session, owner);
    }

    @Override public void nullSafeSet(PreparedStatement st, Object value,
        int index, SessionImplementor session) throws SQLException {
        set(st, clazz.cast(value), index, session);
    }

    @Override public Class<T> returnedClass() { return clazz; }

    @Override public boolean equals(Object x, Object y) {
        return Objects.equals(x, y);
    }

    @Override public int hashCode(Object x) { return x.hashCode(); }

    @Override public Object deepCopy(Object o) { return o; }

    @Override public boolean isMutable() { return false; }

    @Override public Serializable disassemble(Object o) {
        return (Serializable) o;
    }

    @Override public Object assemble(Serializable o, Object owner) { return o; }

    @Override
    public Object replace(Object o, Object target, Object owner) { return o; }

    protected abstract T get(ResultSet rs, String[] names,
        SessionImplementor session, Object owner) throws SQLException;

    protected abstract void set(PreparedStatement st, T value, int index,
        SessionImplementor session) throws SQLException;
}
```

The `@Type` annotation instructs Hibernate to use the `IPv4Type` for mapping the `IPv4` field.

```
@Entity
public class Event {

    @Id @GeneratedValue
    private Long id;

    @Type(type = "com.vladmihalcea.book.hpjp.hibernate.type.IPv4Type")
    @Column(name = "ip", columnDefinition = "inet")
    private IPv4 ip;

    public Event() {}

    public Event(String address) {
        this.ip = new IPv4(address);
    }

    public Long getId() {
        return id;
    }

    public IPv4 getIp() {
        return ip;
    }

    public void setIp(String address) {
        this.ip = new IPv4(address);
    }
}
```

## GiST operators

PostgreSQL 9.4 [added GiST operator support<sup>a</sup>](http://www.postgresql.org/docs/current/static/gist-builtin-opclasses.html) for `inet` and `cidr` column types. To enable this feature, a GiST index with the `inet_ops` operator class must be created on the associated `inet` columns.

```
CREATE INDEX ON event USING gist (ip inet_ops)
```

---

<sup>a</sup><http://www.postgresql.org/docs/current/static/gist-builtin-opclasses.html>

Managing Event(s) is easy when Hibernate takes care of the underlying type conversation.

```
final AtomicReference<Event> eventHolder = new AtomicReference<>();

doInJPA(entityManager -> {
    entityManager.persist(new Event());
    Event event = new Event("192.168.0.231");
    entityManager.persist(event);
    eventHolder.set(event);
});

doInJPA(entityManager -> {
    Event event = entityManager.find(Event.class, eventHolder.get().getId());
    event.setIp("192.168.0.123");
});
```

Running the previous example generates the following SQL statements:

```
INSERT INTO event (ip, id) VALUES (NULL(OTHER), 1)
INSERT INTO event (ip, id) VALUES (`192.168.0.231`, 2)

SELECT e0_.id as id1_0_0_, e0_.ip as ip2_0_0_
FROM event e0_
WHERE e0_.id = 2

UPDATE event SET ip=`192.168.0.123` WHERE id = 2
```

One of the best aspects of using database-specific types is getting access to advanced querying capabilities. Because the GiST index allows `inet_ops` operators, the following query can be used to check if an Event was generated for a given subnetwork:

```
Event matchingEvent = (Event) entityManager.createNativeQuery(
    "SELECT e.* " +
    "FROM event e " +
    "WHERE " +
    "    e.ip && CAST(:network AS inet) = TRUE", Event.class)
    .setParameter("network", "192.168.0.1/24")
    .getSingleResult();

assertEquals("192.168.0.123", matchingEvent.getIp().getAddress());
```

## 9.2 Identifiers

All database tables must have a primary key column, so each row can be uniquely identified (the primary key must be both `UNIQUE` and `NOT NULL`).

Although the SQL standard does not impose primary keys to be immutable, it is [more practical<sup>a</sup>](#) to avoid changing them.

<sup>a</sup>[https://asktom.oracle.com/pls/asktom/f?p=100:11:0::::P11\\_QUESTION\\_ID:5773459616034](https://asktom.oracle.com/pls/asktom/f?p=100:11:0::::P11_QUESTION_ID:5773459616034)

The primary key can have a meaning in the real world, in which case it is a *natural* key, or it can be generated synthetically, in which case it is called a *surrogate* identifier.

For natural keys, unicity is enforced by a real-world unique sequence generator (e.g. National Identification Numbers, Social Security Numbers, Vehicle Identification Numbers). In reality, natural unique numbers might pose problems when the unique constraints do not hold true anymore. For example, a National Identification Number might yield unique numbers, but if the enterprise system must accommodate users coming from multiple countries, it is possible that two different countries assigned the same identifier.

The natural key can be composed of one or multiple columns. Compound natural keys might incur an additional performance penalty because multi-column joins are slower than single-column ones, and multi-column indexes have a bigger memory footprint too.

Natural keys must be sufficiently long to accommodate as many identifiers as the system needs throughout its lifecycle. Because primary keys are often indexed, the longer the key, the more memory an index entry requires. Each joined table includes a foreign key mirroring the parent primary key, and foreign keys are frequently indexed as well.

### Index memory impact

Fixed-size non-numerical keys (e.g. `CHAR`, `VARCHAR`) are less efficient than numerical ones (e.g. `INTEGER`, `BIGINT`) both for joining (a simple key performs better than a compound one) or indexing (the more compact the data type, the less memory space is required by an associated index).

A `CHAR(17)` natural key (e.g. Vehicle Identification Number) requires 17 characters (17 bytes when using ASCII characters and a UTF-8 encoding) as opposed to 4 bytes (32 bit `INTEGER`) or 8 bytes (64 bit `BIGINT`).

Surrogate keys are generated independently of the current row data, so table column constraints may evolve with time (changing a user birthday or email address). The surrogate key can be generated by a numerical sequence generator (e.g. a database identity column or a sequence), or it can be constructed by a pseudorandom number generator (e.g. [GUID](#)<sup>3</sup> or [UUID](#)<sup>4</sup>). Both the numerical and UUID keys have both pros and cons.

The UUID is defined by the [RFC 4122](#)<sup>5</sup> standard and it is stored as a 128-bit sequence. The GUID term refers to any globally unique identifier, which might comprise other non-standard implementations. For consistency, this chapter further refers to unique identifiers as UUID.

A UUID takes 128 bits, which is four times more than an `INTEGER` and twice as `BIGINT`. On the other hand, a UUID number has less chance of a conflict in a Multi-Master database replication topology. To avoid such conflicts, many relational database systems increment the identity or sequence numbers in steps, each node getting its own offset. Because UUIDs are not sequential, they induce fragmentation, and that can really affect the performance of clustered indexes.



Requiring less space and being more index-friendly, numerical sequences are preferred over UUID keys.

### 9.2.1 UUID identifiers

Nevertheless, some enterprise systems use UUID primary keys, so it is worth knowing what Hibernate types work best for this task. The UUID key can either be generated by the application using the `java.util.UUID` class or it can be assigned by the database system.



If the database system does not have a built-in UUID type, a `BINARY(16)` column type is preferred. Although a `CHAR(32)` column could also store the UUID textual representation, the additional space overhead makes it a less favorable pick.

<sup>3</sup>[http://en.wikipedia.org/wiki/Globally\\_Unique\\_Identifier](http://en.wikipedia.org/wiki/Globally_Unique_Identifier)

<sup>4</sup>[http://en.wikipedia.org/wiki/Universally\\_Unique\\_Identifier](http://en.wikipedia.org/wiki/Universally_Unique_Identifier)

<sup>5</sup><https://www.ietf.org/rfc/rfc4122.txt>

## Oracle

There is no UUID type in Oracle, so a `RAW(16)` column must be used instead. The `SYS_GUID()`<sup>a</sup> database function can generate a globally unique identifier.

<sup>a</sup><http://docs.oracle.com/database/121/SQLRF/functions202.htm#SQLRF06120>

## SQL Server

The `uniqueidentifier`<sup>a</sup> data type is used for storing GUID identifiers. The `NEWID()`<sup>b</sup> function can generate a UUID compatible with the RFC 4122 standard.

Because by default SQL Server uses clustered indexes for primary keys, to avoid the fragmentation effect, the `NEWSEQUENTIALID()`<sup>c</sup> function can assign pseudo-sequential UUID numbers (greater than previously generated ones). This guarantee is kept as long as the Windows server is not restarted.

<sup>a</sup><https://msdn.microsoft.com/en-us/library/ms187942.aspx>

<sup>b</sup><https://msdn.microsoft.com/en-us/library/ms190348.aspx>

<sup>c</sup><https://msdn.microsoft.com/en-us/library/ms189786.aspx>

## PostgreSQL

The `UUID type`<sup>a</sup> can store RFC 4122 compliant unique identifiers. The database does not offer a built-in UUID generation function, so the identifier must be generated by the data access layer.

<sup>a</sup><http://www.postgresql.org/docs/current5/static/datatype-uuid.html>

## MySQL

The UUID must be stored in a `BINARY(16)` column type. The `UUID()`<sup>a</sup> functions can generate a 128-bit unique identifier. Because the `UUID()` function might cause problems for statement-based replication, passing the generated identifier as a variable<sup>b</sup> is a workaround to this limitation.

<sup>a</sup>[http://dev.mysql.com/doc/refman/5.7/en/miscellaneous-functions.html#function\\_uuid](http://dev.mysql.com/doc/refman/5.7/en/miscellaneous-functions.html#function_uuid)

<sup>b</sup><http://dev.mysql.com/doc/refman/5.7/en/replication-features-functions.html>

For generating UUID identifiers, Hibernate offers three generators (*assigned*, *uuid*, *uuid2*), which we'll be discussed in greater detail in the following sections.

### 9.2.1.1 The assigned generator

By simply omitting the `@GeneratedValue`<sup>6</sup> annotation, Hibernate falls back to the assigned identifier, which allows the data access layer to control the identifier generation process. The following example maps a `java.util.UUID` identifier to a `BINARY(16)` column type:

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "BINARY(16)")
    private UUID id;

    public Post() {}

    public Post(UUID id) {
        this.id = id;
    }
}
```

When persisting a `post`, Hibernate generates the following insert statement:

```
INSERT INTO post (id) VALUES
([86, 10, -104, 26, 60, -115, 79, 78, -118, -45, 64, 94, -64, -40, 66, 100])
```

The `UUIDBinaryType` translates the `java.util.UUID` to an array of `byte(s)` that is stored in the associated `BINARY(16)` column type.



Because the identifier is generated in the data access layer, the database server is freed from this responsibility, and so it can allocate its resources to other data processing tasks.

Hibernate can also generate a UUID identifier on behalf of the application developer, as described in the following two sections.

---

<sup>6</sup><https://docs.oracle.com/javaee/7/api/javax/persistence/GeneratedValue.html>

## 9.2.2 The legacy UUID generator

The [UUID hex generator](#)<sup>7</sup> is registered under the `uuid` name and generates hexadecimal UUID string representations. Using a 8-8-4-8-4 hex digit layout, the UUID hex generator is not compliant with the RFC 4122 standard, which uses a 8-4-4-4-12 hex digit format. The following code snippet depicts the `UUIDHexGenerator` mapping and the associated insert statement.

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "CHAR(32)")
    @GeneratedValue(generator = "uuid")
    @GenericGenerator(name = "uuid", strategy = "uuid")
    private String id;
}

INSERT INTO post (id) VALUES (402880e451724a820151724a83d00000)
```

### 9.2.2.1 The newer UUID generator

The newer [UUID generator](#)<sup>8</sup> is RFC 4122 compliant (variant 2) and is registered under the `uuid2` name (working with `java.lang.UUID`, `byte[]` and `String` Domain Model object types). Compared to the previous use case, the mapping and the test case look as follows:

```
@Entity @Table(name = "post")
public class Post {

    @Id @Column(columnDefinition = "BINARY(16)")
    @GeneratedValue(generator = "uuid2")
    @GenericGenerator(name = "uuid2", strategy = "uuid2")
    private UUID id;
}

INSERT INTO post (id) VALUES
([77, 2, 31, 83, -45, -98, 70, 40, -65, 40, -50, 30, -47, 16, 30, 124])
```

---

<sup>7</sup><https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/id/UUIDHexGenerator.html>

<sup>8</sup><https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/id/UUIDGenerator.html>



Being RFC 4122 compliant and able to operate with `BINARY` column type, the `UUIDGenerator` is preferred over the legacy `UUIDHexGenerator`.

### 9.2.3 Numerical identifiers

As previously explained, a numerical surrogate key is usually preferred since it takes less space and indexes work better with sequential identifiers. To generate numerical identifiers, most database systems offer either identity (or `auto_increment`) columns or sequence objects.

JPA defines the `GenerationType`<sup>9</sup> enumeration for all supported identifier generator types:

- `IDENTITY` is for mapping the entity identifier to a database identity column.
- `SEQUENCE` allocates identifiers by calling a given database sequence.
- `TABLE` is for relational databases that do not support sequences (e.g. MySQL 5.7), the *table* generator emulating a database sequence by using a separate table.
- `AUTO` decides the identifier generation strategy based on the current database dialect.



As explained in the JDBC part, database sequences work better with batch updates and allow various application-side optimization techniques as well.

#### 9.2.3.1 Identity generator

The identity column type (included in the `SQL:2003`<sup>10</sup> standard) is supported by `Oracle 12c`<sup>11</sup>, `SQL Server`<sup>12</sup> and `MySQL (AUTO_INCREMENT)`<sup>13</sup>, and it allows an `INTEGER` or a `BIGINT` column to be auto-incremented on demand.

The incrementation process is very efficient since it uses a lightweight locking mechanism, as opposed to the more heavyweight transactional course-grain locks. The only drawback is that the newly assigned value can only be known after executing the actual insert statement.

<sup>9</sup><https://docs.oracle.com/javaee/7/api/javax/persistence/GenerationType.html>

<sup>10</sup><http://en.wikipedia.org/wiki/SQL:2003>

<sup>11</sup>[http://docs.oracle.com/database/121/SQLRF/statements\\_7002.htm#SQLRF55657](http://docs.oracle.com/database/121/SQLRF/statements_7002.htm#SQLRF55657)

<sup>12</sup><http://msdn.microsoft.com/en-us/library/ms186775.aspx>

<sup>13</sup><http://dev.mysql.com/doc/refman/5.7/en/example-auto-increment.html>

## Batch updates

Because Hibernate separates the id generation from the actual entity insert statement, entities using the identity generator may not participate in JDBC batch updates. Hibernate issues the insert statement during the `persist()` method call, therefore breaking the transactional write-behind caching semantic used for entity state transitions.

Even if some JDBC drivers allow fetching the associated generated keys when executing a batch update, Hibernate still needs an improvement in this regard.

The identity generator can be mapped as follows:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
}
```

The following example demonstrates how the transaction write-behind caching model is circumvented by the identity column semantics. Although disabled by default, JDBC batching was enabled to compare results between identity and sequence generators.

```
doInJPA(entityManager -> {
    for (int i = 0; i < batchSize; i++) {
        entityManager.persist(new Post());
    }
    LOGGER.debug("Flush is triggered at commit-time");
});
```

Executing the previous test case generates the following output.

```
INSERT INTO post (id) VALUES (DEFAULT)
INSERT INTO post (id) VALUES (DEFAULT)

DEBUG - Flush is triggered at commit-time
```

Because the associated entity identifier can only be known after the insert statement is executed, Hibernate triggers the entity state transition prior to flushing the currently running Persistence Context.

### 9.2.3.2 Sequence generator

A sequence is a database object that generates consecutive numbers. Defined by the SQL:2003 standard, database sequences are supported by Oracle, SQL Server 2012 and PostgreSQL, and, compared to identity columns, sequences offer the following advantages:

- The same sequence can be used to populate multiple columns, even across tables.
- Values may be preallocated to improve performance.
- Allowing incremental steps, sequences can benefit from application-level optimization techniques.
- Because the sequence call can be decoupled from the actual insert statement, Hibernate does not disable JDBC batch updates.

To demonstrate the difference between the identity and the sequence identifier generators, the previous example is changed to use a database sequence this time.

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy=GenerationType.SEQUENCE)
    private Long id;
}
```

Running the previous test case generates the following output:

```
CALL NEXT VALUE FOR hibernate_sequence
CALL NEXT VALUE FOR hibernate_sequence

DEBUG - Flush is triggered at commit-time

INSERT INTO post (id) VALUES (1, 2)
```

When executing the `persist()` method, Hibernate calls the associated database sequence and fetches an identifier for the newly persisted entity. The actual insert statement is postponed until flush time, which allows Hibernate to take advantage of JDBC batching.

### 9.2.3.3 Table generator

Because of the mismatch between the identifier generator and the transactional write-behind cache, JPA offers an alternative sequence-like generator that works even when sequences are not natively supported.

A database table is used to hold the latest sequence value, and row-level locking is employed to prevent two concurrent connections from acquiring the same identifier value.

#### Escaping transactional row-level locking

A database sequence is a non-transactional object because the sequence value allocation happens outside of the transactional context associated with the database connection requesting a new identifier. Database sequences use dedicated locks to prevent concurrent transactions from acquiring the same value, but locks are released as soon as the counter is incremented. This design ensures minimal contention even when the sequence is used concomitantly by multiple concurrent transactions.

Using a database table as a sequence is challenging, as, to prevent two transactions from getting the same sequence value, row-level locking must be used. However, unlike the sequence object locks, the row-level lock is transactional, and, once acquired, it can only be released when the current transaction ends (either committing or rolling back). This would be a terrible scalability issue because a long-running transaction would prevent any other transaction from acquiring a new sequence value.

To cope with this limitation, a separate database transaction is used for fetching a new sequence value. This way, the row-level lock associated with incrementing the sequence counter value can be released as soon as the sequence update transaction ends.

For local transactions, a new transaction means fetching another database connection and committing it after executing the sequence processing logic. This can put additional pressure on the underlying connection pool, especially if there is already a significant contention for database connections.

In a JTA environment, the currently running transaction must be suspended, and the sequence value is fetched in a separate transaction. The JTA transaction manager has to do additional work to accommodate the transaction context switch, and that can also have an impact on the overall application performance.



Without any application-level optimization, the row-level locking approach can become a performance bottleneck if the sequence logic is called way too often.

To continue the previous example, the `post` uses the table generator this time:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    @GeneratedValue(strategy=GenerationType.TABLE)
    private Long id;
}
```

The following output is obtained when inserting a new `post`:

```
SELECT tbl.next_val
FROM hibernate_sequences tbl
WHERE tbl.sequence_name=default
FOR UPDATE

INSERT INTO hibernate_sequences (sequence_name, next_val)
VALUES (default, 1)

UPDATE hibernate_sequences SET next_val=2
WHERE next_val=1 AND sequence_name=default

SELECT tbl.next_val
FROM hibernate_sequences tbl
WHERE tbl.sequence_name=default
FOR UPDATE

UPDATE hibernate_sequences SET next_val=3
WHERE next_val=2 AND sequence_name=default

DEBUG - Flush is triggered at commit-time

INSERT INTO post (id) values (1, 2)
```

The table generator benefits from JDBC batching, but every table sequence update incurs three steps:

- The lock statement is executed to ensure that the same sequence value is not allocated for two concurrent transactions.
- The current value is incremented in the data access layer.
- The new value is saved back to the database and the secondary transaction is committed to release the row-level lock.

Unlike identity columns and sequences, which can increment the sequence in a single request, the table generator entails a significant performance overhead. For this reason, Hibernate comes with a series of optimizers which can improve performance for both sequence and table generators.



Although it is a portable identifier generation strategy, the table generator introduces a serializable execution (the row-level lock), which can hinder scalability. Compared to this application-level sequence generation technique, identity columns and sequences are highly optimized for high concurrency scenarios and should be the preferred choice anyway.

#### 9.2.3.4 Optimizers

As previously mentioned, both the sequence and the table identifier generator have multiple implementations which can improve the performance of the identifier generation process. The sequence and table generators can be split into two categories:

- Legacy implementations (being deprecated since Hibernate 5.0) like `SequenceGenerator`, `SequenceHiLoGenerator` and `MultipleHiLoPerTableGenerator`.
- Newer and more efficient implementations such as `SequenceStyleGenerator` and `TableGenerator`.

These two categories are not compatible, and the application developer must either choose the legacy identifiers or the enhanced ones. Prior to Hibernate 5.0, the legacy identifier generators were provided by default and the application developer could switch to the newer ones by setting the following configuration property:

```
<property name="hibernate.id.new_generator_mappings" value="true"/>
```

Hibernate 5 has decided to drop support for the legacy identifiers and to use the enhanced ones by default.

Among the legacy identifier generators, the `SequenceGenerator` did not offer any optimization, as every new identifier value would require a call to the underlying database sequence. On the other hand, the `SequenceHiLoGenerator` and the `MultipleHiLoPerTableGenerator` offered a *hi/lo* optimization mechanism aimed to reduce the number of calls to a database server. Although these generators are deprecated, the legacy *hi/lo* algorithm is still a valid optimizer even for the newer identifier generators.

### 9.2.3.4.1 The hi/lo algorithm

The hi/lo algorithm splits the sequences domain into *hi* groups. A *hi* value is assigned synchronously, and every *hi* group is given a maximum number of *lo* entries, that can be assigned offline without worrying about identifier value conflicts.

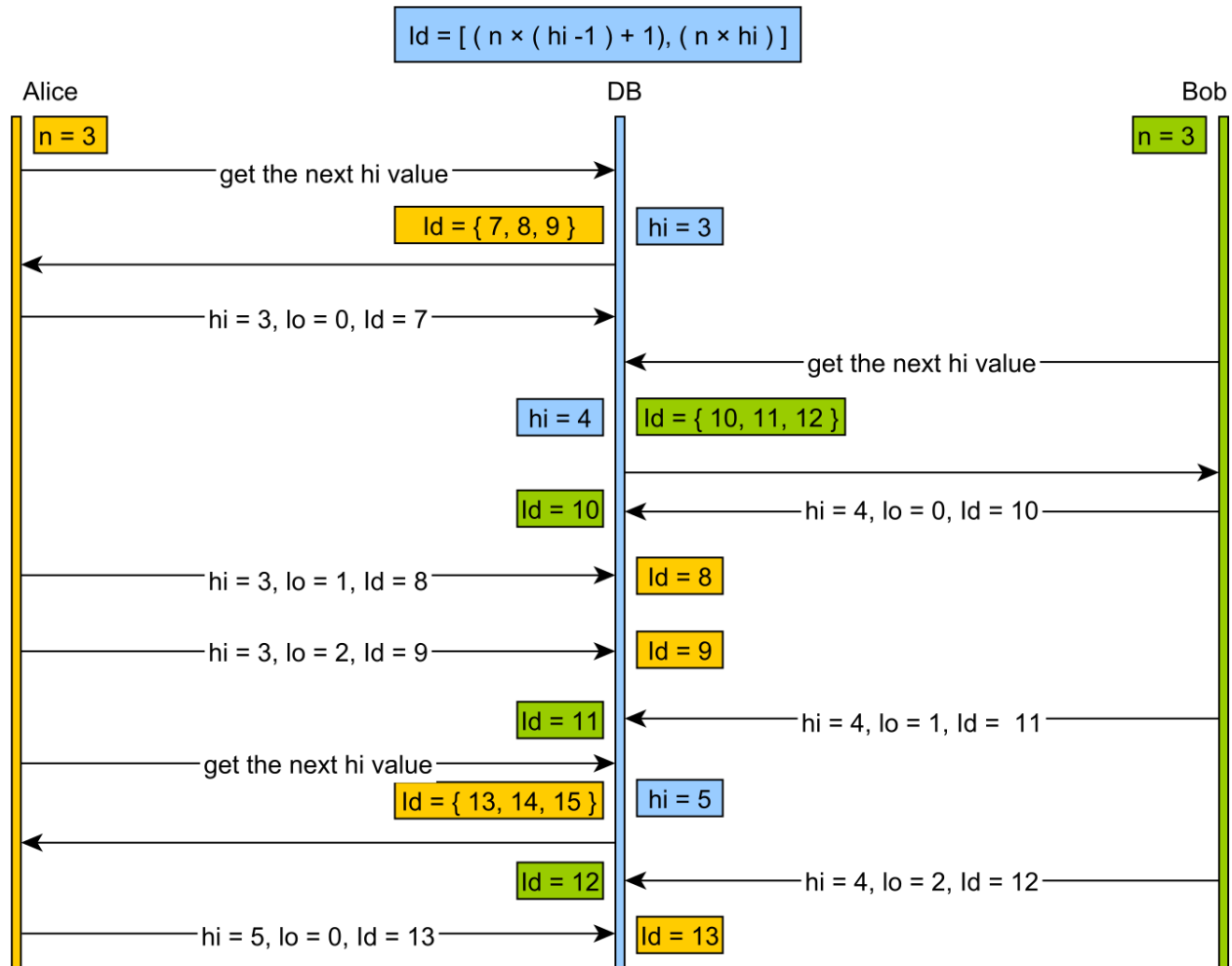


Figure 9.2: The hi/lo algorithm

1. The *hi* token is assigned either by the database sequence or the table generator, so two consecutive calls are guaranteed to see monotonically increasing values.
2. Once a *hi* token is retrieved, the *increment size* ( $n$  - the number of *\*lo\** entries) defines the range of identifier values a transaction can safely allocate. The identifiers range is bounded by the following interval:  $Id = [n \times (hi - 1) + 1, n \times hi]$  and the allocation is done as follows:
  - the current group values start from  $n \times (hi - 1) + 1$
  - the *lo* value is taken from the following interval:  $\{ \$\$ \} [0, n - 1] \{ \$\$ \}$
  - by adding the *lo* value to the initial group value, a unique identifier is obtained.
3. When all *lo* values are used, a new *hi* value is fetched, and the cycle continues.

The following example shows how the hi/lo algorithm works in practice. The entity is mapped as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "hilo")
    @GenericGenerator(
        name = "hilo",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "hilo")
        }
    )
    private Long id;
}
```

Because the increment size is 3, the following test inserts 4 entities to show the number of database sequence calls.

```
doInJPA(entityManager -> {
    for(int i = 0; i < 4; i++) {
        Post post = new Post();
        entityManager.persist(post);
    }
});
```

Running the previous test generates the following output:

```
CALL NEXT VALUE FOR hilo_sequence
CALL NEXT VALUE FOR hilo_sequence

INSERT INTO post (id) VALUES (1)
INSERT INTO post (id) VALUES (2)
INSERT INTO post (id) VALUES (3)
INSERT INTO post (id) VALUES (4)
```

The first sequence call is for the first three values, while the second one is generated when reaching the fourth entity that needs to be persisted. The more inserts a transaction requires, the better the performance gain from reducing the number of database sequence calls.



Unfortunately, this optimizer has a major limitation. Because the database sequence only assigns group values, all database clients must be aware of this algorithm. If the DBA must insert a row in the table above, he must use the hi/lo algorithm to determine the range of values that she can safely use.

For this reason, Hibernate offers other optimizer algorithms that are interoperable with external clients, unaware of the application-level optimization technique in use.

#### 9.2.3.4.2 The default sequence identifier generator

The JPA identifier generator strategy only specifies the identifier type and not the algorithm used for generating such identifiers.

For the sequence generator, considering the following JPA mapping:

```
@Id
@GeneratedValue(generator = "sequence", strategy=GenerationType.SEQUENCE)
@SequenceGenerator(name = "sequence", allocationSize = 3)
private Long id;
```

Hibernate chooses the `SequenceHiLoGenerator` when the `hibernate.id.new_generator_mappings` configuration property is false. This was the default setting for Hibernate 3 and 4. The legacy `SequenceHiLoGenerator` uses the hi/lo algorithm, and, if the allocation size is greater than one, database interoperability could be compromised (every insert must be done according to the hi/lo algorithm rules).

If the aforementioned configuration property is true (the default setting for Hibernate 5), then the JPA mapping above will use the `SequenceStyleGenerator` instead.

Unlike its previous predecessor, the `SequenceStyleGenerator` uses configurable identifier optimizer strategies, and the application developer can even supply its own optimization implementation.

### 9.2.3.4.3 The default table identifier generator

Just like with sequences, the JPA table generator mapping can use a legacy or an enhanced generator, depending on the current Hibernate configuration settings:

```
@Id
@GeneratedValue(generator = "table", strategy=GenerationType.TABLE)
@TableGenerator(name = "table", allocationSize = 3)
private Long id;
```

If the `hibernate.id.new_generator_mappings` configuration property is false, then Hibernate will choose the `MultipleHiLoPerTableGenerator`. This generator requires a single table for managing multiple identifiers, and just like `SequenceHiLoGenerator`, it also uses the hi/lo algorithm by default.

When the enhanced identifier generators are activated, Hibernate uses the `TableGenerator` instead, which can also take configurable optimizer strategies.

For both the enhanced sequence and the table identifier generator, Hibernate comes with the following built-in optimizers:

**Table 9.8: Hibernate identifier optimizers**

| Optimizer type | Implementation class | Description                                                                                                                     |
|----------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------|
| none           | NoopOptimizer        | Every identifier is fetched using a new roundtrip to the database                                                               |
| hi/lo          | HiLoOptimizer        | It allocates identifiers by using the legacy hi/lo algorithm                                                                    |
| pooled         | PooledOptimizer      | It is an enhanced version of the hi/lo algorithm which is interoperable with other systems unaware of this identifier generator |
| pooled-lo      | PooledLoOptimizer    | It is a variation of the pooled optimizer, the database sequence value representing the lo value instead of the hi one          |

By default, the `SequenceStyleGenerator` and `TableGenerator` identifier generators uses the `pooled` optimizer. If the `hibernate.id.optimizer.pooled.prefer_lo` configuration property is set to true, Hibernate will use the `pooled-lo` optimizer by default.

Both the `pooled` and the `pooled-lo` encode the database sequence value into the identifier range boundaries, so allocating a new value using the actual database sequence call does not interfere with the identifier generator allocation process.

#### 9.2.3.4.4 The pooled optimizer

The pooled optimizer can be configured as follows:

```
@Entity
public class Post {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "pooled")
    @GenericGenerator(
        name = "pooled",
        strategy = "org.hibernate.id.enhanced.SequenceStyleGenerator",
        parameters = {
            @Parameter(name = "sequence_name", value = "sequence"),
            @Parameter(name = "initial_value", value = "1"),
            @Parameter(name = "increment_size", value = "3"),
            @Parameter(name = "optimizer", value = "pooled")
        }
    )
    private Long id;
}
```

The increment size gives the range of values allocated by the sequence generator with one database roundtrip. Although it is inefficient to flush the Persistence Context after every persist method call, in this test, the flush outlines when the database sequence was called.

```
doInJPA(entityManager -> {
    for (int i = 0; i < 5; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
    entityManager.unwrap(Session.class).doWork(connection -> {
        try (Statement statement = connection.createStatement()) {
            statement.executeUpdate(
                "INSERT INTO post VALUES NEXT VALUE FOR sequence"
            );
        }
    });
    for (int i = 0; i < 3; i++) {
        entityManager.persist(new Post());
        entityManager.flush();
    }
});
```

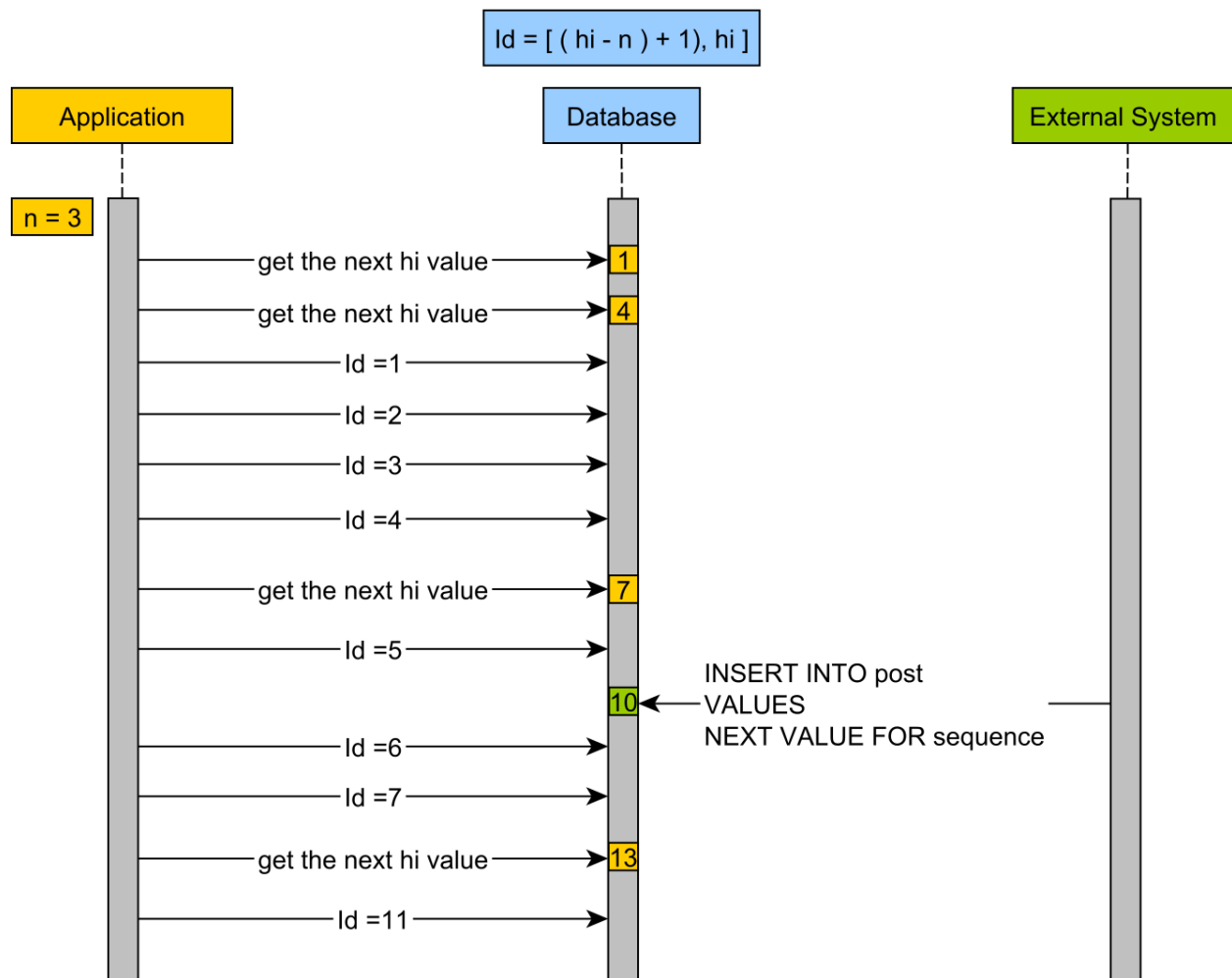


Figure 9.3: The pooled optimizer

If increment size ( $n$ ) is the number of identifiers within a range, the pooled optimizer will generate identifiers with the following formula:  $Id = [(hi - n) + 1, hi]$ .

- The first sequence call generates the *lo* value and the second one determines the *hi* value, so the first range of identifiers is {2, 3, 4}.
- When adding the 5th entity, the pooled optimizer calls the sequence again and obtains the next *hi* value, the next identifier range being {5, 6, 7}.
- After inserting the 5th entity, an external system adds a *post* row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifiers 6 and 7.
- The 8th entity requires a new sequence call, and so a new range is allocated {11, 12, 13}.

### 9.2.3.4.5 The pooled-lo optimizer

By changing the previous mapping to use the `pooled-lo` optimizer, the identifier generation changes as follows:

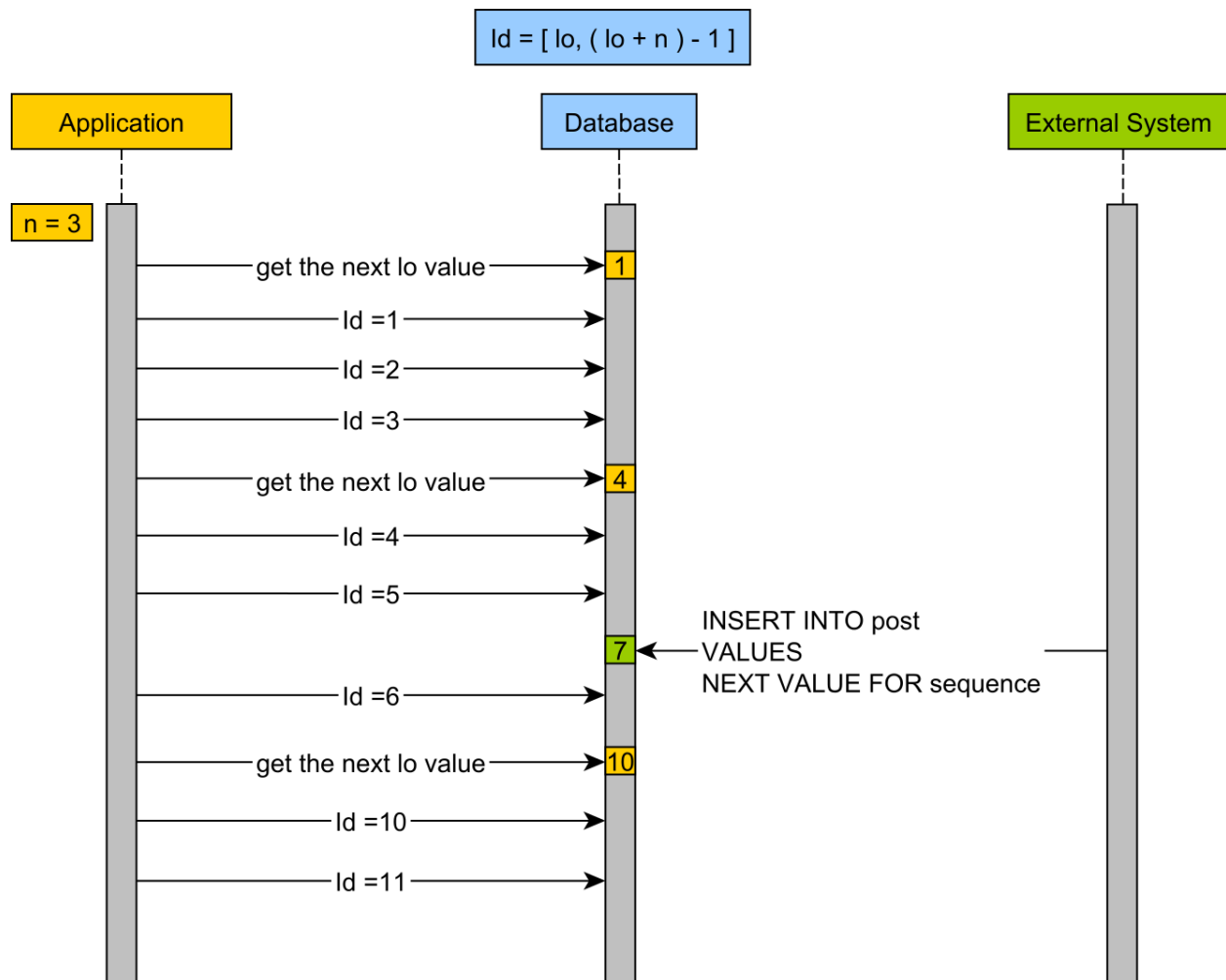


Figure 9.4: The pooled-lo optimizer

If increment size (`n`) is the number of identifiers within a range, the pooled-lo optimizer will generate identifiers with the following formula:  $Id = [lo, (lo + n) - 1]$ .

- The first sequence call generates the `lo` value, so the first range of identifiers is {1, 2, 3}.
- When adding the 4th entity, the pooled-lo optimizer calls the sequence and obtains the next `lo` value, the next identifier range being {4, 5, 6}.
- After inserting the 5th entity, an external system adds a `post` row and assigns the primary key with the value returned by the sequence call.
- The Hibernate application thread resumes and inserts the identifier 6.
- The 7th entity requires a new sequence call, and so a new range is allocated {10, 11, 12}.

### 9.2.3.5 Optimizer gain

To visualize the performance gain of using sequence and table generator optimizers, the following test measures the identifier allocation time when inserting 50 *post* entities and while varying the increment size (1, 5, 10, and 50).

#### 9.2.3.5.1 Sequence generator performance gain

When using a sequence generator with the default pooled optimizer, the following 99th percentile is being recorded:

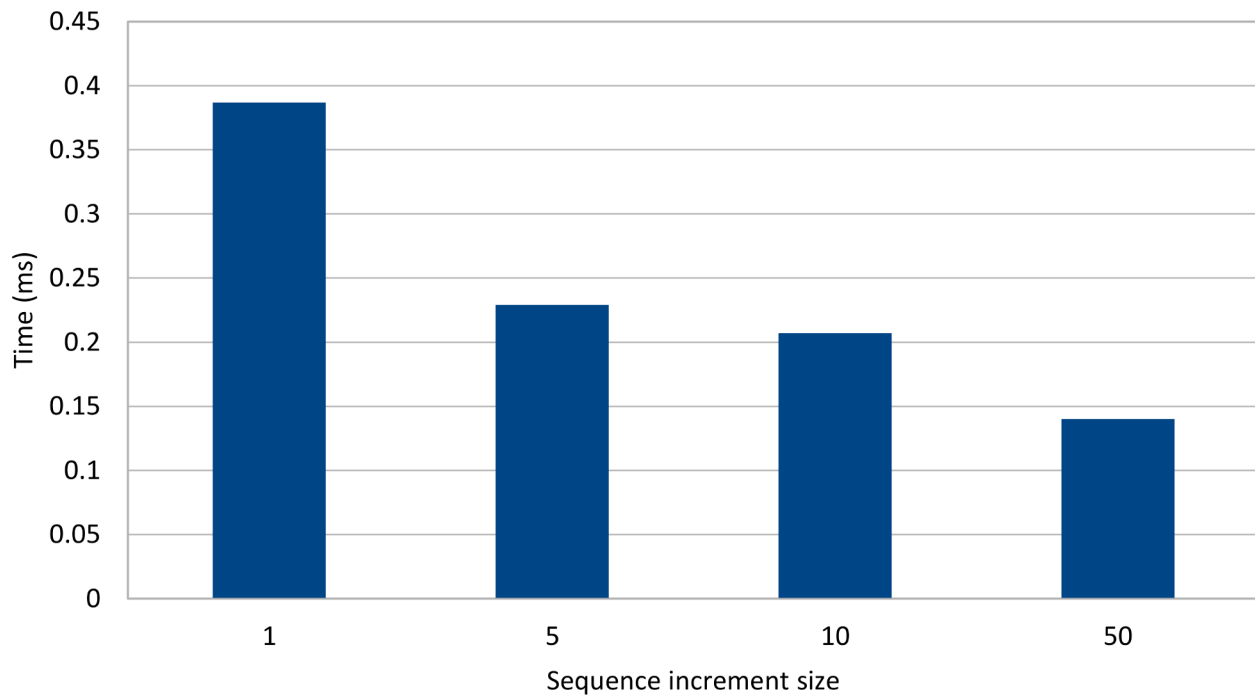


Figure 9.5: Sequence pooled optimizer gain



Database sequences are fast, but, even so, the pooled optimizer manages to reduce the identifier generation time considerably.

For write-intensive applications, the increment size needs to be adjusted according to the number of rows being inserted in one transaction.

### 9.2.3.5.2 Table generator performance gain

The same test suite is run against a table generator with a pooled optimizer, and the increment size also varies between 1, 5, 10, and 50. Because of the row-level locking and the extra database connection switch overhead, the table generator is less efficient than a database sequence.

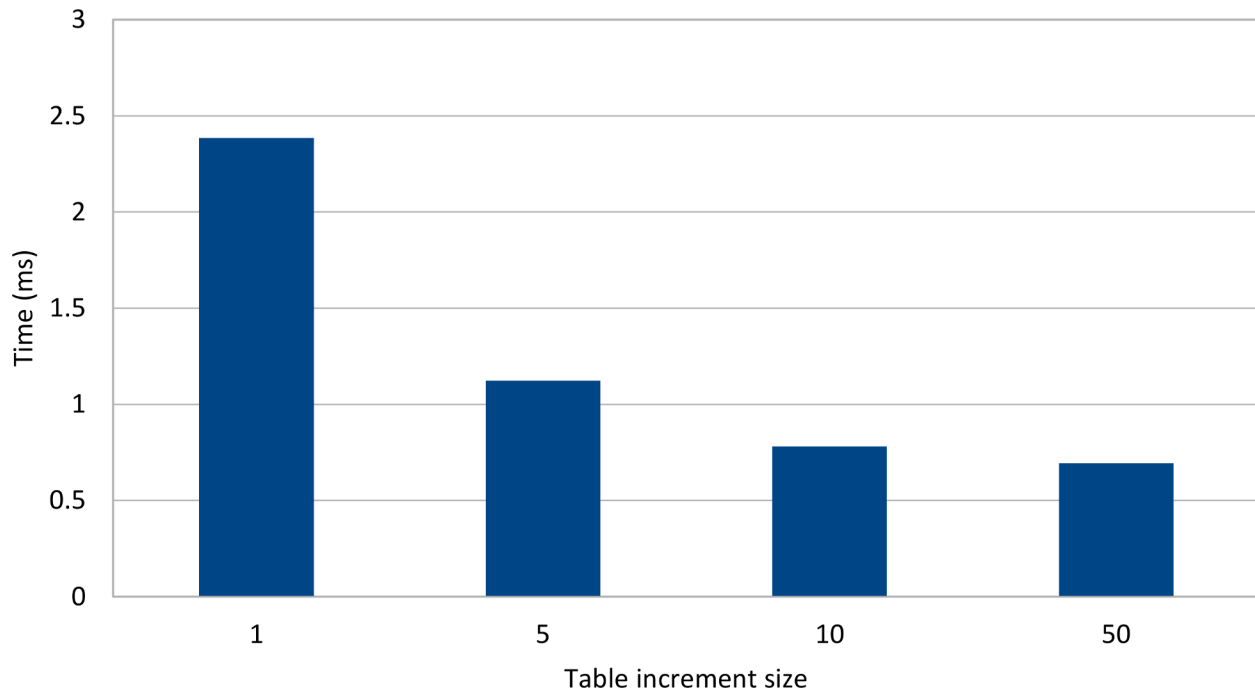


Figure 9.6: Table pooled optimizer gain

Just like with the database sequence, the pooled optimizer managed to reduce the time it took for assigning a new entity identifier.

### 9.2.3.6 Identifier generator performance

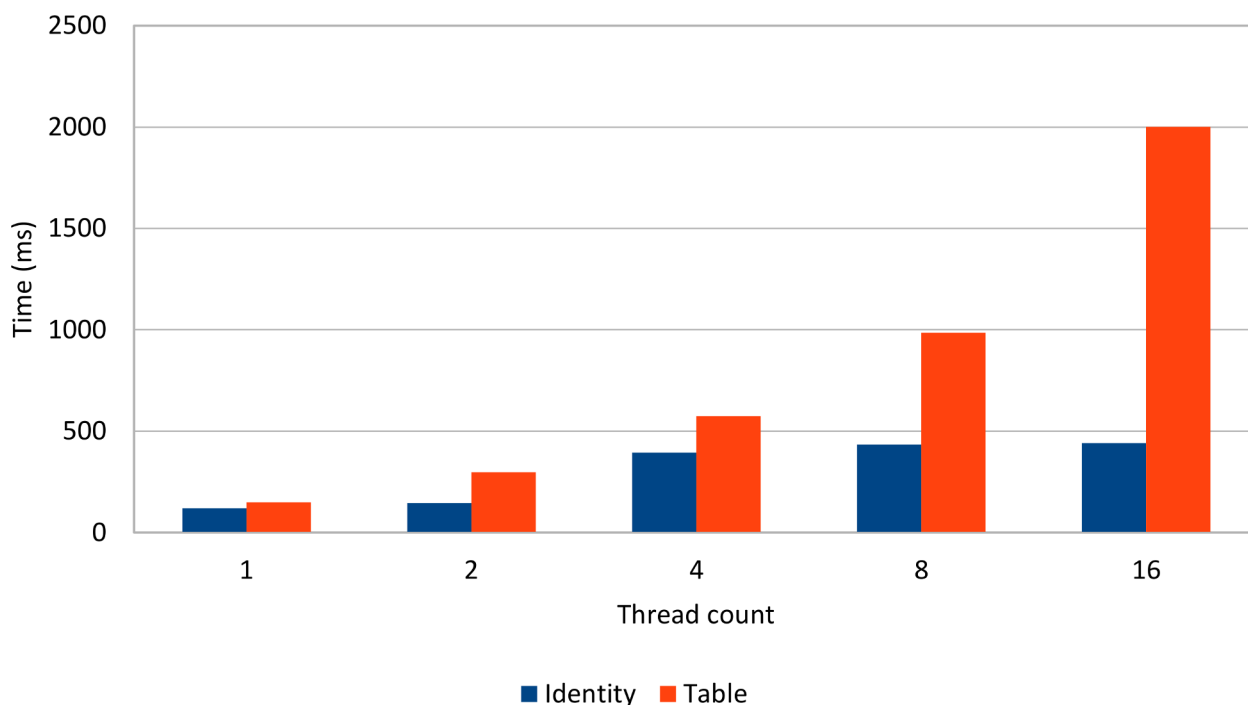
To evaluate the concurrency cost of each identifier generators, the following test measures the time it takes to insert 100 post entities when multiple running threads are involved. JDBC batching is enabled, and the connection pool is adjusted to accommodate the maximum number of database connection required (e.g. 32).



In reality, the application might not be configured with so many database connections, and the table generator connection acquisition cost might be even higher.

The first relational database system under test supports identity columns, so it is worth measuring how the identifier and the table generator compete. Unlike the previous test, this one measures the total time taken for inserting all entities, and not just the identifier allocation time interval.

Each test iteration increases contention by allocating more worker threads that need to execute the same database insert load.



**Figure 9.7: Identity vs. Table**

Even if it cannot benefit from JDBC batching, the identity generator still manages to outperform the table generator, which uses a pooled optimizer with an increment size of 100.

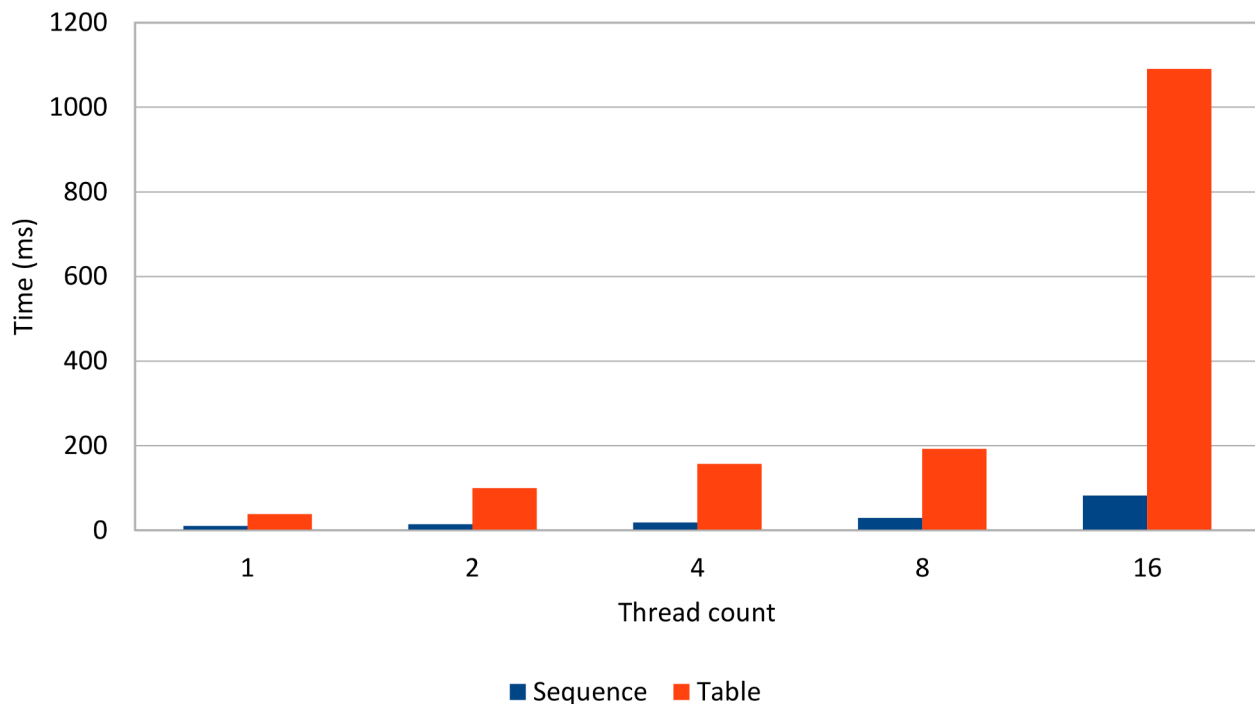


The more threads are used, the less efficient the table generator becomes. On the other hand, identity columns scale much better with more concurrent transactions.

Even if it does not support JDBC batching, native identity columns are still a valid choice, and, in future, Hibernate might even support batch inserts for those as well.

The gap between the sequence and the table generator is even higher because, just like the table generator, the sequence generator can also take advantage of the pooled optimizer as well as JDBC batch inserts.

Running the same test against a relational database supporting sequences, the following results are being recorded:



**Figure 9.8: Sequence vs. Table**

The performance impact of the table generator becomes noticeable in high concurrent environments, where the row-level locking and the database connection switch introduces a serial execution.



Because they use lightweight synchronization mechanisms, database sequences scale better than row-level locking concurrency control mechanisms.

Database sequences are the most efficient Hibernate identifier choice, allowing sequence call optimizers and without compromising JDBC batching.

## 10. Relationships

In a relational database, associations are formed by correlating rows belonging to different tables. A relationship is established when a child table defines a foreign key referencing the primary key of its parent table. Every database association is built on top of foreign keys, resulting three table relationship types:

- *one-to-many* is the most common relationship, and it associates a row from a parent table to multiple rows in a child table.
- *one-to-one* requires the child table Primary Key to be associated via a Foreign Key with the parent table Primary Key column.
- *many-to-many* requires a link table containing two Foreign Key columns that reference the two different parent tables.

The following diagram depicts all these three table relationships:

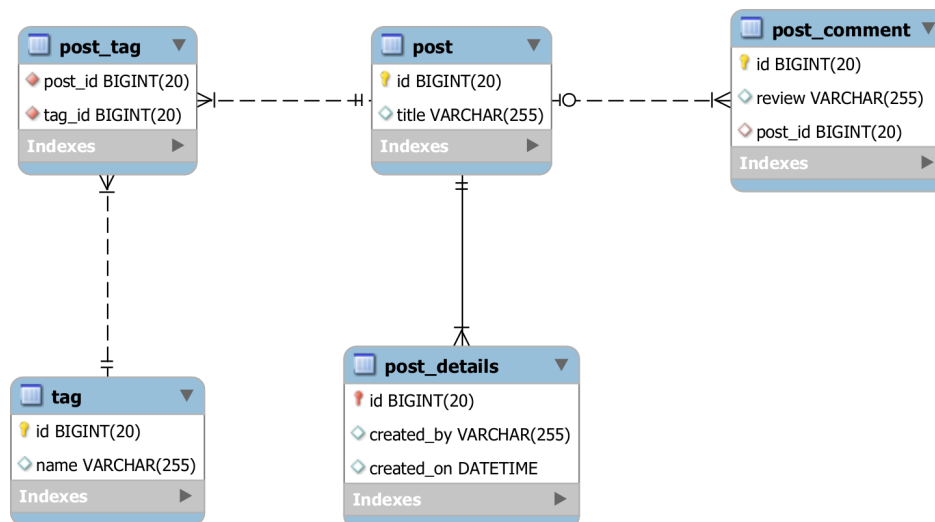


Figure 10.1: Table relationships

The `post` table has a one-to-many relationship with the `post_comment` table because a `post` row might be referenced by multiple comments. The one-to-many relationship is established through the `post_id` column which has a foreign key referencing the `post` table primary key. Because a `post_comment` cannot exist without a `post`, the `post` is the parent-side while the `post_comment` is the child-side.

The `post` table has a one-to-one relationship with the `post_details`. Like the one-to-many association, the one-to-one relationship involves two tables and a foreign key. The foreign key has a uniqueness constraint, so only one child row can reference a parent record.

The `post` and the `tag` are both independent tables and neither one is a child of the other. A `post` can feature several `tag(s)`, while a `tag` can also be associated with multiple `post(s)`. This is a typical many-to-many association, and it requires a junction table to resolve the child-side of these two parent entities. The junction table requires two foreign keys referencing the two parent tables.



The foreign key is, therefore, the most important construct in building a table relationship, and, in a relational database, the child-side controls a table relationship.

In a relational database, the foreign key is associated with the child-side only. For this reason, the parent-side has no knowledge of any associated child relationships, and, from a mapping perspective, table relationships are always unidirectional (the child foreign key references the parent primary key).

## 10.1 Relationship types

When mapping a JPA entity, besides the underlying table columns, the application developer can map entity relationships either in one direction or in a bidirectional way. This is another impedance mismatch between the object-oriented Domain Model and relational database system because, when using an ORM tool, the parent and the child-side can reference each other.

A relationship is unidirectional if only one entity side maps the table relationship and is bidirectional if the table relationship can be navigated in both directions (either from the entity parent-side or the child-side).

To properly represent both sides of an entity relationship, JPA defines four association mapping constructs:

- `@ManyToOne` represents the child-side (where the foreign key resides) in a database one-to-many table relationship.
- `@OneToMany` is associated with the parent-side of a one-to-many table relationship.
- `@ElementCollection` defines a one-to-many association between an entity and multiple value types (basic or embeddable).
- `@OneToOne` is used for both the child-side and the parent-side in a one-to-one table relationship.
- `@ManyToMany` mirrors a many-to-many table relationship.

Because the entity relationship choice has a considerable impact on the overall application performance, this chapter analyzes the data access operation efficiency of all these JPA associations.

## Mapping collections

In a relational database, all table relationships are constructed using foreign keys and navigated through SQL queries. JPA allows mapping both the foreign key side (the child entity has a reference to its parent), as well as the parent side (the parent entity has one or more child entities).

Although `@OneToMany`, `@ManyToMany` OR `@ElementCollection` are convenient from a data access perspective (entity state transitions can be cascaded from parent entities to children), they are definitely not free of cost. The price for reducing data access operations is paid in terms of result set fetching flexibility and performance. A JPA collection, either of entities or value types (basic or embeddables), binds a parent entity to a query that usually fetches all the associated child records. Because of this, the entity mapping becomes sensitive to the number of child entries.

If the children count is relatively small, the performance impact of always retrieving all child entities might be unnoticeable. However, if the number of child records grows too large, fetching the entire children collection may become a performance bottleneck. Unfortunately, the entity mapping is done during the early phases of a project development, and the development team might be unaware of the number of child records a production system exhibits.

Not just the mere size can be problematic, but also the number of attributes of the child entity. Because entities are usually fetched as a whole, the result set is, therefore, proportional to the number of columns the child table contains. Even if a collection is fetched lazily, Hibernate might still require to fully load each entity when the collection is accessed for the first time. Although Hibernate supports extra lazy collection fetching, this is only a workaround and does not address the root problem.

Alternatively, every collection mapping can be replaced by a data access query, which can use a SQL projection that is tailored to the data requirements of each business use case. This way, the query can take business case specific filtering criteria. Although JPA 2.1 does not support dynamic collection filtering, Hibernate offers Persistence Context-bound collection `Filters`.



When handling large data sets, it is good practice to limit the result set size, both for UI (to increase responsiveness) or batch processing tasks (to avoid long running transactions). Just because JPA supports collection mapping, it does not mean they are mandatory for every domain model mapping. Until there is a clear understanding of the number of child records (or if there is even need to fetch child entities entirely), it is better to postpone the collection mapping decision. For high-performance systems, a data access query is often a much more flexible alternative.

## 10.2 @ManyToOne

The `@ManyToOne` relationship is the most common JPA association, and it maps exactly to the one-to-many table relationship. When using a `@ManyToOne` association, the underlying foreign key is controlled by the child-side, no matter the association is unidirectional or bidirectional.

This section focuses on unidirectional `@ManyToOne` relationships only, the bidirectional case being further discussed with the `@OneToMany` relationship. In the following example, the `Post` entity represents the parent-side, while the `PostComment` is the child-side.

As already mentioned, the JPA entity relationship diagram matches exactly the one-to-many table relationship.

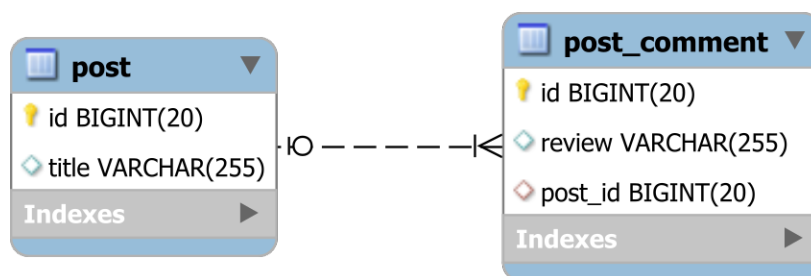


Figure 10.2: The one-to-many table relationship

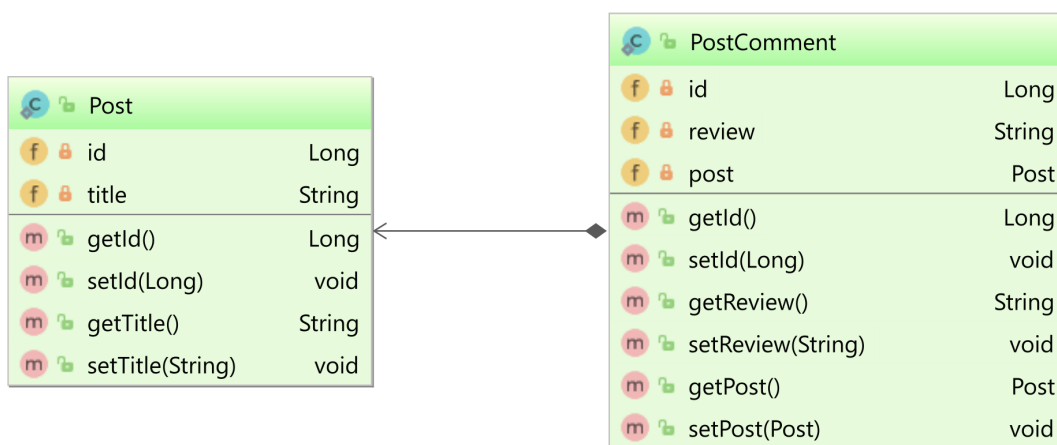


Figure 10.3: `@ManyToOne` relationship

Instead of mapping the `post_id` foreign key column, the `PostComment` uses a `@ManyToOne` relationship to the parent `Post` entity. The `PostComment` can be associated with an existing `Post` object reference, and the `PostComment` can also be fetched along with the `Post` entity.

```
@ManyToOne
@JoinColumn(name = "post_id")
private Post post;
```

Hibernate translates the internal state of the `@ManyToOne` `Post` object reference to the `post_id` foreign key column value.

If the `@ManyToOne` attribute is set to a valid `Post` entity reference:

```
Post post = entityManager.find(Post.class, 1L);
PostComment comment = new PostComment("My review");
comment.setPost(post);
entityManager.persist(comment);
```

Hibernate will generate an insert statement populating the `post_id` column with the identifier of the associated `Post` entity.

```
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My review', 2)
```

If the `Post` attribute is later set to null:

```
comment.setPost(null);
```

The `post_id` column will also be updated with a `NULL` value:

```
UPDATE post_comment SET post_id = NULL, review = 'My review' WHERE id = 2
```



Because the `@ManyToOne` association controls the foreign key directly, the automatically generated DML statements are very efficient.

Actually, the best-performing JPA associations always rely on the child-side to translate the JPA state to the foreign key column value.

This is one of the most important rules in JPA relationship mapping, and it will be further emphasized for `@OneToMany`, `@OneToOne` and even `@ManyToMany` associations.

## 10.3 @OneToMany

While the `@ManyToOne` association is the most natural mapping of the one-to-many table relationship, the `@OneToMany` association can also mirror this database relationship, but only when being used as a bidirectional mapping. A unidirectional `@OneToMany` association uses an additional junction table, which no longer fits the one-to-many table relationship semantics.

### 10.3.1 Bidirectional @OneToMany

The bidirectional `@OneToMany` association has a matching `@ManyToOne` child-side mapping that controls the underlying one-to-many table relationship. The parent-side is mapped as a collection of child entities.

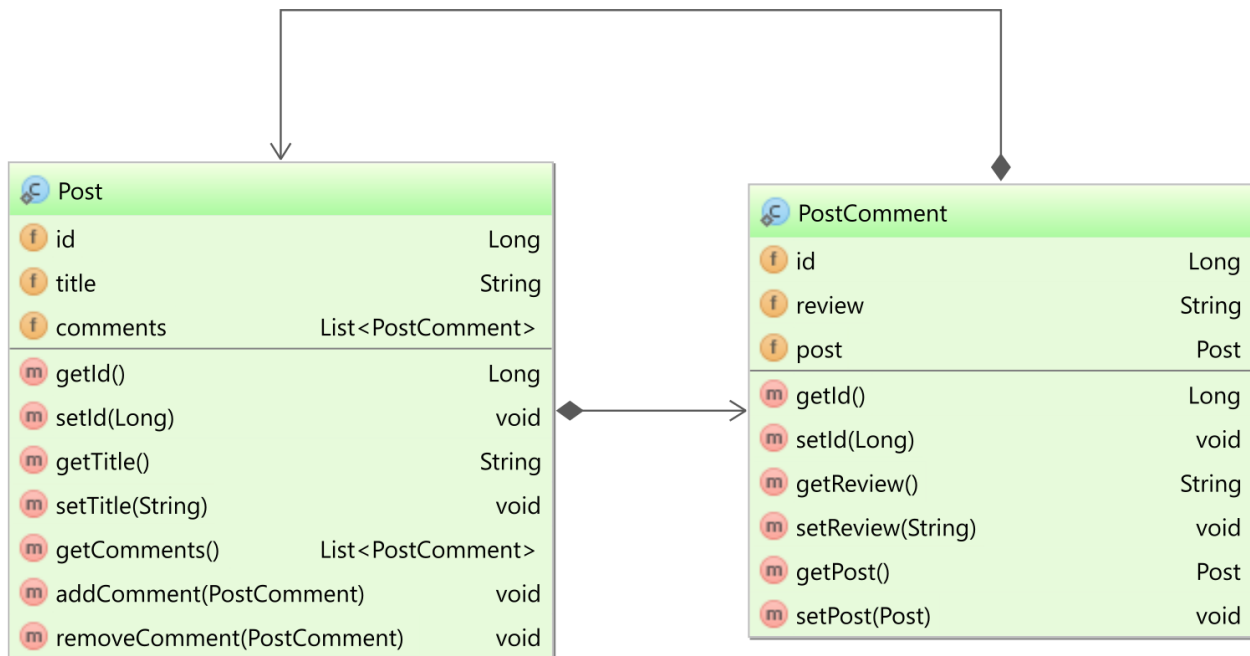


Figure 10.4: Bidirectional `@OneToMany` relationship

In a bidirectional association, only one side can control the underlying table relationship. For the bidirectional `@OneToMany` mapping, it is the child-side `@ManyToOne` association in charge of keeping the foreign key column value in sync with the in-memory Persistence Context. This is the reason why the bidirectional `@OneToMany` relationship must define the `mappedBy` attribute, indicating that it only mirrors the `@ManyToOne` child-side mapping.

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();
```



Even if the child-side is in charge of synchronizing the entity state changes with the database foreign key column value, a bidirectional association must always have both the parent-side and the child-side in sync.

To synchronize both ends, it is practical to provide parent-side helper methods that add/remove child entities.

```
public void addComment(PostComment comment) {
    comments.add(comment);
    comment.setPost(this);
}

public void removeComment(PostComment comment) {
    comments.remove(comment);
    comment.setPost(null);
}
```

One of the major advantages of using a bidirectional association is that entity state transitions can be cascaded from the parent entity to its children. In the following example, when persisting the parent `Post` entity, all the `PostComment` child entities are persisted as well.

```
Post post = new Post("First post");

PostComment comment1 = new PostComment("My first review");
post.addComment(comment1);
PostComment comment2 = new PostComment("My second review");
post.addComment(comment2);

entityManager.persist(post);
```

```
INSERT INTO post (title, id) VALUES ('First post', 1)
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My first review', 2)
INSERT INTO post_comment (post_id, review, id) VALUES (1, 'My second review', 3)
```

When removing a comment from the parent-side collection:

```
post.removeComment(comment1);
```

The orphan removal attribute instructs Hibernate to generate a delete DML statement on the targeted child entity:

```
DELETE FROM post_comment WHERE id = 2
```

## Equality-based entity removal

The helper method for the child entity removal relies on the underlying child object equality for matching the collection entry that needs to be removed.

If the application developer does not choose to override the default `equals` and `hashCode` methods, the `java.lang.Object` identity-based equality is going to be used. The problem with this approach is that the application developer must supply a child entity object reference that is contained in the current child collection.

Sometimes child entities are loaded in one web request and saved in a `HttpSession` or a `Stateful Enterprise Java Bean`. Once the `Persistence Context`, which loaded the child entity is closed, the entity becomes detached. If the child entity is sent for removal into a new web request, the child entity must be reattached or merged into the current `Persistence Context`. This way, if the parent entity is loaded along with its child entities, the removal operation will work properly since the removing child entity is already managed and contained in the children collection.

If the entity has not changed, reattaching this child entity will be redundant and so the `equals` and the `hashCode` methods must be overridden to express equality in terms of a unique business key. In case the child entity has a `@NaturalId` or a unique attribute set, the `equals` and the `hashCode` methods can be implemented on top of that. Assuming the `PostComment` entity has the following two columns whose combination render a unique business key, the equality contract can be implemented as follows:

```
private String createdBy;

@Temporal(TemporalType.TIMESTAMP)
private Date createdOn = new Date();

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    PostComment that = (PostComment) o;
    return Objects.equals(createdBy, that.createdBy) &&
        Objects.equals(createdOn, that.createdOn);
}

@Override
public int hashCode() {
    return Objects.hash(createdBy, createdOn);
}
```

## Identifier-based equality

The `java.lang.Object.equalsa` method Javadoc demands the strategy be *reflexive*, *symmetric*, *transitive*, and *consistent*.

While the first three equality properties (*reflexive*, *symmetric*, *transitive*) are easier to achieve, especially with the `java.util.Objectsb` `equals` and `hashCode` utilities, *consistency* requires more diligence.



For a JPA or Hibernate entity, *consistency* means that the equality result is *reflexive*, *symmetric* and *transitive* across all **entity state transitions** (e.g. *new/transient*, *managed*, *detached*, *removed*).

If the entity has a `@NaturalId` attribute, then ensuring *consistency* is simple since the natural key is assigned even from the transient state, and this attribute never changes afterward. However, not all entities have a natural key to use for equality checks, so another table column must be used instead.

Luckily, most database tables have a primary key, which uniquely identifies each row of a particular table. The only caveat is to ensure *consistency* across all entity state transitions.

A naive implementation would look like this:

```
@Entity
public class Post {

    @Id @GeneratedValue
    private Long id;

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Post)) return false;
        return Objects.equals(id, ((Post) o).getId());
    }

    @Override
    public int hashCode() {
        return Objects.hash(id);
    }
}
```

<sup>a</sup><https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html#equals-java.lang.Object->

<sup>b</sup><https://docs.oracle.com/javase/7/docs/api/java/util/Objects.html>

Now, when running the following test case:

```
Set<Post> tuples = new HashSet<>();
tuples.add(entity);
assertTrue(tuples.contains(entity));

doInJPA(entityManager -> {
    entityManager.persist(entity);
    entityManager.flush();
    assertTrue(tuples.contains(entity));
});
```

The final assertion will fail because the entity can no longer be found in the `Set` collection since the new identifier value is associated with a different `Set` bucket than the one where the entity got stored in.

To fix it, the `equals` and `hashCode` methods must be changed as follows:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Post)) return false;
    return id != null && id.equals(((Post) o).getId());
}

@Override
public int hashCode() {
    return 31;
}
```

When the entity identifier is null, equality can only be guaranteed for the same object references. Otherwise, no transient object is equal to any other transient or persisted object. That's why the `equals` method above skips the identifier check if the current object has a null identifier value.

Using a constant `hashCode` value solves the previous bucket-related problem associated with `Set(s)` or `Map(s)` because, this time, only a single bucket is going to be used. Although in general, using a single `Set` bucket is not very efficient for large collection of objects, in this particular use case, this workaround is valid since managed collections should be rather small to be efficient. Otherwise, fetching a `@OneToMany Set` with thousands of entities is orders of magnitude more costly than the one-bucket search penalty.

When using `List(s)`, the constant `hashCode` value is not an issue at all, and, for bidirectional collections, the Hibernate-internal `PersistentList(s)` are more efficient than `PersistentSet(s)`.



The bidirectional `@OneToMany` association generates efficient DML statements because the `@ManyToOne` mapping is in charge of the table relationship. Because it simplifies data access operations as well, the bidirectional `@OneToMany` association is worth considering when the size of the child records is relatively low.

### 10.3.2 Unidirectional `@OneToMany`

The unidirectional `@OneToMany` association is very tempting because the mapping is simpler than its bidirectional counterpart. Because there is only one side to take into consideration, there is no need for helper methods and the mapping does not feature a `mappedBy` attribute either.

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();
```

Unfortunately, in spite its simplicity, the unidirectional `@OneToMany` association is less efficient than the unidirectional `@ManyToOne` mapping or the bidirectional `@OneToMany` association.

Against any intuition, the unidirectional `@OneToMany` association does not map to a one-to-many table relationship. Because there is no `@ManyToOne` side to control this relationship, Hibernate uses a separate junction table to manage the association between a parent row and its child records.



Figure 10.5: The `@OneToMany` table relationship

The table `post_post_comment` has two foreign key columns, which reference both the parent-side row (the `Post_id` column is a foreign key to the `post` table primary key) and the child-side entity (the `comments_id` references the primary key of the `post_comment` table).

Without going into analyzing the associated data access operations, it is obvious that joining three tables is less efficient than joining just two. Because there are two foreign keys, there need to be two indexes (instead of one), so the index memory footprint increases. However, since this is a regular table mapping for a many-to-many relationship, the extra table and the increased memory footprint are not even the biggest performance issue. The algorithm for managing the collection state is what makes any unidirectional `@OneToMany` association less attractive.

Considering there is a `Post` entity with two `PostComment` child records, obtained by running the following example:

```
Post post = new Post("First post");

post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));

entityManager.persist(post);
```

While for a bidirectional `@OneToMany` association there were three child rows being added, the unidirectional association requires three additional inserts for the junction table records.

```
INSERT INTO post (title, id) VALUES ('First post', 1)
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 2)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
```

When removing the first element of the collection:

```
post.getComments().remove(0);
```

Hibernate generates the following DML statements:

```
DELETE FROM post_post_comment WHERE Post_id = 1
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
DELETE FROM post_comment WHERE id = 2
```

First, all junction table rows associated with the parent entity are deleted, and then the remaining in-memory records are added back again. The problem with this approach is that instead of a single junction table remove operation, the database has way more DML statements to execute.

Another problem is related to indexes. If there is an index on each foreign key column (which is the default for many relational databases), the database engine must delete the associated index entries only to add back the remaining ones. The more elements a collection has, the less efficient a remove operation gets.



The unidirectional `@OneToMany` relationship is less efficient both for reading data (three joins are required instead of two), as for adding (two tables must be written instead of one) or removing (entries are removed and added back again) child entries.

### 10.3.3 Ordered unidirectional `@OneToMany`

If the collection can store the index of every collection element, the unidirectional `@OneToMany` relationship may benefit for some element removal operations. First, an `@OrderColumn` annotation must be defined along the `@OneToMany` relationship mapping:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@OrderColumn(name = "entry")
private List<PostComment> comments = new ArrayList<>();
```

At the database level, the `entry` column is included in the junction table.

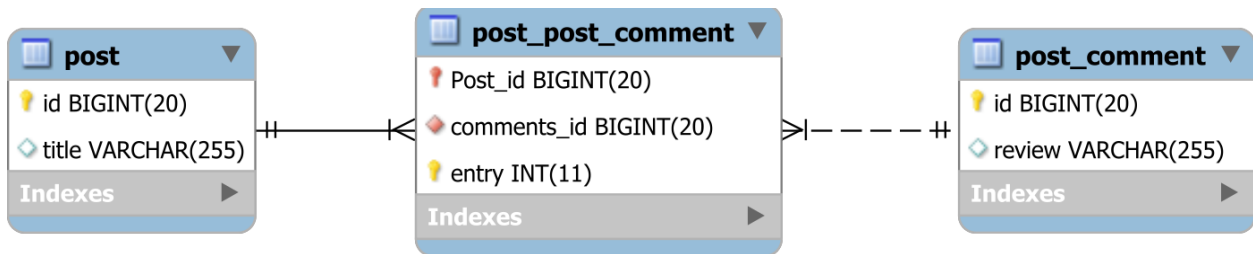


Figure 10.6: The unidirectional `@OneToMany` with an `@OrderColumn`



It is better not to mistake the `@OrderColumn` with the `@OrderBy` JPA annotation. While the former allows the JPA provider to materialize the element index into a dedicated database column so that the collection is sorted using an `ORDER BY` clause, the latter does the sorting at runtime based on the ordering criteria provided by the `@OrderBy` annotation.

Considering there are three `PostComment` entities added for a given `Post` parent entity:

```
post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));
```

The index of every collection element is going to be stored in the `entry` column of the junction table:

```
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 0, 2)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 1, 3)
INSERT INTO post_post_comment (Post_id, entry, comments_id) VALUES (1, 2, 4)
```

When removing elements from the tail of the collection:

```
post.getComments().remove(2);
```

Hibernate only requires a single junction table delete statement:

```
DELETE FROM post_post_comment WHERE Post_id = 1 AND entry = 2
DELETE FROM post_comment WHERE id = 4
```

Unfortunately, this optimization does not hold for entries that are located towards the head of the collection. So, if we remove the first element:

```
post.getComments().remove(0);
```

Hibernate deletes the last entry associated with the parent row from the junction table, and then it updates the remaining entries to preserve the same element ordering as the in-memory collection snapshot:

```
DELETE FROM post_post_comment WHERE Post_id = 1 AND entry = 2
UPDATE post_post_comment SET comments_id = 4 WHERE Post_id = 1 AND entry = 1
UPDATE post_post_comment SET comments_id = 3 WHERE Post_id = 1 AND entry = 0
DELETE FROM post_comment WHERE id = 2
```



If the unidirectional `@OneToMany` collection is used like a stack and elements are always removed from the collection tail, the remove operations will be more efficient when using an `@OrderColumn`. But the closer an element is to the head of the list, the more update statements must be issued, and the additional updates have an associated performance overhead.

### 10.3.4 @OneToMany with @JoinColumn

JPA 2.0 added support for mapping the `@OneToMany` association with a `@JoinColumn` so that it can map the one-to-many table relationship. With the `@JoinColumn`, the `@OneToMany` association controls the child table foreign key, so there is no need for a junction table.

On the JPA side, the class diagram is identical to the aforementioned unidirectional `@OneToMany` relationship, and the only difference is the JPA mapping which takes the additional `@JoinColumn`:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "post_id")
private List<PostComment> comments = new ArrayList<>();
```

When adding three `PostComment` entities, Hibernate generates the following SQL statements:

```
post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));
```

```
INSERT INTO post_comment (review, id) VALUES ('My first review', 2)
INSERT INTO post_comment (review, id) VALUES ('My second review', 3)
INSERT INTO post_comment (review, id) VALUES ('My third review', 4)
UPDATE post_comment SET post_id = 1 WHERE id = 2
UPDATE post_comment SET post_id = 1 WHERE id = 3
UPDATE post_comment SET post_id = 1 WHERE id = 4
```

Besides the regular insert statements, Hibernate issues three update statements for setting the `post_id` column on the newly inserted child records. The update statements are generated by the Hibernate-internal `CollectionRecreateAction` which tries to preserve the element order whenever the collection state changes. In this particular case, the `CollectionRecreateAction` should not be scheduled for execution, however, as of writing (Hibernate 5.2.3), this issue still replicates.

When deleting the last element of the collection:

```
post.getComments().remove(2);
```

Hibernate generates the following SQL statements:

```
UPDATE post_comment SET post_id = null WHERE post_id = 1 AND id = 4
DELETE from post_comment WHERE id = 4
```

Again, there is an additional update statement associated with the child removal operation. When a child entity is removed from the parent-side collection, Hibernate sets the child table foreign key column to null. Afterward, the orphan removal logic kicks in, and it triggers a delete statement against the disassociated child entity.

Unlike the regular `@OneToMany` association, the `@JoinColumn` alternative is consistent in regard to the collection entry position that is being removed. So, when removing the first element of the collection:

```
post.getComments().remove(0);
```

Hibernate still generates an additional update statement:

```
UPDATE post_comment SET post_id = null WHERE post_id = 1 AND id = 2
DELETE from post_comment WHERE id = 2
```

### Bidirectional `@OneToMany` with `@JoinColumn` relationship

The `@OneToMany` with `@JoinColumn` association can also be turned into a bidirectional relationship, but it requires instructing the child-side to avoid any insert and update synchronization:

```
@ManyToOne
@JoinColumn(name = "post_id", insertable = false, updatable = false)
private Post post;
```



The redundant update statements are generated for both the unidirectional and the bidirectional association, so the most efficient foreign key mapping is the `@ManyToOne` association.

### 10.3.5 Unidirectional @OneToMany Set

All the previous examples were using `List(s)`, but Hibernate supports `Set(s)` as well. For the next exercise, the `PostComment` entity uses the following mapping:

```
@Entity(name = "PostComment") @Table(name = "post_comment")
public class PostComment {

    @Id @GeneratedValue
    private Long id;

    private String slug;

    private String review;

    public PostComment() {
        byte[] bytes = new byte[8];
        ByteBuffer.wrap(bytes).putDouble(Math.random());
        slug = Base64.getEncoder().encodeToString(bytes);
    }

    public PostComment(String review) {
        this();
        this.review = review;
    }

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostComment comment = (PostComment) o;
        return Objects.equals(slug, comment.getSlug());
    }

    @Override
    public int hashCode() {
        return Objects.hash(slug);
    }
}
```

This time, the `PostComment` entity uses a `slug` attribute which provide a way to uniquely identify each comment belonging to a given `Post` entity.

Because the `PostComment` references are going to be stored in a `java.util.Set`, it is best to override the `equals` and `hashCode` `Object` methods according to the entity business key. In this particular example, the `PostComment` does not have any meaningful business key, so the `slug` attribute is used for the equality checks.

The parent `Post` entity has a unidirectional `@OneToMany` association that uses a `java.util.Set`:

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
private Set<PostComment> comments = new HashSet<>();
```

When adding three `PostComment` entities:

```
post.getComments().add(new PostComment("My first review"));
post.getComments().add(new PostComment("My second review"));
post.getComments().add(new PostComment("My third review"));
```

Hibernate generates the following SQL statements:

```
INSERT INTO post_comment (review, slug, id)
VALUES ('My second review', 'P+HLCF25scI=', 2)
INSERT INTO post_comment (review, slug, id)
VALUES ('My first review', 'P9y80GLTCyg=', 3)
INSERT INTO post_comment (review, slug, id)
VALUES ('My third review', 'P+fWF+Ck/LY=', 4)

INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 2)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 3)
INSERT INTO post_post_comment (Post_id, comments_id) VALUES (1, 4)
```

The remove operation is much more effective this time because the collection element order needs not be enforced anymore.

When removing the `PostComment` child entities:

```
for(PostComment comment: new ArrayList<>(post.getComments())) {
    post.getComments().remove(comment);
}
```

Hibernate generates one statement for removing the junction table entries and three delete statements for the associated `post_comment` records.

```
DELETE FROM post_post_comment WHERE Post_id = 1
```

```
DELETE FROM post_comment WHERE id = 2
```

```
DELETE FROM post_comment WHERE id = 3
```

```
DELETE FROM post_comment WHERE id = 4
```

To avoid using a secondary table, the `@OneToMany` mapping can use the `@JoinColumn` annotation.

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "post_id")
private Set<PostComment> comments = new HashSet<>();
```

Upon inserting three `PostComment` entities, Hibernate generates the following statements:

```
INSERT INTO post_comment (review, slug, id)
VALUES ('My third review', 'P8pcnLprqcQ=', 2)
INSERT INTO post_comment (review, slug, id)
VALUES ('My second review', 'P+Gau1+Hhxs=', 3)
INSERT INTO post_comment (review, slug, id)
VALUES ('My first review', 'P+kr9LOQTK0=', 4)
```

```
UPDATE post_comment SET post_id = 1 WHERE id = 2
```

```
UPDATE post_comment SET post_id = 1 WHERE id = 3
```

```
UPDATE post_comment SET post_id = 1 WHERE id = 4
```

When deleting all three `PostComment` entities, the generated statements look like this:

```
UPDATE post_comment SET post_id = null WHERE post_id = 1
```

```
DELETE FROM post_comment WHERE id = 2
```

```
DELETE FROM post_comment WHERE id = 3
```

```
DELETE FROM post_comment WHERE id = 4
```



Although it is an improvement over the unidirectional unordered or ordered `List`, the unidirectional `Set` is still less efficient than the bidirectional `@OneToMany` association.

## 10.4 @ElementCollection

Although it is not an entity association type, the `@ElementCollection` is very similar to the unidirectional `@OneToMany` relationship. To represent collections of basic types (e.g. `String`, `int`, `BigDecimal`) or embeddable types, the `@ElementCollection` must be used instead. If the previous associations involved multiple entities, this time, there is only a single `Post` entity with a collection of `String` comments.

Value types inherit the persistent state from their parent entities, so their lifecycle is also bound to the owner entity. Any operation against the entity collection is going to be automatically materialized into a DML statement.

From a database perspective, there is one child table holding both the foreign key column and the collection element value.

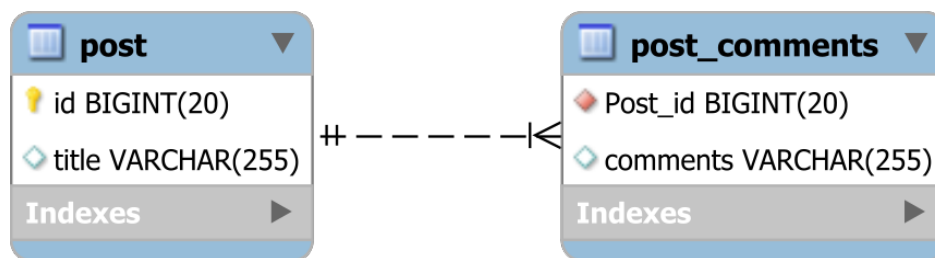


Figure 10.7: The `@ElementCollection` table relationship

### 10.4.1 @ElementCollection List

Let's consider the use case when a `List` is used with `@ElementCollection`:

| Post |                  |              |
|------|------------------|--------------|
| f    | id               | Long         |
| f    | title            | String       |
| f    | comments         | List<String> |
| m    | getId()          | Long         |
| m    | setId(Long)      | void         |
| m    | getTitle()       | String       |
| m    | setTitle(String) | void         |
| m    | getComments()    | List<String> |

Figure 10.8: The `@ElementCollection` relationship

The mapping for the `comments` collection looks as follows:

```
@ElementCollection
private List<String> comments = new ArrayList<>();
```

When it comes to adding or removing child records, the `@ElementCollection` behaves like a unidirectional `@OneToMany` relationship, annotated with `CascadeType.ALL` and `orphanRemoval`.

To persist three comments, the data access layer only has to add them to the parent entity collection:

```
post.getComments().add("My first review");
post.getComments().add("My second review");
post.getComments().add("My third review");
```

Hibernate issues the insert statements during Persistence Context flushing:

```
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My first review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```

Unfortunately, the remove operation uses the same logic as the unidirectional `@OneToMany` association, so when removing the first collection element:

```
post.getComments().remove(0);
```

Hibernate deletes all the associated child-side records and re-inserts the in-memory ones back into the database table:

```
DELETE FROM Post_comments WHERE Post_id = 1
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```



Using an `@ElementCollection` with a `List` is not very efficient because the association is considered to be a bag, in Hibernate terminology.

A bag does not guarantee that elements are uniquely identifiable, hence Hibernate needs to delete and reinsert the elements associated with a given parent entity whenever a change occurs to the `@ElementCollection List`.

## 10.4.2 @ElementCollection Set

On the other hand, when using a `Set`, the `@ElementCollection` no longer behaves like a bag, and the generated SQL statements are going to be more efficient.

So, let's change the `comments` collection to use a `Set` this time:

```
@ElementCollection
private Set<String> comments = new HashSet<>();
```

After inserting the same 3 comments, as we did when using a `List` with `@ElementCollection`, let's see how the `Set` behaves when it comes to removing elements.

When removing the first element, we added to the `Set`:

```
post.getComments().remove("My first review");
```

Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My first review'
```

When removing the last element, we added to the `Set`:

```
post.getComments().remove("My third review");
```

Again, Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My third review'
```

So, if you want to use an `@ElementCollection` and benefit from its simplicity, it's much better if you are using a `Set`, instead of a `List`.



However, even if using a `Set` can render more efficient SQL statements than when using a `List`, it's still much better if you replace the `@ElementCollection` with either a bidirectional `@OneToMany` association or just a simple `@ManyToOne` association that maps the Foreign Key column.

This way, you can remove elements without having to fetch the collection via the parent entity.

## 10.5 @OneToOne

From a database perspective, the one-to-one association is based on a foreign key that is constrained to be unique. This way, a parent row can be referenced by at most one child record only.

In JPA, the `@OneToOne` relationship can be either unidirectional or bidirectional.

### 10.5.1 Unidirectional @OneToOne

In the following example, the `Post` entity represents the parent-side, while the `PostDetails` is the child-side of the one-to-one association.

As already mentioned, the JPA entity relationship diagram matches exactly the one-to-one table relationship.

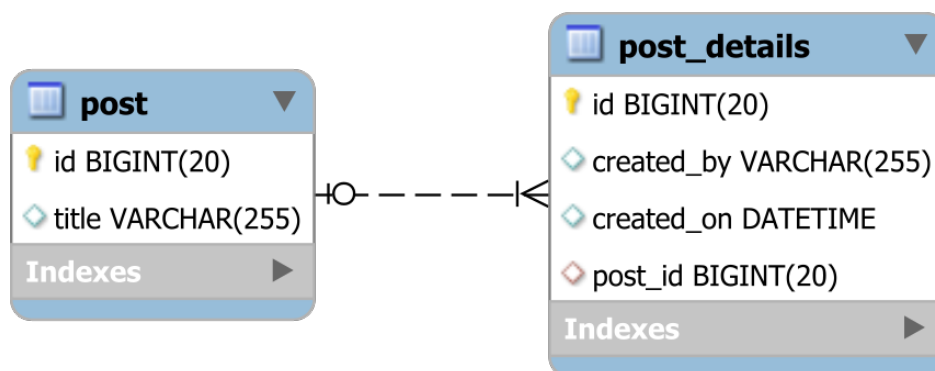


Figure 10.9: The one-to-one table relationship

Even from the Domain Model side, the unidirectional `@OneToOne` relationship is strikingly similar to the unidirectional `@ManyToOne` association.

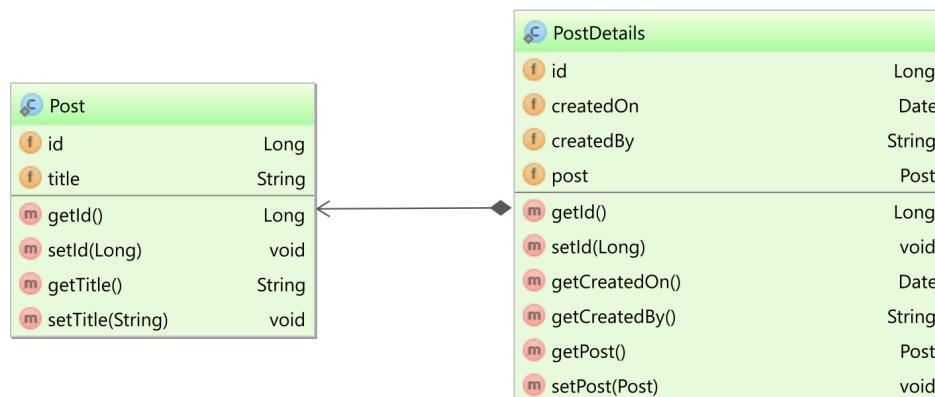


Figure 10.10: The unidirectional `@OneToOne` relationship

The mapping is done through the `@OneToOne` annotation, which, just like the `@ManyToOne` mapping, might also take a `@JoinColumn` as well.

```
@OneToOne
@JoinColumn(name = "post_id")
private Post post;
```

The unidirectional `@OneToOne` association controls the associated foreign key, so, when the `post` attribute is set:

```
Post post = entityManager.find(Post.class, 1L);
PostDetails details = new PostDetails("John Doe");
details.setPost(post);
entityManager.persist(details);
```

Hibernate populate the foreign key column with the associated `post` identifier:

```
INSERT INTO post_details (created_by, created_on, post_id, id)
VALUES ('John Doe', '2016-01-08 11:28:21.317', 1, 2)
```

Even if this is a unidirectional association, the `Post` entity is still the parent-side of this relationship. To fetch the associated `PostDetails`, a JPQL query is needed:

```
PostDetails details = entityManager.createQuery(
    "select pd " +
    "from PostDetails pd " +
    "where pd.post = :post", PostDetails.class)
    .setParameter("post", post)
    .getSingleResult();
```

If the `Post` entity always needs its `PostDetails`, a separate query might not be desirable. To overcome this limitation, it is important to know the `PostDetails` identifier prior to loading the entity.

One workaround would be to use a `@NaturalId`, which might not require a database access if the entity is stored in the second-level cache. Fortunately, there is even a simpler approach which is also portable across JPA providers as well. The JPA 2.0 specification added support for derived identifiers, making possible to link the `PostDetails` identifier to the `post` table primary key.

This way, the `post_details` table primary key can also be a foreign key referencing the `post` table identifier.

The `PostDetails @OneToOne` mapping is changed as follows:

```
@OneToOne
@MapsId
private Post post;
```

This time, the table relationship does not feature any additional foreign key column since the `post_details` table primary key references the `post` table primary key:

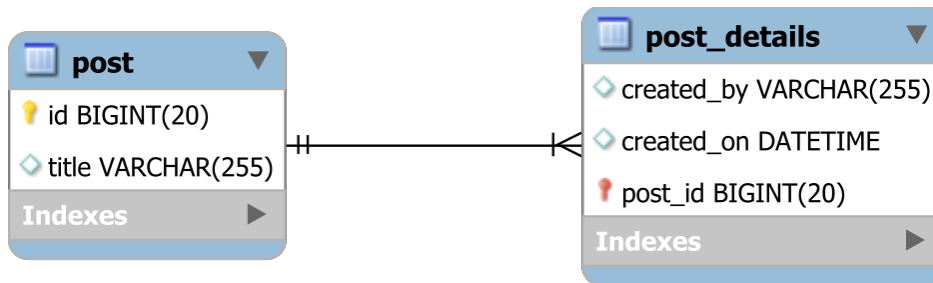


Figure 10.11: The shared key one-to-one

Because `PostDetails` has the same identifier as the parent `Post` entity, it can be fetched without having to write a JPQL query.

```
PostDetails details = entityManager.find(PostDetails.class, post.getId());
```

### The shared primary key efficiency

First of all, the shared primary key approach reduces the memory footprint of the child-side table indexes since it requires a single indexed column instead of two. The more records a child table has, the better the improvement gain for reducing the number of indexed columns.

More, the child entity can now be simply retrieved from the second-level cache, therefore preventing a database hit. In the previous example, because the child entity identifier was not known, a query was inevitable. To optimize the previous use case, the *query cache* would be required as well, but the query cache is not without issues either.



Because of the reduced memory footprint and enabling the second-level cache direct retrieval, the JPA 2.0 *derived identifier* is the preferred `@OneToOne` mapping strategy. The shared primary key is not limited to unidirectional associations, being available for bidirectional `@OneToOne` relationships as well.

## 10.5.2 Bidirectional @OneToOne

A bidirectional `@OneToOne` association allows the parent entity to map the child-side as well:

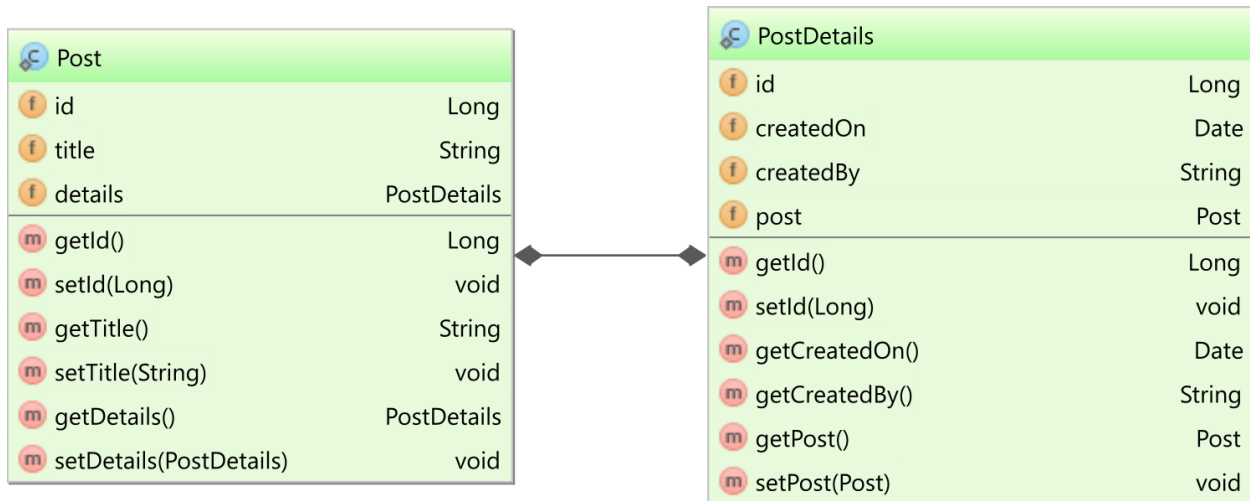


Figure 10.12: The bidirectional `@OneToOne` relationship

The parent-side defines a `mappedBy` attribute because the child-side (which can still share the primary key with its parent) is still in charge of this JPA relationship:

```
@OneToOne(mappedBy = "post", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private PostDetails details;
```

Because this is a bidirectional relationship, the `Post` entity must ensure that both sides of this relationship are set upon associating a `PostDetails` entity:

```
public void setDetails(PostDetails details) {
    if (details == null) {
        if (this.details != null) this.details.setPost(null);
    }
    else details.setPost(this);
    this.details = details;
}
```

Unlike the parent-side `@OneToMany` relationship where Hibernate can simply assign a proxy even if the child collection is empty, the `@OneToOne` relationship must decide if to assign the child reference to null or to an `Object`, be it the actual entity object type or a runtime `Proxy`.

This is an issue that affects the parent-side `@OneToOne` association, while the child-side, which has an associated foreign key column, knows whether the parent reference should be null or not. For this reason, the parent-side must execute a secondary query to know if there is a mirroring foreign key reference on the child-side.

Even if the association is lazy, when fetching a `Post` entity:

```
Post post = entityManager.find(Post.class, 1L);
```

Hibernate fetches the child entity as well, so, instead of only one query, Hibernate requires two select statements:

```
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_  
FROM   post p  
WHERE  p.id = 1
```

```
SELECT pd.post_id AS post_id3_1_0_, pd.created_by AS created_1_1_0_,  
       pd.created_on AS created_2_1_0_  
FROM   post_details pd  
WHERE  pd.post_id = 1
```

If the application developer only needs parent entities, the additional child-side secondary queries will be executed unnecessarily, and this might affect application performance. The more parent entities are needed to be retrieved, the more obvious the secondary queries performance impact gets.

## Limitations

Even if the foreign key is `NOT NULL` and the parent-side is aware about its non-nullability through the optional attribute (e.g. `@OneToOne(mappedBy = "post", fetch = FetchType.LAZY, optional = false)`), Hibernate still generates a secondary select statement.

For every managed entity, the Persistence Context requires both the entity type and the identifier, so the child identifier must be known when loading the parent entity, and the only way to find the associated `post_details` primary key is to execute a secondary query. Because the child identifier is known when using `@MapsId`, in future, [HHH-10771<sup>a</sup>](https://hibernate.atlassian.net/browse/HHH-10771) should address the secondary query issue.



Bytecode enhancement is the only viable workaround. However, it only works if the parent side is also annotated with `@LazyToOne(LazyToOneOption.NO_PROXY)`. Because it's simpler and more predictable, the unidirectional `@OneToOne` relationship is often preferred.

---

<sup>a</sup><https://hibernate.atlassian.net/browse/HHH-10771>

## 10.6 @ManyToMany

From a database perspective, the `@ManyToMany` annotation mirrors a many-to-many table relationship:



Figure 10.13: The many-to-many table relationship

Like the `@OneToMany` and `@OneToOne` relationships, the `@ManyToMany` association can be either unidirectional or bidirectional.

### 10.6.1 Unidirectional @ManyToMany List

In our case, if we need to map a unidirectional `@ManyToMany` association between the `Post` and `Tag` entities, it makes sense to have the `Post` entity map the `@ManyToMany` relationship since there is not much need for navigating this association from the `Tag` side (although we can still do it with a JPQL query).

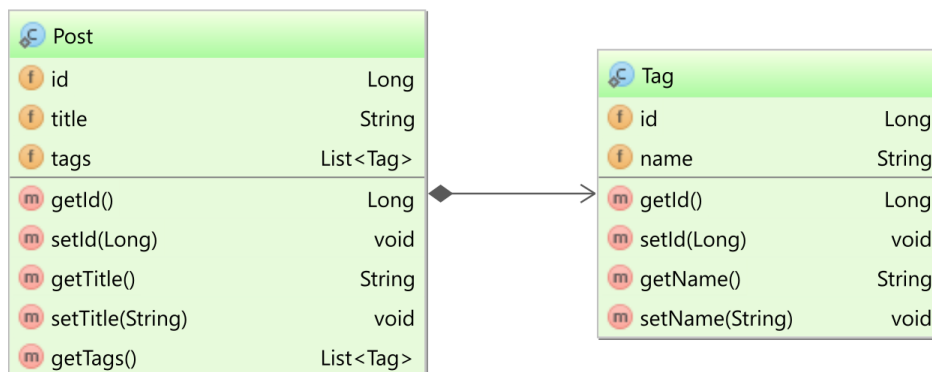


Figure 10.14: The unidirectional `@ManyToMany` relationship

In the `Post` entity, the `tags` collection using a `List`, like this:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id"))
private List<Tag> tags = new ArrayList<>();
```

When adding several entities:

```
Post post1 = new Post("JPA with Hibernate");
Post post2 = new Post("Native Hibernate");

Tag tag1 = new Tag("Java");
Tag tag2 = new Tag("Hibernate");

post1.getTags().add(tag1);
post1.getTags().add(tag2);
post2.getTags().add(tag1);

entityManager.persist(post1);
entityManager.persist(post2);
```

Hibernate manages to persist both the `Post` and the `Tag` entities along with their junction records.

```
INSERT INTO post (title, id) VALUES ('JPA with Hibernate', 1)
INSERT INTO post (title, id) VALUES ('Native Hibernate', 4)

INSERT INTO tag (name, id) VALUES ('Java', 2)
INSERT INTO tag (name, id) VALUES ('Hibernate', 3)

INSERT INTO post_tag (post_id, tag_id) VALUES (1, 2)
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
INSERT INTO post_tag (post_id, tag_id) VALUES (4, 2)
```

## Cascading

For `@ManyToMany` associations, `CascadeType.REMOVE` does not make too much sense when both sides represent independent entities. In this case, removing a `Post` entity should not trigger a `Tag` removal because the `Tag` can be referenced by other posts as well. The same arguments apply to orphan removal since removing an entry from the `tags` collection should only delete the junction record and not the target `Tag` entity.



For both unidirectional and bidirectional associations, it is better to avoid the `CascadeType.REMOVE` mapping. Instead of `CascadeType.ALL`, the cascade attributes should be declared explicitly (e.g. `CascadeType.PERSIST`, `CascadeType.MERGE`).

But just like the unidirectional `@OneToMany` association, problems arise when it comes to removing the junction records from the `List`:

```
post1.getTags().remove(tag1);
```

Hibernate deletes all junction rows associated with the `Post` entity whose `Tag` association is being removed and inserts back the remaining ones:

```
DELETE FROM post_tag WHERE post_id = 1
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
```

### 10.6.2 Unidirectional @ManyToMany Set

When using a `List`, the `@ManyToMany` association behaves like a bag, and, as explained in the [@ElementCollection](#) section, Hibernate bags are not very efficient.

Therefore, let's change the `tags` collection to use a `Set`:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();
```

Assuming, we added the same 3 `Tag` entities to the `Post` entity, if we remove the first `Tag` from the `tags` collection:

```
post1.getTags().remove(tag1);
```

Hibernate is going to generate a single SQL delete statement, this time:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 2
```



For a `@ManyToMany` association, it's better to use the `Set` collection type, instead of `List`.

### 10.6.3 Bidirectional @ManyToMany

The bidirectional `@ManyToMany` relationship can be navigated from both the `Post` and the `Tag` side.

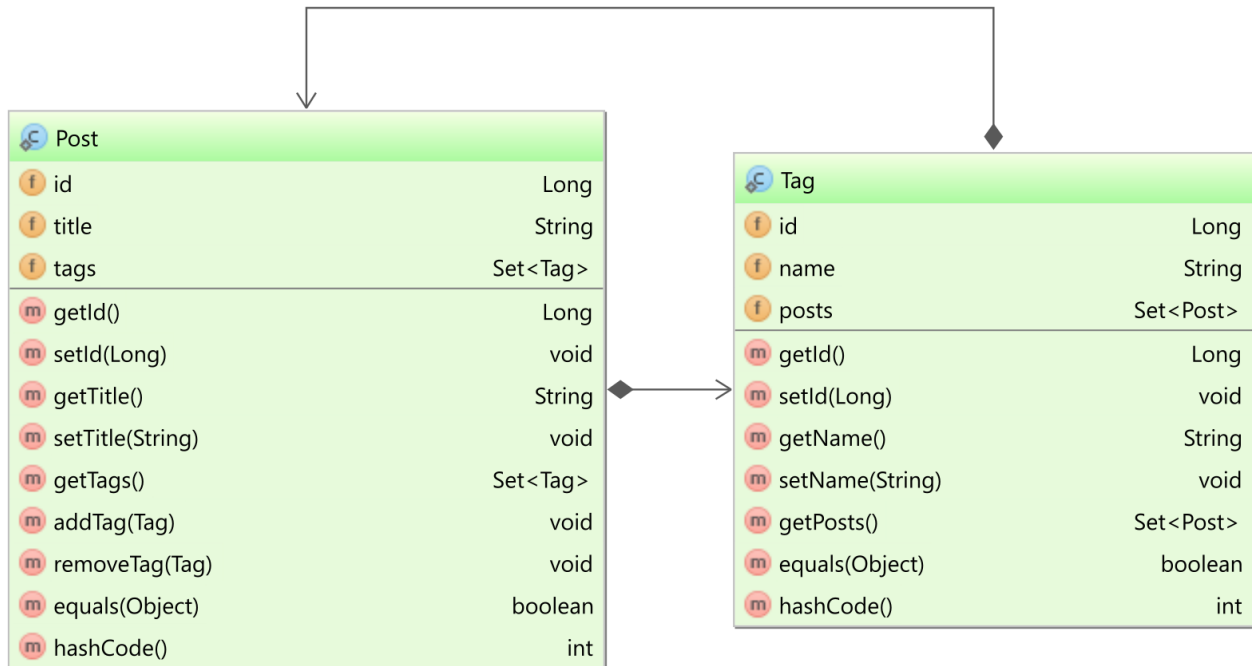


Figure 10.15: The bidirectional `@ManyToMany` relationship

While in the one-to-many and many-to-one associations the child-side is the one holding the foreign key, for a many-to-many table relationship both ends are parent-sides and the junction table plays the child-side role.

Because the junction table is hidden when using the default `@ManyToMany` mapping, the application developer must choose an owning and a `mappedBy` side.

The `Post` entity uses the same mapping as shown in the unidirectional `@ManyToMany` section:

```

@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id"))
private Set<Tag> tags = new HashSet<>();
  
```

while the `Tag` entity adds a `@ManyToMany` association that looks as follows:

```

@ManyToMany(mappedBy = "tags")
private Set<Post> posts = new HashSet<>();
  
```

The `mappedBy` attribute tells Hibernate that the `Post` entity is the owning side of this association. In our case, the `mappedBy` attribute of the `@ManyToMany` annotation in the `Tag` entity points to the `tags` collection of the `Post` entity, meaning that the `Post` entity is going to propagate the association state changes to the underlying junction table.



Hibernate manages each side of a bidirectional `@ManyToMany` relationship just like a unidirectional `@OneToMany` association between the parent-side (e.g. `Post` or the `Tag`) and the hidden child-side (e.g. the `post_tag` table `post_id` or `tag_id` foreign keys).

For this reason, it's much more efficient to use a `Set` collection type, no matter if the `@ManyToMany` is unidirectional or bidirectional.

Like any bidirectional associations, helper methods are needed to keep both parent sides in sync. For a bidirectional `@ManyToMany` association, the helper methods must be added to the entity that is more likely to interact with. In our case, the root entity is the `Post`, so the helper methods are added to the `Post` entity:

```
public void addTag(Tag tag) {
    tags.add(tag);
    tag.getPosts().add(this);
}

public void removeTag(Tag tag) {
    tags.remove(tag);
    tag.getPosts().remove(this);
}
```

Both `Post` and `Tag` entities have unique attributes which can simplify the entity removal operation even when mixing detached and managed entities.

### 10.6.4 The `@OneToMany` alternative

We can also map a many-to-many table relationship using bidirectional `@OneToMany` associations from the parent entities to the child entity that maps the junction table.

Therefore, the `post_tag` child table is mapped by the `PostTag` entity, and its composite identifier is encapsulated in the `PostTagId` embeddable type as illustrated by the following diagram.

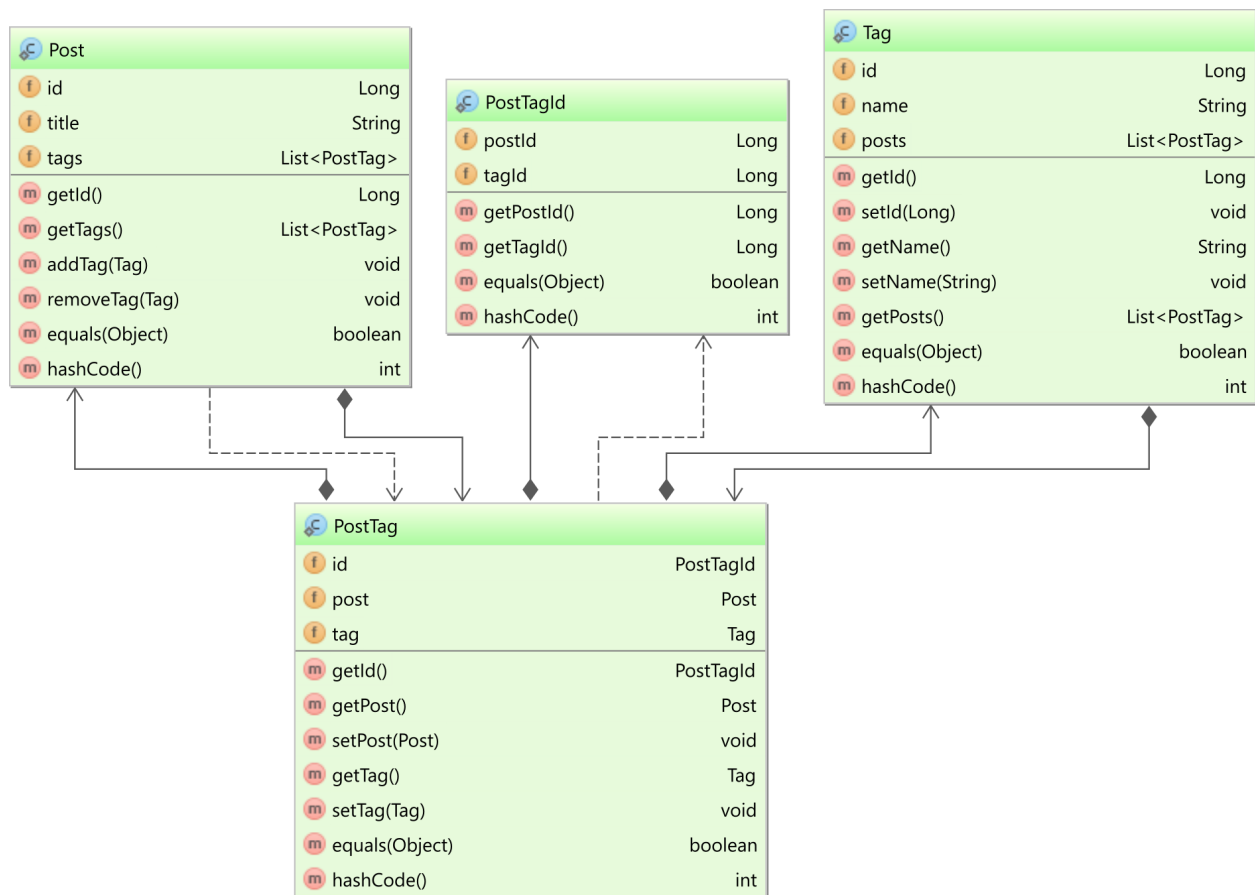


Figure 10.16: The @OneToMany as a many-to-many table relationship

The **PostTagId** maps the **postId** and **tagId** foreign key columns, and the **PostTag** entity uses the **postId** and **tagId** properties of the **PostTagId** identifier to map the **post** and the **tag** @ManyToMany associations.

The **Post** entity maps the bidirectional @OneToMany side of the **post** @ManyToMany association from the **PostTag** child entity:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> tags = new ArrayList<>();
```

The **Tag** entity maps the bidirectional @OneToMany side of the **tag** @ManyToMany association defined by the **PostTag** child entity:

```
@OneToMany(mappedBy = "tag", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> posts = new ArrayList<>();
```

This way, the bidirectional @ManyToMany relationship is transformed in two bidirectional @OneToMany associations.

The PostTag entity has a composed identifier made out of the post\_id and tag\_id columns.

```
@Embeddable
public class PostTagId implements Serializable {

    @Column(name = "post_id")
    private Long postId;

    @Column(name = "tag_id")
    private Long tagId;

    public PostTagId() {}

    public PostTagId(Long postId, Long tagId) {
        this.postId = postId;
        this.tagId = tagId;
    }

    public Long getPostId() {
        return postId;
    }

    public Long getTagId() {
        return tagId;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTagId that = (PostTagId) o;
        return Objects.equals(postId, that.getPostId()) &&
            Objects.equals(tagId, that.getTagId());
    }

    @Override
    public int hashCode() {
        return Objects.hash(postId, tagId);
    }
}
```

The JPA specification requires the composite identifiers embeddable types to implement the `Serializable` interface, as well as implementing `equals` and `hashCode` based on the associated foreign key column values.

The `PostTag` entity defines two `@ManyToOne` associations, as follows:

```
@Entity
@Table(name = "post_tag")
public class PostTag {

    @EmbeddedId
    private PostTagId id;

    @ManyToOne
    @MapsId("postId")
    private Post post;

    @ManyToOne
    @MapsId("tagId")
    private Tag tag;

    private PostTag() {}

    public PostTag(Post post, Tag tag) {
        this.post = post;
        this.tag = tag;
        this.id = new PostTagId(post.getId(), tag.getId());
    }

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTag that = (PostTag) o;
        return Objects.equals(post, that.getPost()) &&
            Objects.equals(tag, that.getTag());
    }

    @Override
    public int hashCode() {
        return Objects.hash(post, tag);
    }
}
```

The `@MapsId` annotation instructs Hibernate that the `post_id` and `tag_id` foreign key columns are managed by the `PostTagId` composite identifier, not by the `@ManyToOne` property.

Just like with any bidirectional association, we need to define the helper methods that allow us to keep the `@ManyToOne` and `@OneToMany` sides in sync.

However, as opposed to a standard bidirectional `@OneToMany` association, the `removeTag` helper method in the `Post` entity is much more complex because it needs to locate the `PostTag` associated with the current `Post` entity, as well as the `Tag` object reference in the `tags` collection that needs to be disassociated.

```
public void removeTag(Tag tag) {
    for (Iterator<PostTag> iterator = tags.iterator(); iterator.hasNext(); ) {
        PostTag postTag = iterator.next();
        if (postTag.getPost().equals(this) && postTag.getTag().equals(tag)) {
            iterator.remove();
            postTag.getTag().getPosts().remove(postTag);
            postTag.setPost(null);
            postTag.setTag(null);
            break;
        }
    }
}
```

Notice that, the `removeTag` method relies on the `PostTag` `equals` and `hashCode` methods in order to remove the desired entry.

When rerunning the entity removal example featured in the unidirectional `@ManyToMany` section:

```
post1.removeTag(tag1);
```

Hibernate issues a single delete statement, therefore targeting a single `PostTag` junction record:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 3
```

Because the `@ManyToOne` side manages the foreign key column changes, the internal collection state of the `@OneToMany` side is not taken into consideration unless we remove an entry.

For this reason, reversing the order of the `tags` elements has absolutely no effect:

```
post1.getTags().sort(
    Comparator.comparing(
        (PostTag postTag) -> postTag.getId().getTagId()
    )
    .reversed()
);
```

To materialize the order of elements, the `@OrderColumn` must be used instead:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
@OrderColumn(name = "entry")
private List<PostTag> tags = new ArrayList<>();
```

When using the aforementioned mapping, the `post_tag` junction table features an `entry` column storing the collection element order.

Therefore, if we reverse the element order as we did previously, Hibernate is going to update the `entry` column to maintain the current `PostTag` element order:

```
UPDATE post_tag SET entry = 0 WHERE post_id = 1 AND tag_id = 4
UPDATE post_tag SET entry = 1 WHERE post_id = 1 AND tag_id = 3
```



Mapping the junction entity of a many-to-many table relationship allows us to persist additional column into the junction table (e.g. `created_on`, `created_by`), which cannot be done when using the JPA `@ManyToMany` annotation.

## 10.7 Hypersistence Optimizer

Knowing which are the most efficient associations is of paramount importance when it comes to designing a high-performance data access layer. But choosing the right mappings is not sufficient. You also have to check the Hibernate configuration properties, as well as the queries that get executed.

For this purpose, I created the [Hypersistence Optimizer](https://vladmihalcea.com/hypersistence-optimizer/)<sup>1</sup> tool. While you could manually investigate the JPA and Hibernate mappings and configurations, having a tool to do this task on your behalf is going to save you a lot of time.

More, Hypersistence Optimizer can validate your JPA and Hibernate usage during testing, and you could even assert the number of issues detected by the tool and trigger a build failure whenever a performance-impacting issue is detected. This way, you can ensure that future data access logic changes won't affect the performance of your application.

Because you purchased this book, you have a **25\$** discount for any Hypersistence Optimizer license. All you have to do is to use the **HO25OFF** coupon code when making the purchase. Or, you can use [this link](https://vladmihalcea.com/hypersistence-optimizer/?coupon_code=HO25OFF)<sup>2</sup> to quickly navigate to the Hypersistence Optimizer sales page with the discount coupon activated.

### 10.7.1 Testimonials

“It really pays off when it comes to analyzing complex applications. For architectures that are difficult to manage, it quickly provides clues for further analysis.

This is a huge help both when setting up a new implementation and when optimizing a legacy application.”

– Kevin Peters (Software Engineer - codecentric AG)

“Let's face it, JPA and Hibernate are very powerful but not simple tools. Even experienced developers make mistakes that may result in very heavy database queries, dramatically lowering overall application performance.

Hypersistence Optimizer feels like an additional team member - a JPA expert that is there to help and show how can you optimize your mappings and configurations before you ship them to production.”

– Maciej Walkowiak (Freelance Tech Lead)

---

<sup>1</sup><https://vladmihalcea.com/hypersistence-optimizer/>

<sup>2</sup>[https://vladmihalcea.teachable.com/p/hypersistence-optimizer/?coupon\\_code=HO25OFF](https://vladmihalcea.teachable.com/p/hypersistence-optimizer/?coupon_code=HO25OFF)

# 11. Inheritance

Java, like any other object-oriented programming language, makes heavy use of inheritance and polymorphism. Inheritance allows defining class hierarchies that offer different implementations of a common interface.

Conceptually, the Domain Model defines both data (e.g. persisted entities) and behavior (business logic). Nevertheless, inheritance is more useful for varying behavior rather than reusing data (composition is much more suitable for sharing structures). Even if the data (persisted entities) and the business logic (transactional services) are decoupled, inheritance can still help varying business logic (e.g. [Visitor pattern](https://en.wikipedia.org/wiki/Visitor_pattern)<sup>1</sup>).

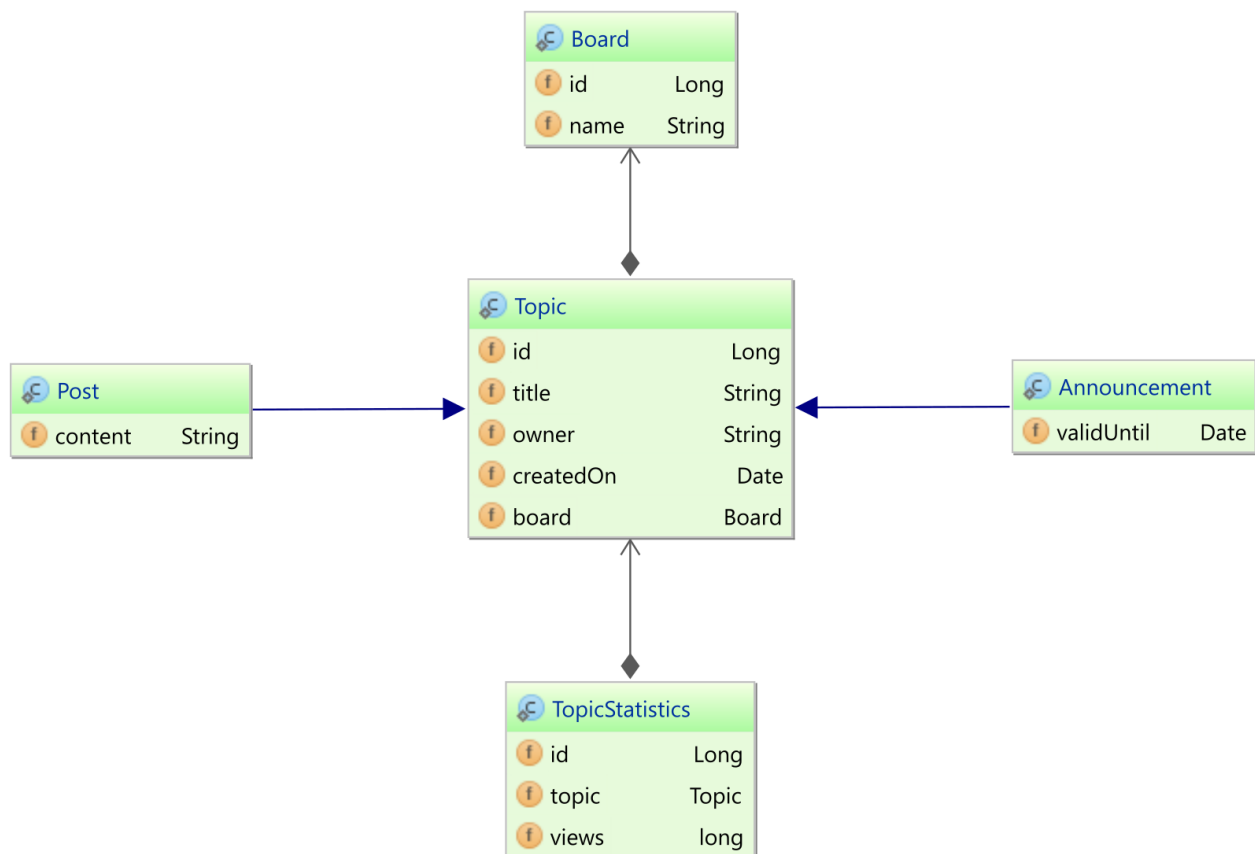


Figure 11.1: Domain Model Inheritance

<sup>1</sup>[https://en.wikipedia.org/wiki/Visitor\\_pattern](https://en.wikipedia.org/wiki/Visitor_pattern)

The root entity of this Domain Model is the `Board` entity because, either directly or indirectly, all the other entities are associated with a `Board`.

```
@Entity @Table(name = "board")
public class Board {

    @Id @GeneratedValue
    private Long id;

    private String name;

    //Getters and setters omitted for brevity
}
```

The end user can submit either a `Post` or an `Announcement` on a particular `Board`. Because the `Post` and the `Announcement` share the same functionality (differing only in data), they both inherit from a `Topic` base class.

The `Topic` class defines a relationship to a `Board` entity, hence the `Post` and the `Announcement` entities can also be associated with a `Board` instance.

```
@Entity @Table(name = "topic")
public class Topic {

    @Id @GeneratedValue
    private Long id;

    private String title;

    private String owner;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdOn = new Date();

    @ManyToOne(fetch = FetchType.LAZY)
    private Board board;

    //Getters and setters omitted for brevity
}
```

Both the `Post` and the `Announcement` entities extend the `Topic` class and define their own specific attributes.

```
@Entity @Table(name = "post")
public class Post extends Topic {

    private String content;

    //Getters and setters omitted for brevity
}

@Entity @Table(name = "announcement")
public class Announcement extends Topic {

    @Temporal(TemporalType.TIMESTAMP)
    private Date validUntil;

    //Getters and setters omitted for brevity
}
```

The `TopicStatistics` is at the bottom of this Domain Model as it is only needed for monitoring purposes, without being directly associated with the main business logic. Because statistics are needed for both `Post` and `Announcement` entities, the `TopicStatistics` defines a `Topic` entity association.

```
@Entity @Table(name = "topic_statistics")
public class TopicStatistics {

    @Id @GeneratedValue
    private Long id;

    @OneToOne @JoinColumn(name = "id") @MapsId
    private Topic topic;

    private long views;

    //Getters and setters omitted for brevity
}
```



Another approach would be to add this relationship into the `Topic` class, and the `topic` rows would then reference the `topic_statistics` records. For the sake of demonstrating how entity association polymorphism works, the `TopicStatistics` was chosen to be the child-side.

As natural as this Domain Model may be represented in an object-oriented programming language, transposing it to a relational database is anything but straightforward. Although SQL-99 added support for type inheritance, this feature is seldom implemented. Therefore, relational database systems are relying on tuples and relational algebra for representing and manipulating data. For this reason, mapping inheritance in a relational database is one of the most obvious object-relational impedance mismatches.

Without native support from the database system, inheritance can only be emulated through table relationships. In the *Patterns of Enterprise Application Architecture* book, Martin Fowler defines three ways of mapping inheritance into a relational database:

- [Single Table Inheritance](#)<sup>2</sup>, which uses a single database table to represent all classes in a given inheritance hierarchy.
- [Class Table Inheritance](#)<sup>3</sup>, which maps each class to a table, and the inheritance association is resolved through table joins.
- [Concrete Table Inheritance](#)<sup>4</sup>, where each table defines all fields that are either defined in the subclass or inherited from a superclass.

The JPA specification defines all these three inheritance mapping models through the following strategies:

- `InheritanceType.SINGLE_TABLE`
- `InheritanceType.JOINED`
- `InheritanceType.TABLE_PER_CLASS`.

JPA also covers the case when inheritance is only available in the Domain Model, without being mirrored into the database (e.g. `@MappedSuperclass`).

Whenever the data access layer implements a functionality without support from the underlying database system, care must be taken to ensure that application performance is not compromised. This chapter aims to analyze what trade-offs are required for employing inheritance as well as its impact on application performance.

<sup>2</sup><http://martinfowler.com/eaCatalog/singleTableInheritance.html>

<sup>3</sup><http://martinfowler.com/eaCatalog/classTableInheritance.html>

<sup>4</sup><http://martinfowler.com/eaCatalog/concreteTableInheritance.html>

## 11.1 Single table

The single table inheritance is the default JPA strategy, funneling a whole inheritance Domain Model hierarchy into a single database table.

To employ this strategy, the `Topic` entity class must be mapped with one of the following annotations:

- `@Inheritance` (being the default inheritance model, it is not mandatory to supply the strategy when using single table inheritance).
- `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)`.

The `Post` and the `Announcement` entities do not need any extra mapping (the Java inheritance semantics being sufficient).

Preserving the same layout as depicted in the Domain Model class diagram, the table relationships associated with this inheritance strategy look like this:

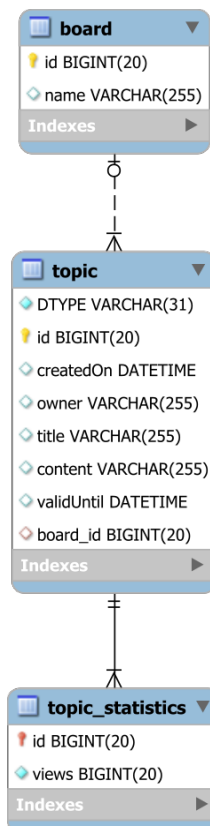


Figure 11.2: Single table

The `topic` table contains columns associated with the `Topic` base class as well as columns related to attributes from `Post` and `Announcement` entities.

In the following example, one `Post` and one `Announcement` entities are going to be persisted along with their associated `@OneToOne TopicStatistics` relations.

```
Post post = new Post();
post.setOwner("John Doe");
post.setTitle("Inheritance");
post.setContent("Best practices");
post.setBoard(board);

entityManager.persist(post);

Announcement announcement = new Announcement();
announcement.setOwner("John Doe");
announcement.setTitle("Release x.y.z.Final");
announcement.setValidUntil(Timestamp.valueOf(LocalDate.now().plusMonths(1)));
announcement.setBoard(board);

entityManager.persist(announcement);

TopicStatistics postStatistics = new TopicStatistics(post);
postStatistics.incrementViews();
entityManager.persist(postStatistics);

TopicStatistics announcementStatistics = new TopicStatistics(announcement);
announcementStatistics.incrementViews();
entityManager.persist(announcementStatistics);
```

Both the `Post` and the `Announcement` entities are saved in the `topic` table whose primary key is shared with the `topic_statistics` table.

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES (1, '2016-01-17 09:22:22.11', 'John Doe', 'Inheritance',
       'Best practices', 'Post', 2)

INSERT INTO topic (board_id, createdOn, owner, title, validUntil, DTYPE, id)
VALUES (1, '2016-01-17 09:22:22.11', 'John Doe', 'Release x.y.z.Final',
       '2016-02-17 09:22:22.114', 'Announcement', 3)

INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

One advantage of using inheritance in the Domain Model is the support for polymorphic queries. When the application developer issues a select query against the `Topic` entity:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
    .setParameter("boardId", 1L)
    .getResultList();
```

Hibernate goes to the `topic` table, and, after fetching the result set, it maps every row to its associated subclass instance (e.g. `Post` or `Announcement`) by analyzing the discriminator column (e.g. `DTYPE`) value.

```
SELECT t.id AS id2_1_, t.board_id AS board_id8_1_, t.createdOn AS created03_1_,
       t.owner AS owner4_1_, t.title AS title5_1_, t.content AS content6_1_,
       t.validUntil AS validUnt7_1_, t.DTYPE AS DTYPE1_1_
FROM topic t
WHERE t.board_id = 1
```

Domain Model inheritance allows base class entity associations to be automatically resolved upon being retrieved. When loading a `TopicStatistics` along with its `Topic` relation:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
    .setParameter("topicId", topicId)
    .getSingleResult();
```

Hibernate joins the `topic_statistics` and the `topic` tables so that it can create a `TopicStatistics` entity with an actual `Post` or `Announcement` attribute object reference.

```
SELECT
    ts.id AS id1_2_0_, t.id AS id2_1_1_, ts.views AS views2_2_0_,
    t.board_id AS board_id8_1_1_, t.createdOn AS created03_1_1_,
    t.owner AS owner4_1_1_, t.title AS title5_1_1_, t.content AS content6_1_1_,
    t.validUntil AS validUnt7_1_1_, t.DTYPE AS DTYPE1_1_1_
FROM topic_statistics ts
INNER JOIN topic t ON ts.id = t.id
WHERE t.id = 2
```

Even if not practical in this particular example, `@OneToMany` associations are also possible.

The `Board` entity can map a bidirectional `@OneToMany` relationship as follows:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the aforementioned `Topic` entity query. When fetching the collection eagerly, Hibernate requires a single table join.

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
    .setParameter("id", id)
    .getSingleResult();
```

```
SELECT b.id AS id1_0_0_, t.id AS id2_1_1_, b.name AS name2_0_0_,
       t.board_id AS board_id8_1_1_, t.createdOn AS created03_1_1_,
       t.owner AS owner4_1_1_, t.title AS title5_1_1_, t.content AS content6_1_1_,
       t.validUntil AS validUnt7_1_1_, t.DTYPE AS DTYPE1_1_1_,
       t.board_id AS board_id8_1_0_, t.id AS id2_1_0_
FROM   board b
INNER JOIN topic t ON b.id = t.board_id
WHERE  b.id = 1
```

## Performance and data integrity considerations

Since only one table is used for storing entities, both reads and writes are fast. Even when using a `@ManyToOne` or a `@OneToOne` base class association, Hibernate needs a single join between parent and child tables. The `@OneToMany` base class entity relationship is also efficient since it either generates a secondary select or a single table join.

Because all subclass attributes are collocated in a single table, `NOT NULL` constraints are not allowed for columns belonging to subclasses. Being automatically inherited by all subclasses, the base class attributes may be non-nullable. From a data integrity perspective, this limitation defeats the purpose of Consistency (guaranteed by the ACID properties).

Nevertheless, the data integrity rules can be enforced through database trigger procedures or `CHECK` constraints (a column non-nullability is accounted based on the class discriminator value). Another approach is to move the check into the data access layer. Bean Validation can validate `@NotNull` attributes at runtime. JPA also defines callback methods (e.g. `@PrePersist`, `@PreUpdate`) as well as entity listeners (e.g. `@EntityListeners`) which can throw an exception when a non-null constraint is violated.

### 11.1.1 Data integrity constraints

The SQL standard defines the `CHECK` constraint which can be used to apply a row-level verification for each table record that is inserted. Depending on the underlying database, the `CHECK` constraint can be enforced (e.g. Oracle, SQL Server, PostgreSQL, [MySQL 8.0.16<sup>5</sup>](https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html)) or ignored (e.g. [MySQL prior to 8.0.16<sup>6</sup>](https://dev.mysql.com/doc/refman/5.7/en/create-table.html)).

For the aforementioned database tables, the `content` column must never be null if the underlying record is a `Post`, and the `validUntil` column should not be null if the database row represents an `Announcement` entity. Luckily, the default `DTYPE` column specifies the entity type associated with each particular table row.

To ensure the aforementioned data integrity rules, the following `CHECK` constraints needs to be added:

```
ALTER TABLE topic ADD CONSTRAINT post_content_check CHECK (
    CASE WHEN DTYPE = 'Post'
        THEN CASE WHEN content IS NOT NULL THEN 1 ELSE 0 END
        ELSE 1
    END = 1
)

ALTER TABLE topic ADD CONSTRAINT announcement_validUntil_check CHECK (
    CASE WHEN DTYPE = 'Announcement'
        THEN CASE WHEN validUntil IS NOT NULL THEN 1 ELSE 0 END
        ELSE 1
    END = 1
)
```

With these `CHECK` constraints in place, when trying to insert a `Post` entity without a `content`:

```
entityManager.persist(new Post());
```

PostgreSQL generates the following error message:

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES ((NULL(BIGINT), '2016-07-15 13:45:16.705', NULL(VARCHAR), NULL(VARCHAR),
        NULL(VARCHAR), 'Post', 4)

-- SQL Error: 0, SQLState: 23514
-- new row for relation "topic" violates check constraint "post_content_check"
```

<sup>5</sup><https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html>

<sup>6</sup><https://dev.mysql.com/doc/refman/5.7/en/create-table.html>

For MySQL versions older than 8.0.16, the same outcome can be achieved with database triggers. For instance, you could add the following `BEFORE INSERT` triggers:

```
CREATE
TRIGGER post_content_check BEFORE INSERT
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Post'
    THEN
        IF NEW.content IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Post content cannot be NULL';
        END IF;
    END IF;
END;

CREATE
TRIGGER announcement_validUntil_check BEFORE INSERT
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Announcement'
    THEN
        IF NEW.validUntil IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Announcement validUntil cannot be NULL';
        END IF;
    END IF;
END;
```

When running the previous `Post` insert, MySQL generates the following output:

```
INSERT INTO topic (board_id, createdOn, owner, title, content, DTYPE, id)
VALUES ((NULL(BIGINT), '2016-07-15 13:50:51.989', NULL(VARCHAR), NULL(VARCHAR),
        NULL(VARCHAR), 'Post', 4)

-- SQL Error: 1644, SQLState: 45000
-- Post content cannot be NULL
```

You also need to add `BEFORE UPDATE` triggers, so that you can cover modifications for the `post` and `announcement` tables, which could compromise data integrity if the subclass-specific columns were set to `null`.

```

CREATE
TRIGGER post_content_update_check BEFORE UPDATE
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Post'
    THEN
        IF NEW.content IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Post content cannot be NULL';
        END IF;
    END IF;
END;

CREATE
TRIGGER announcement_validUntil_update_check BEFORE UPDATE
ON topic
FOR EACH ROW
BEGIN
    IF NEW.DTYPE = 'Announcement'
    THEN
        IF NEW.validUntil IS NULL
        THEN
            signal sqlstate '45000'
            set message_text = 'Announcement validUntil cannot be NULL';
        END IF;
    END IF;
END;

```

Now, if we try to update an `Announcement` entity and set the `validUntil` column to `null`, MySQL will fail with the following error message:

```

UPDATE topic
SET board_id = 1, createdOn = '2016-07-15 13:50:51.989', OWNER = 'John Doe',
    title = 'John Doe, Release x.y.z.Final', validUntil = NULL(TIMESTAMP)
WHERE id = 3

-- SQL Error: 1644, SQLState: 45000
-- Announcement validUntil cannot be NULL

```

Although a little bit verbose, the `CHECK` and `TRIGGER` constraints are very useful to ensure data integrity when using single table inheritance.

## 11.2 Join table

The join table inheritance resembles the Domain Model class diagram since each class is mapped to an individual table. The subclass tables have a foreign key column referencing the base class table primary key.

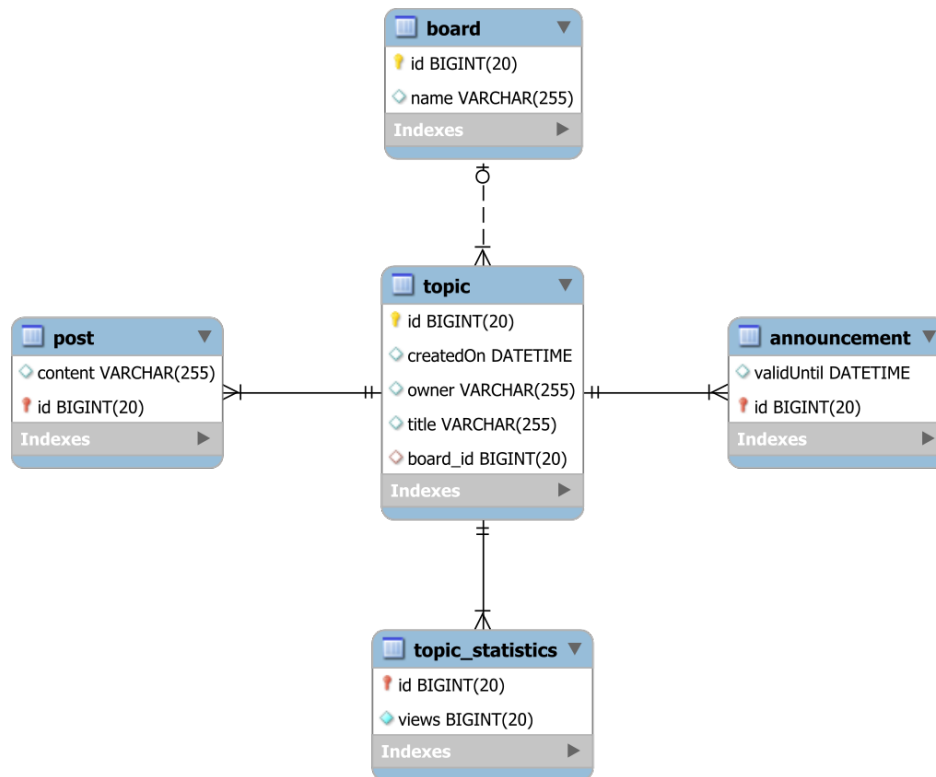


Figure 11.3: Join table

To use this inheritance strategy, the `Topic` entity must be annotated with:

```
@Inheritance(strategy = InheritanceType.JOINED)
```



The `Post` and the `Announcement` entities can use a `@PrimaryKeyJoinColumn` mapping to define the base class foreign key column.

By default, the subclass table primary key column is used as a foreign key as well.

When persisting the same entities defined in the single table section, Hibernate generates the following SQL statements:

```
INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Inheritance', 2)

INSERT INTO post (content, id) VALUES ('Best practices', 2)

INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Release x.y.z.Final', 3)

INSERT INTO announcement (validUntil, id) VALUES ('2016-02-17 09:27:10.698', 3)

INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

The base class information goes into the `topic` table while the subclass content goes in the `post` or the `announcement` tables. When fetching all `Topic` entities associated with a specific `Board`:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
    .setParameter("boardId", 1L)
    .getResultList();
```

Hibernate must join the base class with each individual subclass table.

```
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS created02_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t1_.content AS content1_2_,
    t2_.validUntil AS validUnt1_0_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
         WHEN t2_.id IS NOT NULL THEN 2
         WHEN t.id IS NOT NULL THEN 0
    END AS clazz_
FROM topic t
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.board_id = 1
```

When loading a `TopicStatistics` entity along with its `Topic` association:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

Hibernate must join four tables to construct the desired result set:

```
SELECT
    ts.id AS id1_4_0_,
    t.id AS id1_3_1_,
    ts.views AS views2_4_0_,
    t.board_id AS board_id5_3_1_,
    t.createdOn AS created02_3_1_,
    t.owner AS owner3_3_1_,
    t.title AS title4_3_1_,
    t1_.content AS content1_2_1_,
    t2_.validUntil AS validUnt1_0_1_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
         WHEN t2_.id IS NOT NULL THEN 2
         WHEN t.id IS NOT NULL THEN 0
    END AS clazz_1_
FROM topic_statistics ts
INNER JOIN topic t ON ts.id = t.id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.id = 2
```

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
    .setParameter("id", id)
    .getSingleResult();
```

Hibernate requires three joins to fetch all topic-related information.

#### SELECT

```
b.id AS id1_1_0_, t.id AS id1_3_1_, b.name AS name2_1_0_,
t.board_id AS board_id5_3_1_, t.createdOn AS created02_3_1_,
t.owner AS owner3_3_1_, t.title AS title4_3_1_,
t1_.content AS content1_2_1_, t2_.validUntil AS validUnt1_0_1_,
CASE WHEN t1_.id IS NOT NULL THEN 1
      WHEN t2_.id IS NOT NULL THEN 2
      WHEN t.id IS NOT NULL THEN 0
END AS clazz_1_,
t.board_id AS board_id5_3_0__, t.id AS id1_3_0__
FROM board b
INNER JOIN topic t ON b.id = t.board_id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE b.id = 1
```

### Performance considerations

Unlike single table inheritance, the joined table strategy allows nullable subclass attribute columns.

When writing data, Hibernate requires two insert statements for each subclass entity, so there's a performance impact compared to single table inheritance. The index memory footprint also increases because instead of a single table primary key, the database must index the base class and all subclasses primary keys.

When reading data, polymorphic queries require joining the base class with all subclass tables, so, if there are  $n$  subclasses, Hibernate will need  $N+1$  joins. The more joins, the more difficult it is for the database to calculate the most efficient execution plan.

## 11.3 Table-per-class

The table-per-class inheritance model has a table layout similar to the joined table strategy, but, instead of storing base class columns in the `topic` table, each subclass table also stores columns from the `topic` table. There is no foreign key between the `topic` and the `post` or `announcement` subclass tables, and there is no foreign key in the `topic_statistics` table either.

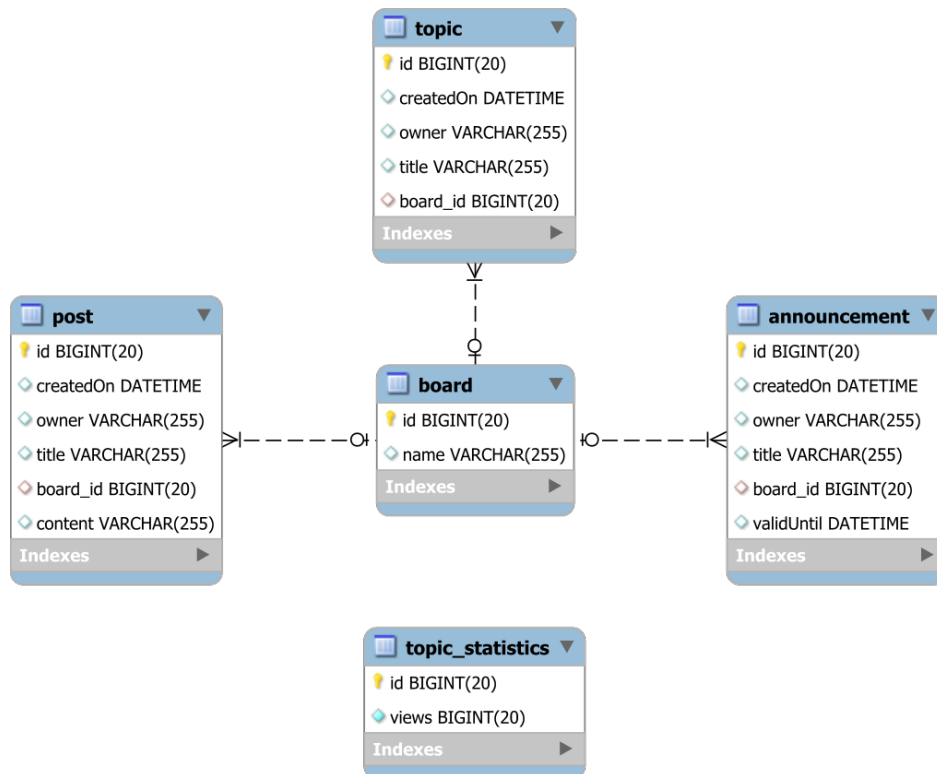


Figure 11.4: Table-per-class

To use this inheritance strategy, the `Topic` must be annotated with `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`.

Inserting the `Post` and the `Announcement` entities defined in the single table inheritance section generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Inheritance',
       'Best practices', 2)
```

```
INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Release x.y.z.Final',
       '2016-02-17 09:31:12.023', 3)
```

```
INSERT INTO topic_statistics (views, id) VALUES (1, 2)
```

```
INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

Unlike the joined table inheritance, each persisted subclass entity requires a single insert statement.

When fetching all `Topic` entities associated with a specific `Board`:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
    .setParameter("boardId", 1L)
    .getResultList();
```

Hibernate uses `UNION ALL` to fetch rows from the base class and every subclass table in this particular inheritance tree.

```
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS created02_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t.content AS content1_2_,
    t.validUntil AS validUnt1_0_,
    t.clazz_ AS clazz_
FROM (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id, content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content, validUntil, 2 AS clazz_
    FROM announcement ) t
WHERE t.board_id = 1
```

When loading a `TopicStatistics` while also fetching its associated `Topic` relation, Hibernate must use `UNION ALL` for the inheritance tables to construct the desired result set:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

#### SELECT

```
ts.id AS id1_4_0_, t.id AS id1_3_1_, ts.views AS views2_4_0_,
t.board_id AS board_id5_3_1_, t.createdOn AS created02_3_1_,
t.owner AS owner3_3_1_, t.title AS title4_3_1_, t.content AS content1_2_1_,
t.validUntil AS validUnt1_0_1_, t.clazz_ AS clazz_1_
FROM topic_statistics ts
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t ON ts.id = t.id
WHERE t.id = 2
```



The identity generator is not allowed with this strategy because rows belonging to different subclasses would share the same identifier, therefore conflicting in polymorphic `@ManyToOne` or `@OneToOne` associations.

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the `topics` collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
    .setParameter("id", id)
    .getSingleResult();
```

Hibernate requires a join with the result of unifying all three topic-related tables.

```
SELECT
    b.id AS id1_1_0_, t1.id AS id1_3_1_, b.name AS name2_1_0_,
    t1.board_id AS board_id5_3_1_, t1.createdOn AS created02_3_1_,
    t1.owner AS owner3_3_1_, t1.title AS title4_3_1_,
    t1.content AS content1_2_1_, t1.validUntil AS validUnt1_0_1_,
    t1.clazz_ AS clazz_1_, t1.board_id AS board_id5_3_0_, t1.id AS id1_3_0_
FROM board b
INNER JOIN (
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 0 AS clazz_
    FROM topic
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        content,
        CAST(NULL AS TIMESTAMP) AS validUntil, 1 AS clazz_
    FROM post
    UNION ALL
    SELECT id, createdOn, owner, title, board_id,
        CAST(NULL AS VARCHAR(100)) AS content,
        validUntil, 2 AS clazz_
    FROM announcement
) t1 ON b.id = t1.board_id
WHERE b.id = 1
```

### Performance considerations

While write operations are faster than for the joined table strategy, the read operations are only efficient when querying against the actual subclass entities. Polymorphic queries can have a considerable performance impact because Hibernate must select all subclass tables and use `UNION ALL` to build the whole inheritance tree result set. As a rule of thumb, the more subclass tables, the less efficient the polymorphic queries get.

## 11.4 Mapped superclass

If the `Topic` class is not required to be a stand-alone entity, it will be more practical to leave inheritance out of the database. This way, the `Topic` can be made abstract and marked with the `@MappedSuperclass` annotation so that JPA can acknowledge the inheritance model on the entity-side only.

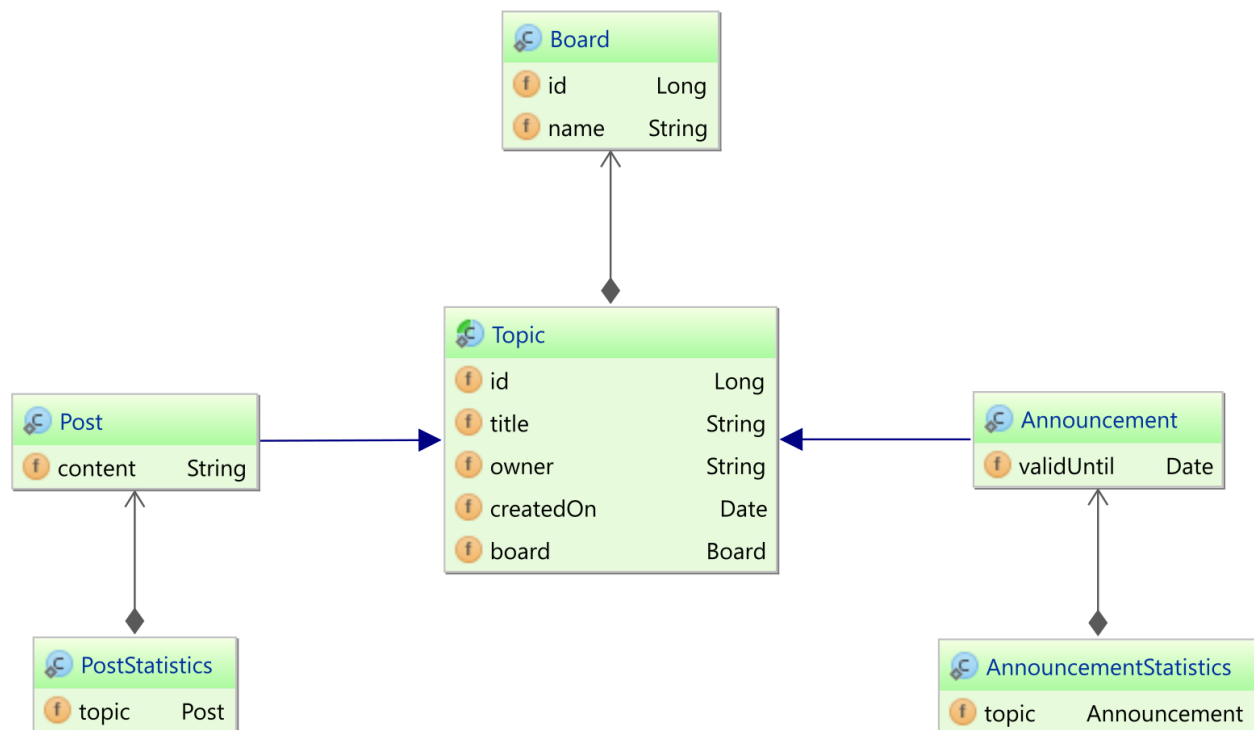


Figure 11.5: Mapped superclass class diagram

Having a single `TopicStatistics` entity with a `@OneToOne` `Topic` association is no longer possible because the `Topic` class is not an entity. This way, each `Topic` subclass must define its own statistics associations (e.g. `PostStatistics`, `AnnouncementStatistics`).

The `PostStatistics` and the `AnnouncementStatistics` entities looks as follows:

```
@Entity @Table(name = "post_statistics")
public class PostStatistics
    extends TopicStatistics<Post> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Post topic;

    private PostStatistics() {}

    public PostStatistics(Post topic) {
        this.topic = topic;
    }

    @Override
    public Post getTopic() {
        return topic;
    }
}

@Entity @Table(name = "announcement_statistics")
public class AnnouncementStatistics
    extends TopicStatistics<Announcement> {

    @OneToOne
    @JoinColumn(name = "id")
    @MapsId
    private Announcement topic;

    private AnnouncementStatistics() {}

    public AnnouncementStatistics(Announcement topic) {
        this.topic = topic;
    }

    @Override
    public Announcement getTopic() {
        return topic;
    }
}
```

To retain the inheritance semantics, the base class attributes are going to be merged with the subclass ones, so the associated subclass entity table contains both. This is similar to the table-per-class inheritance strategy, with the distinction that the base class is not mapped to a database table (hence, it cannot be used in polymorphic queries or associations).

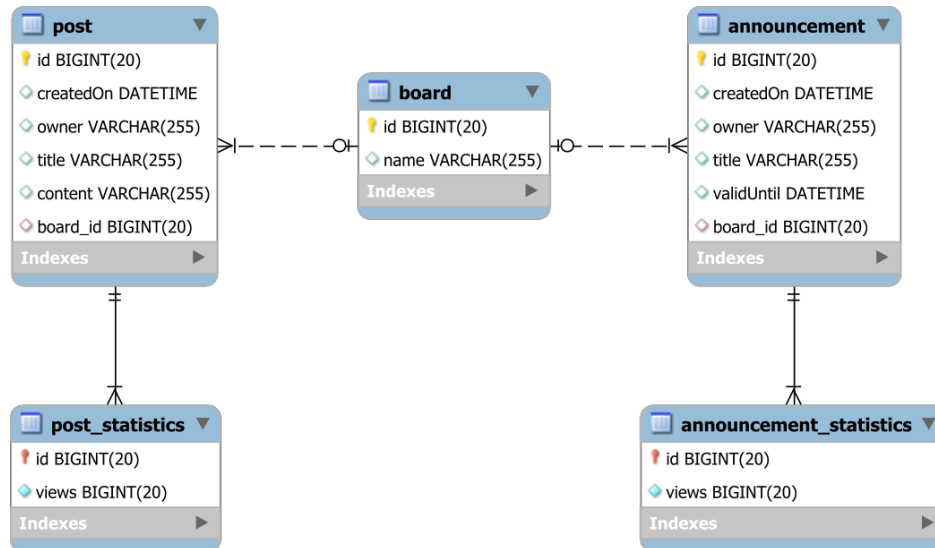


Figure 11.6: Mapped superclass

The `post` and the `announcement` tables feature columns that were inherited from the `Topic` base class. When persisting the same `Post` and `Announcement` entities and saving statistics using the newly defined entity classes:

```
TopicStatistics postStatistics = new PostStatistics(post);
postStatistics.incrementViews();
entityManager.persist(postStatistics);
```

```
TopicStatistics announcementStatistics =
    new AnnouncementStatistics(announcement);
announcementStatistics.incrementViews();
entityManager.persist(announcementStatistics);
```

Hibernate generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id )
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Inheritance',
       'Best practices', 1)
```

```
INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:11:07.525', 'John Doe', 'Release x.y.z.Final',
       '2016-02-17 09:11:07.529', 2)
```

```
INSERT INTO post_statistics (views, id) VALUES (1, 1)
```

```
INSERT INTO announcement_statistics (views, id) VALUES (1, 2)
```

Querying for statistics require specifying the actual `Topic` subclass statistics:

```
PostStatistics statistics = entityManager.createQuery(
    "select s from PostStatistics s join fetch s.topic t where t.id = :postId",
    PostStatistics.class)
.setParameter("postId", postId)
.getSingleResult();
```

This entity query generates a single SQL join statement:

```
SELECT
  s.id AS id1_4_0_,
  p.id AS id1_3_1_,
  s.views AS views2_4_0_,
  p.board_id AS board_id6_3_1_,
  p.createdOn AS created02_3_1_,
  p.owner AS owner3_3_1_,
  p.title AS title4_3_1_,
  p.content AS content5_3_1_
FROM post_statistics s
INNER JOIN post p ON s.id = p.id
WHERE p.id = 1
```

Polymorphic queries against the `Topic` class are not permitted and the `Board` entity cannot define a `@OneToMany Topic` collection either.

## Performance considerations

Although polymorphic queries and associations are no longer permitted, the `@MappedSuperclass` yields very efficient read and write operations. Like single and table-per-class inheritance, write operations require a single insert statement and only one table is used for reading data.

## Inheritance best practices

All the inheritance mapping models above require trading something in order to accommodate the impedance mismatch between the relational database system and the object-oriented Domain Model.

The default single table inheritance performs the best in terms of reading and writing data, but it forces the application developer to overcome the column nullability limitation. This strategy is useful when the database can provide support for trigger procedures, and the number of subclasses is relatively small.

The join table is worth considering when the number of subclasses is higher, and the data access layer does not require polymorphic queries. When the number of subclass tables is large, polymorphic queries require many joins, and fetching such a result set has a noticeable impact on application performance. This issue can be mitigated by restricting the result set (e.g. pagination), but that only applies to queries and not to `@OneToMany` or `@ManyToMany` associations. On the other hand, polymorphic `@ManyToOne` and `@OneToOne` associations are fine since, in spite of joining multiple tables, the result set can have at most one record only.

Table-per-class is the least effective when it comes to polymorphic queries or associations. If each subclass is stored in a separate database table, the `@MappedSuperclass` Domain Model inheritance might often be a better alternative.



Although a powerful concept, Domain Model inheritance should be used sparingly and only when the benefits supersede trade-offs.

## 12. Flushing

As explained in the [write-based optimizations](#) section, the Persistence Context acts as a transactional write-behind cache. The `Hibernate Session` is commonly referred to as the first-level cache since every managed entity is stored in a `Map`, and, once an entity is loaded, any successive request serves it from the cache, therefore avoiding a database roundtrip.

However, aside from caching entities, the Persistence Context acts as an entity state transition buffer. Both the `EntityManager` and the `Hibernate Session` expose various entity state management methods:

- The `persist` method takes a transient entity and makes it managed.
- The `merge` method copies the internal state of a detached entity onto a freshly loaded entity instance.
- Hibernate also supports reattaching entity instances (e.g. `update`, `saveOrUpdate` or `lock`) which, unlike `merge`, does not require fetching a new entity reference copy.
- To remove an entity, the `EntityManager` defines the `remove` method, while Hibernate offers a `delete` method.

Like any write-behind cache, the Persistence Context requires flushing in order to synchronize the in-memory persistent state with the underlying database. At flush time, Hibernate can detect if a managed entity has changed since it was loaded and trigger a table row update. This process is called dirty checking, and it greatly simplifies data access layer operations.

So, when using JPA, the application developer can focus on entity state changes, and the Persistence Context takes care of the underlying DML statements. This way, the data access layer logic is expressed using Domain Model state transitions, rather than through insert, update, or delete SQL statements.

This approach is very convenient for several reasons:

- Entity state changes being buffered, the Persistence Context can delay their execution, therefore minimizing the row-level lock interval, associated with every database write operation.
- Being executed at once, the Persistence Context can use JDBC batch updates to avoid executing each statement in a separate database roundtrip.

However, having an intermediate write-behind cache is not without challenges and the Persistence Context can be subject to data inconsistencies. Since efficiency is meaningless if effectiveness is being compromised, this chapter aims to analyze the inner-workings of the flushing mechanism, so the application developer knows how to optimize it without affecting data consistency.